ՌՈՒԶԱՆՆԱ ՍՏԵՓԱՆՅԱՆ, ԱՆՈՒՀ ԹՈՐՈՍՅԱՆ,
ՍԱՄՎԵԼ ՄԱՐՏԻՐՈՍՅԱՆ, ՀՈՒՀԱՆԻԿ ԱՄԻՐԽԱՆՅԱՆ,
ԼՈՒՍԻՆԵ ՄԱՏԻՆՅԱՆ

ԹՎԱՅԻՆ ԳՐԱԳԻՏՈՒԹՅՈՒՆ ԵՎ ՀԱՍԱԿԱՐԳՉԱՅԻՆ ԳԻՏՈՒԹՅՈՒՆ

11

Հանրակրթական ավագ դպրոցի
11–րդ դասարանի դասագիրք

ԶԱՆԳԱԿ
ՀՐԱՏԱՐԱԿՉՈՒԹՅՈՒՆ
ԵՐԵՎԱՆ • 2025

ԵՐԱՇԽԱՎՈՐՎԵԼ Է ՀՀ ԿՐԹՈՒԹՅԱՆ, ԳԻՏՈՒԹՅԱՆ, ՄՇԱԿՈՒՅԹԻ ԵՎ ՍՊՈՐՏԻ
ՆԱԽԱՐԱՐՈՒԹՅԱՆ ԿՈՂՄԻՑ

ՀՏԴ 000:000(000.0)
ԳՄԴ 0000գ00
Թ 000

Տպագրվել է ՀՀ ԿԳՄՍ նախարարության պատվերով

Թ 000 Թվային գրագիտություն և համակարգչային գիտություն 11: Հանրակրթական ավագ դպրոցի 11–րդ դասարանի դասագիրք / Ռ. Ստեփանյան և ուրիշներ.— եր.: «Զանգակ» հրատ., 2025.— 112 էջ:

ՀՏԴ 000:000(000.0)
ԳՄԴ 0000գ00

Դասագրքին կից տեսաձայնային նյութերը
կարող եք գտնել շապիկին պատկերված
QR կոդի միջոցով կամ հետևյալ հղումով.
www.zangak.am/9789939992839
Դասագրքի էջերում տեղադրված QR կոդերը
համապատասխան նյութերի անմիջական
հղումներն են:

ISBN 978–9939–99–283–9

© «Զանգակ–97» ՍՊԸ, 2025
© Ստեփանյան Ռ., 2025
© Թորոսյան Ա., 2025
© Մարտիրոսյան Ս., 2025
© Ամիրխանյան Շ., 2025
© Մատինյան Լ., 2025
© ՀՀ ԿԳՄՍՆ, 2025

ՆԱԽԱԲԱՆ

«Թվային գրագիտություն և համակարգչային գիտություն» առարկայի 11–րդ դասարանի այս դասագիրքը կօգնի զարգացնելու նախորդ տարիներին ձեռք բերած գիտելիքները և ուսումնասիրելու թվային տեխնոլոգիաների լայն հնարավորու–թյունները:

Այս դասագիրքը կդառնա ձեր ուղեցույցը թվային տեխնոլոգիաների ուսումնա–սիրության ճանապարհին՝ առաջարկելով նոր մարտահրավերներ և հետաքրքիր լուծումներ:

Դասագիրքը կառուցված է չորս հիմնական գաղափարների շուրջ.

Թվային գործիքներ և համագործակցություն

Ժամանակակից թվային աշխարհում համագործակցությունն ու արդյունավետ աշխատանքը կարևոր են յուրաքանչյուրի հաջողության համար: Դասագրքում ներառված տարբեր թվային գործիքները կհեշտացնեն ձեր աշխատանքը: Դուք կուսումնասիրեք, թե ինչպես կիրառել այդ գործիքները նախագծերի և թիմային աշխատանքների ընթացքում, որպեսզի կարողանաք արդյունավետ համագոր–ծակցել և հասնել մեծ հաջողությունների:

Հաշվողականություն և հասարակություն

Թվային տեխնոլոգիաների ազդեցությունը մեր կյանքի բոլոր ոլորտների վրա անհերքելի է, և կախված նրանից, թե որքան գիտակ և հմուտ կլինեք, կորոշվի այդ ազդեցության դրական կամ բացասական ուղղվածությունը:

Դասագրքի օգնությամբ կծանոթանաք թվային էթիկայի և անվտանգության հիմ–նախնդիրներին, ինչպես նաև այն կօգնի խորացնելու ձեր գիտելիքները էլեկտրո–նային անվտանգության, սարքերի և անձնական տվյալների պաշտպանության հար–ցում, կօգնի ճիշտ կողմնորոշվելու անընդհատ փոփոխվող և զարգացող թվային աշ–խարհում:

Հաշվողական համակարգեր

Հաշվողական համակարգերի հիմունքները հասկանալը կարևոր է յուրաքանչյուր ապագա ծրագրավորողի համար: Այս բաժնում դուք կծանոթանաք հաշվողական համակարգերի կառուցվածքին և գործառույթներին: Կուսումնասիրեք, թե ինչ սկզբունքով են աշխատում համակարգիչները, ինչպես են կառավարվում տվյալները, ինչ սարքավորումներ են կիրառվում ծրագրերի աշխատանքն ապահովելու համար:

Հաշվարկային և ալգորիթմական մտածողություն

Ալգորիթմական մտածողությունը խնդիրների լուծման տրամաբանական և համակարգված մոտեցում է: Այս բաժնում կսովորեք, թե ինչպես ստեղծել և կիրառել ալգորիթմներ տարբեր խնդիրների լուծման համար: Այդ նպատակով կուսումնասիրեք C++ ծրագրավորման լեզուն: C++–ը հզոր և հանրահայտ լեզու է, որը կիրառվում է բազմաթիվ ծրագրերի և հավելվածների ստեղծման համար: Կսովորեք գրել ծրագրեր, որոնք կօգնեն ավելի լավ հասկանալու ծրագրավորման հիմունքները և նորովի բացահայտելու ալգորիթմական մտածողությունը: Նաև կուսումնասիրեք հիմնական ալգորիթմները և դրանց կիրառման մեխանիզմները, ինչպես նաև կզարգացնեք ձեր տրամաբանական մտածողությունը և ծրագրավորման հմտությունները:

Այս դասագիրքը կօգնի ձեզ թվային տեխնոլոգիաների ոլորտում դառնալու առավել վստահ և հմուտ: Այն կտա գործիքներ, որոնք անհրաժեշտ են նորարարական լուծումներ գտնելու և ապագայի մարտահրավերներին պատրաստ լինելու համար:

Մաղթում ենք հետաքրքիր, արդյունավետ և օգտակար ուսումնական տարի:

ԹԵՄԱ 1 — ԷԼԵԿՏՐՈՆԱՅԻՆ ԱՊԱՀՈՎՈՒԹՅՈՒՆ

Ժամանակակից աշխարհում թվային վճարային համակարգերը դարձել են ամենօրյա կյանքի անբաժան մասը: Սակայն դրանց հարմարավետությունն ուղեկցվում է տարբեր վտանգներով: Ուստի, թվային ֆինանսների հետ անվտանգ աշխատելու համար անհրաժեշտ է ոչ միայն տեխնոլոգիական գրագիտություն, այլև քննադատական մտածողություն ու զգոնություն:

Ուսումնասիրելով այս թեման.

- Ձեռք կբերեք ձեր ֆինանսներն ու անձնական տվյալները պաշտպանելու կարևոր գիտելիքներ:
- Կսովորեք, թե ինչպես կարելի է խուսափել թվային խարդախություններից և կեղծ առաջարկներից:
- Կկարողանաք անվտանգ և պատասխանատու կերպով օգտվել բանկային քարտերից ու առցանց վճարային հարթակներից:
- Կիմանաք, թե ինչպես է պետք վարվել կասկածելի հաղորդագրությունների, հղումների և կայքերի դեպքում:

ԴԱՍ 1 — ԱՌՑԱՆՑ ՎՃԱՐԱՅԻՆ ՀԱՄԱԿԱՐԳԵՐ

Օրեցօր զարգացող թվային տեխնոլոգիաները մեր կյանքը դարձնում են ավելի ու ավելի հարմարավետ, գիտելիքի ձեռքբերումը՝ առավել հասանելի, շփումը՝ ավելի մատչելի և այլն: Մյուս կողմից՝ բոլորս հասկանում ենք, որ անընդհատ առաջանում են նոր վտանգներ. հաքերային հարձակումները, համացանցային զեղծարարությունները շատանում են և դառնում ավելի բազմազան, ավելի «խորամանկ»:

Կիբերհանցագործությունների մեծ մասը նպատակաուղղված է գումար շորթելուն, հափշտակելուն: Քանի որ այսօր ավագ դպրոցականներն արդեն կարողանում են գումար վաստակել, մոտ ապագայում կկարողանան տնօրինել սեփական միջոցները՝ օգտվելով էլեկտրոնային դրամապանակներից, բանկային հաշիվներից, կատարելով առցանց վճարումներ: Շատ կարևոր է, որ յուրաքանչյուրը հստակ իմանա, թե ինչպես պաշտպանի սեփական գումարները թվային վտանգներից:

Մարդկանց կամ կազմակերպությունների ֆինանսների հափշտակմանն ուղղված կիբերհարձակումները հիմնականում կատարվում են մի քանի ուղղությամբ.

- Բացահայտ խաբեություն, երբ զեղծարարները մարդուն համոզում են իրենց տրամադրել բանկային քարտի տվյալները, գաղտնաբառերը, նույնիսկ գումարը:
- Տեխնիկական բնույթի հարձակումներ, երբ հատուկ մնասակար ծրագրերով վարակվում է հեռախոսը կամ համակարգիչը, որը կիբերհանցագործներին հնարավորություն է տալիս կորզելու բանկային քարտի տվյալները կամ բանկային հաշվի գաղտնաբառը:

Նման հարձակումներից պաշտպանվելու համար եկե՛ք նախ պարզենք, թե ինչ տվյալներ կան թվային տիրույթում, որոնք կարող են մարդու ֆինանսների ապահովության համար խոցելի լինել:

Նախ հասկանանք, թե ֆինանսների հետ աշխատող ինչ հարթակներ և համակարգեր կան:

Առաջինը ավանդական բանկերն են, որոնք աշխատում են բանկային քարտերի և բանկային առցանց հավելվածների միջոցով:

Քարտը հնարավորություն է տալիս անկանխիկ վճարումներ իրականացնելու ինչպես ավանդական և առցանց խանութներում, այնպես էլ ցանկացած վայրում որտեղ ընդունվում են բանկային քարտերը:

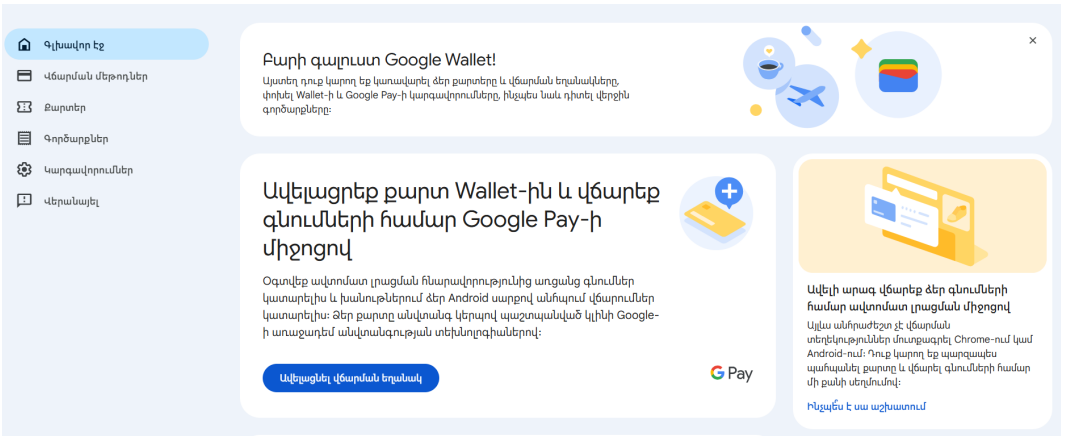
Բանկային առցանց հավելվածը հնարավորություն է տալիս գործարքներ իրականացնելու հեռավար՝ համացանցային կապի միջոցով: Դուք կարող եք փոխանցումներ կատարել ձեր հաշիվների միջև, գումար ուղարկել մեկ ուրիշի, գնումներ և վճարումներ կատարել: Նաև կարող եք գումար փոխանցել այլ բանկային քարտի, և փոխանցված գումարը անմիջապես հասանելի կլինի քարտապանին: Նույնիսկ հնարավոր է գումար փոխանցել՝ իմանալով միայն հասցեատիրոջ հեռախոսահամարը: Բանկային հավելվածներն ունեն բազմաթիվ հնարավորություններ, որոնք հեշտացնում են մեր կյանքը, օրինակ՝ մեկ հպումով կարողանում ենք վճարել կոմունալ բոլոր ծառայությունների դիմաց: Կարելի է առցանց ձևակերպել վարկ, այսինքն՝ բանկից տոկոսով գումար վերցնել և վճարել պարտքը բանկի կողմից սահմանված ամիսների կամ տարիների ընթացքում:

Բացի նշված համակարգերից՝ գոյություն ունեն թվային դրամապանակներ, որոնք թույլ են տալիս գումարը ծախսել հեռախոսի միջոցով՝ բառացիորեն մեկ հպումով: Սա ևս հեշտացնում է կյանքը, քանի որ երբեմն չունենալով գրպանում կանխիկ գումար, բանկային քարտ՝ հեռախոսի մեջ առկա էլեկտրոնային դրամապանակի միջոցով հնարավոր է կատարել գրեթե կամայական գործարք: Հայաստանում գործում են մի շարք թվային դրամապանակներ:

Կան նաև զանազան միջազգային թվային հարթակներ, որոնք թույլ են տալիս տնօրինել գումարները: Ասպես, օրինակ՝ Google Pay, Apple Pay գործիքները հնարավորություն են տալիս հեռախոսի կամ խելացի ժամացույցի, այսօր նույնիսկ խելացի մատանու միջոցով մեկ տեղում հավաքելու բոլոր բանկային քարտերը և իրականացնելու վճարումներ թե՛ առցանց, թե՛ ավանդական խանութներում, օդանավակայաններում, կայարաններում և ամենուր: Փաստացիորեն սրանք ևս թվային դրամապանակներ են, որոնք գործում են միջազգային մակարդակով:

Google Pay թվային դրամապանակ

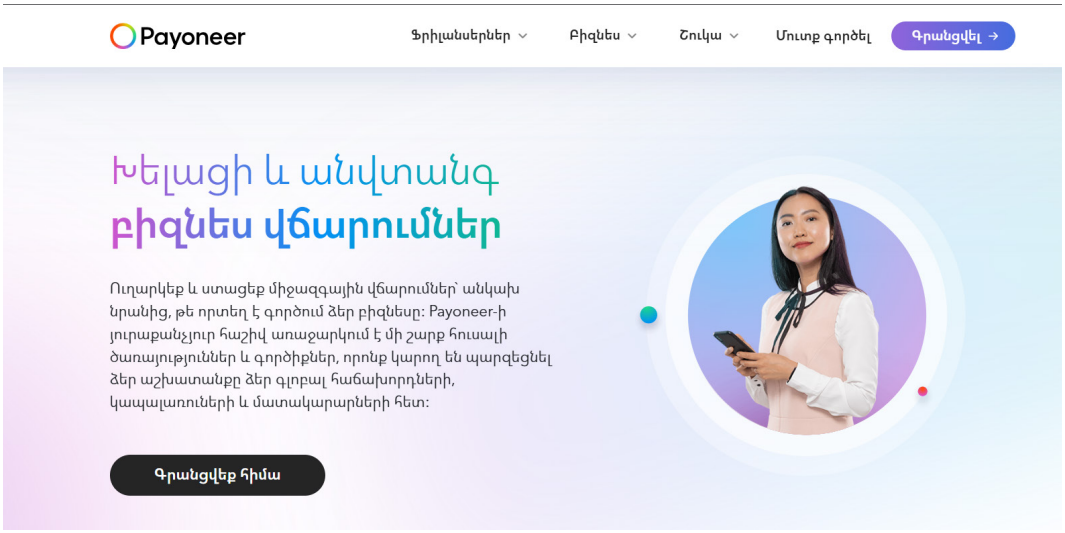
Թվային դրամապանակներն ունեն մեկ առավելություն. եթե առցանց վճարելիս դուք պարտավոր չեք տրամադրել ձեր բանկային քարտերը, ապա տվյալ դեպքում դրա կարիքը չկա, ընդամենը անհրաժեշտ է ձեր բանկային քարտերը կցել Google Pay, Apple Pay կամ այլ թվային դրամապանակի և արդեն վճարումները կատարել դրանց միջոցով: Դրամապանակում բանկային քարտի տվյալները մնում են քողարկված, որն բարձրացնում է պաշտպանվածության մակարդակը: Իհարկե, նման դեպքերում միշտ պետք է հիշել թվային անվտանգության կանոնները և պաշտպանել Google, iCloud հաշիվները, որոնց կցված են Google Pay, Apple Pay վճարային գործիքները:



Google pay վճարային գործիք

Միջազգային արագ փոխանցումներ իրականացնելու, դրանք հեշտացնելու, թղթային քաշքշուկներից խուսափելու համար ստեղծվել են և գործում են մի շարք ֆինանսական միջազգային թվային հարթակներ: *Օրինակ՝* PayPal, Payoneer, Revolut և այլն: Սրանք հարթակներ են, որոնք առաջարկում են կախված չլինել որևէ երկրի բանկային համակարգից: Նման հարթակներից հաճախ օգտվում են նրանք, որոնք աշխատում են հեռավար, այլ երկների ընկերությունների համար և ուզում են աշխատավարձ ստանալ՝ կախված չլինելով այդ երկրի բանկերից, ստիպված չլինելով երկար գրանցումներ կատարել և այլն:

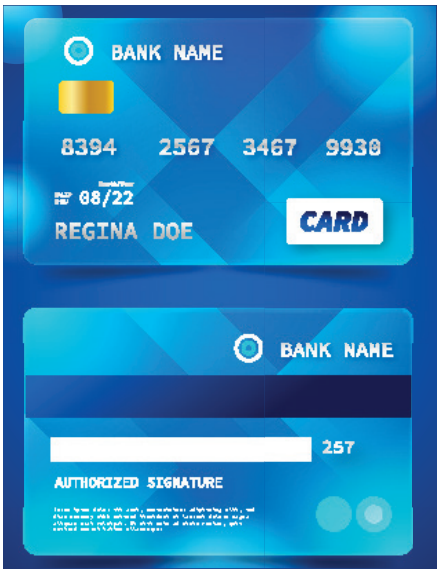
Payoneer թվային ֆինանսական միջազգային հարթակ



Դուք տեսաք, թե միայն ավանդական դրամի հետ աշխատող ինչ տարբեր հարթակներ կան: Սրանց ավելացնենք կրիպտոարժույթով աշխատող զանազան հարթակները: Դրանք որքան հեշտացնում են գումարի հետ մեր աշխատանքը, զանազան վճարումները, դրամի շրջանառությունը, այնքան հրապուրիչ են դառնում կիբերհանցագործների համար, որոնք անընդհատ փնտրում են հնարավորություններ օգտվելու թվային գումարների դյուրին շարժից:

Եվ այսպես՝ որո՞նք են նոր ժամանակների թվային ֆինանսների խոցելի հանգույցները:

Բանկային քարտ: Միանգամից հասկանանք մի կարևոր բան. բանկային քարտի վրայի բոլոր տվյալները պետք է գաղտնի պահել: Միայն այն դեպքում, երբ անհրաժեշտ է բանկային քարտի փոխանցում կատարել, կարելի է փոխանցել քարտի վրա դաջված 16–նիշանոց թիվը, միայն ու միայն դա, և գերադասելի է՝ միայն վստահելի ծանոթ մարդու:



Բանկային քարտի տվյալները ներմուծվում են օնլայն հարթակներ, եթե դուք պետք է գնում կամ ծառայության դիմաց վճարում կատարեք: Ու դա հարկավոր է անել միայն վստահելի, ստուգված հարթակներում, քանի որ եթե զեղծարարների մոտ հայտնվեն քարտի տվյալները, նրանք կկարողանան ձեր անունից ինքնուրույն գործարքներ իրականացնել կամ պարզապես գումար գողանալ:

Եվ այսպես՝ քարտի վրա գրված կարևոր տվյալներն են.

- քարտապանի անունը,
- քարտի վրայի 16–նիշանոց թիվը՝ բուն քարտի համարը,
- քարտի վավերականության ժամկետը,
- քարտի հակառակ կողմում նշված եռանիշ կամ քառանիշ թիվը՝ CVV կոդը:

Նույնիսկ առաջին երեք տվյալներն իմանալով՝ հնարավոր է որոշ գործարքներ իրականացնել այդ քարտով, իսկ բոլոր չորս տվյալներն ունենալու դեպքում հանցագործները կկարողանան ամբողջությամբ զրկել ձեզ գումարից, որոշ դեպքերում, եթե քարտը թույլ է տալիս, անգամ պարտքով գումար շորթել:

Ինչպե՞ս պաշտպանվել քարտի տվյալների արտահոսքից: Ներկայացնենք մի քանի կարևոր քայլեր, որոնք թույլ կտան այդ խնդիրը լուծել:

Աշխատե՛ք քարտի տվյալները չտրամադրել այնպիսի հարթակների, կայքերի, որոնց իրական լինելը չի ստուգվել նախապես: Պարզապես մի՛ շտապեք ամենուր տեղադրել քարտի տվյալները: Հարցուփո՛րձ արեք, համացանցում, որոնե՛ք կարծիքներ այդ կայքի մասին, հասկացե՛ք, թե ուրիշները դրա հետ կապված ինչ փորձ ունեն:

Օգտվե՛ք օնլայն բանքինգի՝ առցանց բանկային հավելվածի հնարավորություններից:

Առաջին՝ պահե՛ք հատուկ քարտ միայն օնլայն գործարքների համար: Այսօր նաև հնարավոր է ստեղծել վիրտուալ քարտ, որը ֆիզիկապես գոյություն չունի, բայց դրանով հնարավոր է ինտերնետային գրծարքներ իրականացնել:

Երկրորդ՝ տվյալ քարտում պահե՛ք այնքան գումար, որքան անհրաժեշտ է տվյալ գործարքն իրականացնելու համար: Ավարտելուց հետո մնացորդը օնլայ ռեժիմով փոխանցե՛ք ձեր բանկային այլ հաշվի, որին ոչ մի քարտ կցված չէ:

Մյուս հնարավոր վտանգը գալիս է գաղտնաբառերի արտահոսքից: Այսպես, *օրինակ*՝ տարբեր ֆինանսական հաստատություններ՝ բանկեր, վարկային կամակերպություններ առաջարկում են արագ վարկ ստանալու տարբերակներ, որը հաճախ իրականացվում է SMS–ով՝ կարճ հաղորդագրությունով եկած մեկանգամյա գաղտնաբառի, կոդի հաստատման միջոցով: Ջեղծարարները կարող են զանգել կամ գրել կապի տարբեր միջոցներով, ներկայանալ որպես բանկի ներկայացուցիչ և տարբեր պատրվակներով պահանջել անհրաժեշտ կոդը կամ այլ գաղտնի տվյալներ, *օրինակ*՝ բանկային հաշվի գաղտնաբառը, բանկային քարտի տվյալները և այլն:

Կարևոր է հիշել.

Իրական բանկի կամ նման այլ կազմակերպության ներկայացուցիչը երբեք հեռախոսով կամ այլ հեռավար միջոցով ձեզնից չի հարցնի գաղտնաբառ կամ այլ գաղտնի տվյալ:

Եվ, իհարկե, հարկավոր է պահպանել թվային անվտանգության ձեզ հայտնի բոլոր կանոնները, հետևել սարքերի և հաշիվների անվտանգությանը, քանի որ, օրինակ, եթե ձեր համակարգչում կամ հեռախոսում հայտնվի վնասակար ծրագիր՝ վիրուս, ապա այն կկարողանա հեշտությամբ գողանալ բանկային քարտի, բանկային հաշվի տվյալները և փոխանցել հանցագործներին:

Մենք խոսեցինք հիմնականում տեխնիկական պաշտպանության մասին: Բայց ֆինանսական ոլորտում առավել կարևոր է նաև հոգեբանական բաղադրիչը: Մարդ կարող է ամբողջությամբ տեխնիկապես պաշտպանել իր սարքերը, հաշիվները, բայց հեշտությամբ հավատա, որ հեռավոր մի երկրի թագավոր մահացել է և թողել իրեն 500 միլիոն դոլարի ժառանգություն: Եվ այստեղ շատ կարևոր է հիշել համացանցով բոլոր ֆինանսական առաջարկների հետ կապված կանոնը.

Եթե համացանցով ստացել եք շատ հրապուրիչ, գայթակղիչ առաջարկ, ապա խոր շո՛ւնչ քաշեք, հանգստացե՛ք, սառը գլխով փորձե՛ք հասկանալ, թե որքանով է այն իրական: Եթե ինքներդ լավ չեք հասկանում, անպայման խորհո՛ւրդ հարցրեք լավատեղյակ մարդուց: Երբեք մի՛ շտապեք:

Այս ամենը կարդալիս ի՞նչն է կարևոր պահել հիշողության մեջ: Այն փաստը, որ այս պահին դպրոցականները քայլ առ քայլ մտնում են կյանք, հնարավորություն ստանում օգտվելու նաև զանազան ծառայություններից, ֆինանսական ոլորտը զարգանում է աննախադեպ, հեղափոխական քայլերով: Երբևէ վճարային միջոցները չեն

փոխվել նման արագությամբ: Եթե դեռ դարեր առաջ սերունդների փոփոխությունը չէր բերում վճարային համակարգի էական փոփոխության, ապա այսօր իրավիճակը հիմնովին տարբերվում է:

Ամփոփում

Տեխնոլոգիական առաջընթացը նպաստել նաև վճարային համակարգերի զարգացմանն ու թվայնացմանը, ինչն էլ իր հերթին ստեղծել է նոր ռիսկեր: Այսօր մարդիկ, ներառյալ դպրոցականները, օգտագործում են տարբեր թվային գործիքներ՝ գումար վաստակելու, փոխանցելու, ծախսելու համար: Սակայն աճել են նաև կիբեռհանցագործությունները: Անվտանգության համար անհրաժեշտ է պաշտպանել բանկային քարտի տվյալները, օգտագործել վիրտուալ կամ առանձին քարտեր, զգուշանալ խաբեություններից և չտրամադրել գաղտնի տեղեկություններ: Բացի տեխնիկական պաշտպանության միջոցներից կարևոր է նաև խուսափել հոգեբանական ազդեցություններից: Ներկայիս արագ զարգացող թվային աշխարհում կարևոր է մշտապես տեղեկացված լինել և խելամիտ օգտվել ֆինանսական նորարարություններից:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ վճարային համակարգեր են ձևավորվել համացանցային ժամանակաշրջանում:
2. Բանկային քարտի ո՞ր տվյալներն են կարևոր:
3. Ինչպե՞ս է անհրաժեշտ պաշտպանվել քարտային տվյալների կորստից:
4. Տանը բոլոր մեծահասակների հետ անցկացրե՛ք փոքրիկ հանդիպում առցանց վճարային համակարգերի մասին և համոզվե՛ք, որ բոլորը հետևում են անվտանգության հիմնական կանոններին:

ԹԵՄԱ 2

ԷԼԵԿՏՐՈՆԱՅԻՆ ՀԱՂՈՐԴԱԿՑՈՒԹՅՈՒՆ

Ժամանակակից աշխարհում էլեկտրոնային հաղորդակցությունն ու սոցիալական ցանցերը դարձել են անբաժանելի մաս ինչպես առօրյայի, այնպես էլ անձի թվային ներկայության ձևավորման մեջ: Թվային հարթակներում յուրաքանչյուր գրառում, լուսանկար կամ մեկնաբանություն ստեղծում է մարդու թվային կենսագրությունը, որը կարող է ազդեցություն ունենալ նրա սոցիալական ու մասնագիտական վարկանիշի վրա: Ուստի անհրաժեշտ է ոչ միայն ակտիվորեն մասնակցել թվային միջավայրին, այլև՝ դա անել գիտակցված, պատասխանատու և նպատակաուղղված ձևով:

Ուսումնասիրելով այս թեման՝

- Կիմանաք սոցիալական ցանցերից օգտվելու առավելությունները և թերությունները:
- Կկարողանաք ձևավորել ձեր թվային կենսագրությունը՝ հստակ ու վստահելի կերպով:
- Կսովորեք, թե ինչպես ներկայացնել ինքներդ ձեզ բլոգների, փողքասթների և տեսաբլոգների միջոցով:
- Կկարողանաք ստեղծել գիտակցված և համաչափ թվային հետք՝ արժեհամակարգի, բարոյականության և անվտանգության սկզբունքների հիման վրա:

ԴԱՍ 2

ԷԼԵԿՏՐՈՆԱՅԻՆ ՀԱՂՈՐԴԱԿՑՈՒԹՅՈՒՆ: ԹՎԱՅԻՆ ԿԵՆՍԱԳՐՈՒԹՅԱՆ ԶԵՎԱՎՈՐՈՒՄ

Այսօր դժվար է գտնել որևէ 11–րդ դասարանցու, որը չի օգտվում գոնե մեկ սոցիալական ցանցից կամ զրուցարանից: Սա համատարած երևույթ է՝ իր դրական և բացասական ազդեցություններով: Փորձենք գտնել մեզ համար օգտակար գործիքները, զերծ մնալ այն ամենից, ինչը խոչընդոտում է մեր ինքնակրթմանն ու զարգացմանը:

Սոցիալական ցանցերը շատերն օգտագործում են ոչ նպատակային, պարզապես անիմաստ ժամանակ են վատնում: Մյուսները համարում են դրանք մեծագույն չարիք և հնարավորինս խուսափում հաճախակի օգտագործումից:

Եկե՛ք փորձենք չխուսափել սոցիալական ցանցերից և համացանցային այլ հարթակներից, հետ չմնանք ժամանակի զարգացումներից, բայց աշխատենք նպաստավոր, լավ գործերին նվիրենք այդ ժամանակակից գործիքները:

Սովորաբար սոցցանցերում տասնմեկերորդ դասարանցիներից շատերն ունեն իրենց օգտահաշիվները: Մեծ մասն այն օգտագործում է միայն ընկերների հետ շփվելու, ուրիշներին հետևելու կամ պարզապես տեսանյութեր դիտելու համար: Եվ որքան երկար ես օգտվում սոցիալական ցանցերից, այնքան մեծ հետք ես թողնում, ավելի շատ նյութեր են կուտակվում քո էջերում: Հաճախ դա արվում է անգիտակցաբար: Բայց անկախ նրանից՝ օգտատերը շատ է ակտիվ սոցցանցերում, թե չափավոր, միևնույնն է, տեղեկույթը երկարող պոչի պես անընդհատ կուտակվում է համացանցում: Կարևոր է նաև հասկանալ, որ հաճախ այդ տեղեկույթի մի մասը չի վերանում, եթե անգամ օգտատերը ջնջում է իր սոցիալական ցանցերի օգտահաշիվները: Դրանով է բացատրվում նաև այն հանգամանքը, երբ որոշ օգտատերեր, չցանկանալով հրապարակել ինչ–որ տեղեկույթ, այն պահպանում են իրենց անձնական տիրույթում, սակայն որոշ ժամանակ անց այդ գրառումը, այնուամենայնիվ, հայտնվում է որևէ լրատվամիջոցում կամ հարթակում:

Այսպիսով՝ հիշենք, որ կամա թե ակամա մենք ստեղծում ենք մեր վիրտուալ պատկերը սոցցանցերում և մյուս օգտատերերի կողմից սկսում ենք ընկալվել այնպիսին, ինչպիսին մեզ ներկայացնում ենք համացացում: Տարիների ընթացքում ձևավորում ենք մեր վիրտուալ կերպարը և ստեղծում մեր ցանցային թվային կենսագրությունը: Սա շատ կարևոր է գիտակցել, քանի որ ակամայից կարող եք այնպիսի թվային հետքեր թողնել, սեփական անձի մասին այնպիսի պատկեր ձևավորել, որ անգամ տարիների ընթացքում չկարողանաք ազատվել դրանից:

Ինչո՞ւ է կարևոր ճիշտ, գիտակցված, վերահսկելի թվային կենսագրություն ձևավորելը, ճիշտ թվային հետքեր թողնելը: Նախ դա կարևոր է հասարակական դիրք ձևավորելու համար, այսինքն՝ թե ինչպես անհատին կընկալեն ոչ միայն իր մտերիմ ընկերները, այլև նրանք, ովքեր հեռակա են ծանոթ կամ նոր են ուզում ծանոթանալ: Իսկ այսօր ծանոթությունների բավականին մեծ մասը ստեղծվում է համացանցում:

Մյուս կարևոր հանգամանքն այն է, որ հետագայում մեծ աշխարհ մտնելիս, աշխատանք փնտրելիս կամ սեփական բիզնեսը ստեղծելիս թվային կենսագրությունը, համացանցում ձևավորված պատկերը կարևոր է դառնում այլոց կողմից որոշումներ կայացնելու համար: *Օրինակ*՝ աշխատանքի ընդունելիս այսօր գործատուն հաճախ մանրամասն ուսումնասիրում է մարդու թվային կենսագրությունը, հենց սոցցանցերի միջոցով է փորձում ձևավորել կարծիք ապագա աշխատակցի մասին: Իհարկե, տասնմեկերորդ դասարանցիների մտքով հնարավոր է չանցնի, բայց չի բացառվում, որ տարիներ անց ձեզնից որոշները կդառնան հանրային կամ քաղաքական գործիչներ, մյուսները կանցնեն պետական աշխատանքի, կդառնան նույնիսկ նախարար–

ներ: Եվ, իհարկե, նման դեպքերում շատերը մանրազնին կսկսեն ուսումնասիրել սոցիալական ցանցերը, որպեսզի հասկանան, թե ինչպիսի մարդ եք դուք, ինչ խոցելի կետեր ունեք:

Ինչո՞ւ է կարևոր ճիշտ ձևավորել թվային կենսագրությունը, ճիշտ հետքեր թողնել համացանցում: Եթե ամփոփենք, ապա թվային կենսագրություն ստեղծելու կարևորության վերաբերյալ կարող ենք նշել մի քանի կետ.

1. Ձեր ինքնության ներկայացումը: Թվային կենսագրությունը թույլ է տալիս ներկայացնել ձեր անձնական և մասնագիտական փորձը, ձեր ճաշակը, ձեր հմտությունները և հետաքրքրությունները, որը թույլ է տալիս ավելի լավ պատկերացում կազմել ձեր կարողությունների մասին:

2. Այսպես կոչված «անձնական բրենդինգ»: Սա կարևոր գործիք է, որը կարող է օգնել ձևավորելու ձեր անձնական կերպարը՝ ցուցադրելով ձեր արժանիքներն ու հաջողությունները, հնարավորություն տալով կարճ ժամանակում սեղմ, համառոտ պատկերացում կազմելու ձեր մասին:

3. Ճանաչելիության բարձրացում: Թվային կենսագրությունն օգնում է դառնալու ավելի ճանաչելի ու վստահելի:

4. Հնարավորությունների ստեղծում: Երբ թվային կենսագրությունը հասանելի է համացանցում, այն կարող է նպաստել նոր կապերի, առաջարկների ու համագործակցությունների հնարավորությունների ստեղծմանը:

5. Վստահության ստեղծում: Հստակ և ամբողջական թվային կենսագրությունը կարող է վստահություն ձևավորել՝ ցույց տալով ձեր հմտությունները, փորձը և ձեռքբերումները, որը կարևոր է ցանկացած, հատկապես մասնագիտական հարաբերությունների համար:

Եվ, անկասկած, հարց է ծագում, թե ինչպես ճիշտ ձևավորել թվային կենսագրությունը, ինչի վրա ուշադրություն դարձնել:

Ահա թվային կենսագրությունը պահպանելու մի քանի արդյունավետ մեթոդ.

1. Կանոնավոր թարմացումներ: Շարունակաբար թարմացրե՛ք ձեր թվային կենսագրությունը, որպեսզի այն արտացոլի ձեր անձը ներկա իրավիճակում:

2. Պարբերաբար վերանայում: ժամանակ առ ժամանակ հայա՛ցք նետեք ձեր սոցցանցային էջերին, փորձե՛ք ուրիշի աչքերով նայել դրանց:

3. Լուսանկարների թարմացում: Ավելացրե՛ք բարձրորակ լուսանկարներ, ինֆոգրաֆիկաներ կամ տեսանյութեր, որոնք տեսողական կբարձրացնեն կենսագրության որակն ու տպավորությունը:

4. Հղումների ներառում: Տարիների ընթացքում որքան երկար է մարդը ստեղծագործում, այնքան ավելի շատ է ներգրավվում բազմաթիվ աշխատանքների, նախագծերի մեջ, գրում հոդվածներ, տալիս հարցազրույցներ և այլն: Այս ամենը անհրաժեշտ է հնարավորության դեպքում ներկայացնել ձեր հարթակներում: Նաև կարևոր է հղում տալ ուրիշների հետաքրքիր, որակյալ աշխատանքներին:

Նաև պետք է հիշել, որ ցանկացած տեսանյութ, լուսանկար, գրառում տարածելիս պետք է չմոռանալ նշել սկզբնաղբյուրը, իրական հեղինակին:

Կարևոր է նաև, որ բոլոր գրառումները, տեսանյութերը, լուսանկարները, որոնք դուք տեղադրում եք ձեր էջերում, բավարարեն բարոյականության նորմերին, լինեն խելացի, հետաքրքիր, ճաշակով և գեղեցիկ: Իհարկե, սա վերաբերում է մարդու կամայական դրսևորմանը, սակայն թվային կենսագրությունը ևս մարդու բնութագրի կարևոր հատվածն է:

Գոյություն ունեն նաև տեխնիական պայմաններ.

1. Գրեթե բոլոր սոցցանցերում գործում են այսպես կոչված ալգորիթմներ: Դրանք արհեստական բանականության վրա հիմնված համակարգեր են, որոնք որոշում են, թե որ գրառումը, լուսանկարը, տեսանյութը որքան շատ կամ քիչ մարդ է տեսնելու, ով է տեսնելու և երբ: Ալգորիթմները պարբերաբար փոփոխվում են, թարմանում: Ու հարթակից հարթակ դրանք տարբերվում են: Պետք է ուսումնասիրել դրանց յուրահատկությունները՝ հասկանալու համար, թե ինչպես կարող եք առավել արդյունավետ ներկայացնել ձեր գրառումները:
2. Տարբեր հարթակներ ունեն տարբեր լսարաններ: *Օրինակ*՝ Հայաստանում երիտասարդությունն ավելի շատ օգտվում է Instagram, Snapchat հարթակներից, միջին և բարձր տարիքի մարդիկ ավելի շատ՝ Facebook–ից, իսկ LinkedIn ցանցն ավելի մասնագիտական ուղղվածություն ունի: Բայց կարող են հայտնվել նոր հարթակներ, լսարանները կարող են փոխվել:

Այստեղից կարող ենք եզրակացնել, որ անհրաժեշտ է անընդհատ ուսումնասիրելու հետևել փոփոխություններին՝ տեխնիկական և ընդհանուր զարգացումներին համընթաց շարժվելու համար:

Եվ, իհարկե, կրկին ու կրկին **հիշե՛ք**.

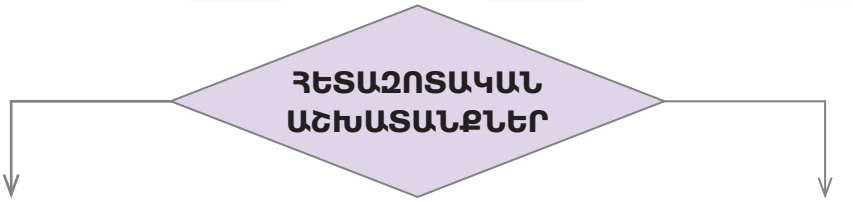
Պետք է պարտադիր հետևել թվային անվտանգության բոլոր կանոններին, որպեսզի տարիների ընթացքում ձեր հարթակներում կուտակված տեղեկույթը, ձևավորված թվային կենսագրությունը մեկ ակնթարթում չփոշիանան:

ԱՄՓՈՓՈՒՄ

Էլեկտրոնային հաղորդակցությունն այսօր ոչ միայն շփման միջոց է, այլև անձի թվային ինքնության հիմնական ձևավորման գործիք: Յուրաքանչյուր օգտատեր, գիտակցելով կամ ոչ, իր սոցիալական ցանցերում և առցանց հարթակներում ստեղծում է թվային կենսագրություն, որը կարող է երկարաժամկետ ազդեցություն ունենալ հասարակական ընկալման, մասնագիտական հնարավորությունների և նույնիսկ աշխատանքի տեղավորման վրա: Ուստի կարևոր է, որ բոլորը գիտակցեն թվային ներկայության կարևորությունը, ստեղծեն հավասարակշռված և վստահելի թվային կերպար՝ կիրառելով հմտություններ, ինչպես օրինակ՝ տեղեկատվության վերահսկում, նպատակային բովանդակության ստեղծում և թվային անվտանգության պահպանում: Այսպիսով, թվային գրագիտությունը դառնում է ոչ միայն տեխնիկական կարողություն, այլ՝ անձնական զարգացման ռազմավարություն:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ինչո՞ւ է կարևոր թվային կենսագրության ձևավորումը:
2. Որո՞նք են թվային կենսագրության ձևավորման հիմնական տեխնիկական պայմանները:
3. Ընդհանուր առմամբ ինչպիսի՞ն պետք է լինի սոցիալական ցանցերի բովանդակությունը:
4. Կազմակերպեք բանավեճ հետևյալ թեմայով. «Անվտանգության նպատակով պետք է արդյո՞ք, որ պետությունը վերահսկի մեր առցանց գործունեությունը, թե՞ դա խախտում է անձնական ազատությունը»:



➲ Գտե՛ք երկու տարբեր ոլորտներ ներկայացնող հայազգի հանրային դեմքերի սոցիալական ցանցերը: Վերլուծե՛ք, թե ինչպես են սոցիալական ցանցերում հրապարակված անձնական տվյալները, լուսանկարները, մեկնաբանությունները և օնլայն ակտիվությունը ձևավորում նրանց թվային կենսագրականը: Պարզեք, թե նրանց թվային կենսագրականը ինչպես է անդրադառնում նրանց մասնագիտական գործունեության վրա: Ինչպե՞ս են նրանք օգտագործում իրենց հայտնիությունը ազգային դիրքորոշման տեսանկյունից:

Հետազոտական հարցեր.

– Ի՞նչն է ամենաշատ տեսանելի նրանց թվային կենսագրականներում:
– Ի՞նչ հետևանքներ կարող է ունենալ անզգույշ հրապարակումը:
– Կարո՞ղ է թվային կենսագրականը ազդել ուսման, աշխատանքի կամ հարաբերությունների վրա:
– Ի՞նչ միջոցներով է հնարավոր ձևավորել անվտանգ, գրագետ և դրական թվային կենսագրություն:
– Ինչպե՞ս է հնարավոր օգտագործել թվային կենսագրությունը ազգայինը առաջ տանելու հարցում:

Հետազոտական աշխատանքի գնահատման ռուբրիկ

N	Չափանիշ	Նկարագրություն	Միավորը
1	Թեմայի ըմբռնում և ներկայացում	Հստակ բացատրված է, թե ինչ է թվային կենսագրականը, ինչպիսի տվյալներ է այն պարունակում: Ուսումնասիրությունը համապատասխանում է առաջադրանքին:	2 միավոր
2	Տվյալների հավաքագրում և վերլուծություն	Վերլուծվել են սոցիալական ցանցերի տվյալները (ինքնակենսագրություն, նկարներ, գրառումներ), ներկայացված են կոնկրետ օրինակներ կամ դիտարկումներ:	2.5 միավոր
3	Ռիսկերի և ազդեցության գնահատում	Ներկայացված են հնարավոր ռիսկերը, խնդիրները, ազդեցությունը անձնական և հասարակական կյանքում:	2 միավոր
4	Առաջարկների ձևավորում և եզրակացություն	Ներկայացված են թվային ինքնությունը գրագետ ներկայացնելու կամ պաշտպանելու առաջարկներ: Կա հստակ եզրակացություն:	2 միավոր
5	Ձևավորման և շարադրանք	Աշխատանքը կազմված է կոկիկ, լեզվական և կառուցվածքային առումով ճշգրիտ: Կարող է ներառել գրաֆիկ, աղյուսակ կամ պատկերներ:	1.5 միավոր

ԹԵՄԱ 3 → ՏԵՂԵԿԱՏՎԱԿԱՆ ՏԵԽՆՈԼՈԳԻԱՆԵՐԸ ԵՎ ՀԱՍԱՐԱԿՈՒԹՅՈՒՆԸ

Համակարգչային ծրագրերը մեր օրերում անչափ կարևոր դեր ունեն բոլոր ոլորտներում՝ տնտեսություն, բժշկություն, կրթություն, գիտություն, մշակույթ և այլ բնագավառներ: Ծրագրերը, ինչպես և ցանկացած այլ տեխնոլոգիա, ունեն տարբեր տեսակի լիցենզիաներ, որոնք կարգավորում են նրանց օգտագործման պայմանները: Համակարգչային ծրագրերի լիցենզիաների տեսակները կարևոր նշանակություն ունեն՝ նպաստելով հեղինակների իրավունքների պաշտպանությանը, օգտագործողների իրավունքների ապահովմանը, ինչպես նաև ծրագրերի ազատության և հասանելիության աստիճանի որոշմանը:

Ուսումնասիրելով այս թեման՝

- Կնկարագրեք համակարգչային ծրագրերի լիցենզիաների տարբեր տեսակների նշանակությունը
- Կվերլուծեք տարբեր լիցենզիաների իրավական պայմաններն ու սահմանափակումները
- Կիմանաք բաց աղբյուրով ծրագրերի լիցենզիաների ու պայմանների տարբերությունների մասին՝ միավորելով դրանք ծրագրավորման և օգտագործման տարբեր նպատակների համար:
- Կհասկանաք, թե ինչպես են կառուցված տարբեր ծրագրային լիցենզիաները և ինչպիսի օրինական պայմաններով կարող են օգտագործվել համակարգչային ծրագրերը:

ԴԱՍ 3

ԾՐԱԳՐԱՅԻՆ ԱՐՏՈՆԱԳՐԵՐ ԵՎ ԴՐԱՆՑ ԱԶԴԵՑՈՒԹՅՈՒՆԸ ՀԱՍԱՐԱԿՈՒԹՅԱՆ ՎՐԱ

Համակարգչային ծրագրերը կամ ավելի ընդգրկուն՝ ծրագրային ապահովումը (software) այսօր հանդիպում է մեզ ամենուր՝ հեռախոսներում ու համակարգիչներում, ավտոմեքենաներում և հեռուստացույցներում, գրեթե ամենուր, որտեղ մենք գործողություններ ենք կատարում լայն իմաստով մեքենաների հետ:

Ծրագրային ապահովման աշխարհում բոլոր ծրագրերն օգտագործման իրավունքի տեսանկյունից հավասար չեն ստեղծված: Տարբեր տեսակի ծրագրային ար-

տոնագրերը թույլ են տալիս կամ սահմանափակում են օգտատերերին և այլ ծրագրավորողների տարբեր կերպ օգտագործելու դրանք:

Ինչի՞ մասին է խոսքը: Ծրագրային ապահովման արտոնագիրը սահմանում է, թե ինչ կարող եք և չեք կարող անել տվյալ ծրագրային ապահովման հետ՝ սկսած տեղադրումից ու պատճենումից մինչև խմբագրում և տարածում: Արտոնագրերի այս տարբերակները հասկանալը կարևոր է այն աշխարհում, որտեղ ծրագրային ապահովումն անբաժան մասն է կազմում մեր առօրյա կյանքում՝ դպրոցում և աշխատանքում, նույնիսկ առօրյա գործերում: Այս դասին մենք կուսումնասիրենք ծրագրային արտոնագրերի տեսակները, բնութագրերը և դրանց ազդեցությունը հասարակության վրա:

Ծրագրային արտոնագրերի տեսակները

Ծրագրային արտոնագրերը կարելի է դասակարգել մի քանի հիմնական տեսակի՝ հիմնվելով դրանց օգտագործման իրավունքների և սահմանափակումների վրա:

Ներկայացնենք ամենատարածված տեսակները:

1. Փակ արտոնագիր (անգլերեն՝ Proprietary Software License)

Փակ ծրագրային ապահովումը, որը հաճախ կոչվում է փակ կամ փակ կոդով ծրագրային ապահովում (closed–source software), սովորաբար ստեղծվում է այնպիսի ընկերությունների կողմից, որոնք պահպանում են ամբողջական վերահսկողություն ծրագրի տարածման և օգտագործման վրա:

Օրինակ՝ Microsoft Office, Adobe Photoshop և մի շարք գործավար համակարգեր՝ macOS, Windows և այլն:

Ինչո՞վ է յուրահատուկ այս տեսակի արտոնագիրը:

- Օգտատերերը գնում են ծրագրի արտոնագիր՝ այն օգտագործելու համար, բայց ծրագիրն իրենց չի պատկանում:
- Ծրագրի կոդը գաղտնագրված է օգտատերերից, որը նշանակում է, որ նրանք չեն կարող այն խմբագրել կամ դիտել: Սա վերաբերում է ոչ միայն «հասարակ» օգտատերերին, նույնիսկ ծրագրավորողը չի կարող դիտարկել այն ծրագրային կոդը, որով գրված է տվյալ ծրագիրը: Փաստացի, ծրագրային կոդը գաղտնիք է:
- Հաճախ պահանջվում է պարբերական վճար կամ միանվագ վճարում ծրագրին հասանելիություն ունենալու համար:

Իսկ ի՞նչ ազդեցություն ունի արտոնագիրը: Փակ ծրագրային ապահովումը թույլ է տալիս ընկերություններին ֆինանսավորել զարգացումները և ապահովել աջակցություն, սակայն այն սահմանափակում է օգտատերերին, որը կարող է նվազեցնել հա–

մագործակցությունը և բարձրացնել կախվածությունը ընկերություններից: Անհատը կամ ծրագրավորողների համայնքը ոչ մի ազդեցություն չունի տվյալ ծրագրի վրա, մենաշնորհը պատկանում է արտադրող ընկերությանը:

2. Ազատ օգտագործման ծրագրեր (Freeware)

Ազատ օգտագործման ծրագրային ապահովումը բաշխվում է անվճար, սակայն այն չի ներառում կոդին հասանելիություն:

Օրինակ՝ Skype, Adobe Acrobat Reader և բազմաթիվ բջջային հավելվածներ:

Ազատ օգտագործման ծրագրերի յուրահատկությունները

- Օգտատերերը կարող են օգտագործել ծրագիրն անվճար, բայց չեն կարող խմբագրել այն:
- Հաճախ ֆինանսավորվում է գովազդներից, նվիրատվություններից կամ վճարովի ծառայություններից:
- Կարող է սահմանափակված լինել բիզնեսի կամ առևտրային օգտագործման համար:

Ազատ օգտագործման ծրագրերը թույլ են տալիս ավելի շատ մարդու օգտվել պիտանի գործիքներից՝ առանց ծախսերի: Սակայն դրա սահմանափակումները թույլ չեն տալիս օգտատերերին համապատասխանեցնել ծրագիրը իրենց կարիքներին: Այս հասանելիությունը կարող է նպաստել թվային ընդգրկվածությանը, բայց կարող է առաջացնել անհանգստություն գաղտնիության կամ տվյալների հավաքագրման վերաբերյալ:

Ավելին՝ շատ կարևոր է հասկանալ, որ եթե ծրագիրը անվճար է, դա կարող է լինել այն պատճառով, որ տվյալ ընկերությունը հավաքագրում է բոլորիս տվյալները և հետագայում դրանք վաճառում, ***օրինակ***՝ գովազդով, մարքեթինգային ծառայություններով կամ վաճառքներով զբաղվող ընկերություններին: Այսինքն՝ հաճախ անվճարը նշանակում է, որ վաճառվում են մեր տվյալները, և հենց դրանից է եկամուտ ստանում արտադրող ընկերությունը:

Իհարկե, կարող են լինել ֆինանսավորման այլ տարբերակներ: ***Օրինակ***՝ կան օգտակար ծրագրեր, որոնք հանգանակություններ են ստանում անհատներից և մեծ ընկերություններից և նման ֆինանսական մոդելի շնորհիվ մնում են անվճար: Այսպես, ***օրինակ***՝ այսօրվա ամենաանվտանգ զրուցարաններից մեկը՝ Signal–ը, որը չի վաճառում մեր տվյալները, անվճար է շնորհիվ հանգանակությունների: Իսկ, օրինակ, Telegram, Viber, WhatsApp զրուցարաններն անվճար են, քանի որ օգտագործում են մեր տվյալները, գովազդ են ցույց տալիս, վճարովի ծառայություններ ունեն իրենց մեջ ներառված:

Նաև անվճար են լինում բաց կոդով ծրագրերից շատերը:

Դիտարկենք հենց այդպիսի արտոնագիրը:

3. Բաց կոդով արտոնագիր (Open–Source License)

Բաց կոդով ծրագրային ապահովումը թողարկվում է այնպիսի արտոնագրով, որը թույլ է տալիս օգտատերերին ազատորեն տեսնել, խմբագրել և տարածել կոդը:

Օրինակ՝ Linux գործավար համակարգը, Mozilla Firefox և WordPress: Շատերի հեռախոսներում տեղադրված Android գործավար համակարգը, որը արտադրում է Google ընկերությունը, նույնպես բաց կոդով է, մինչդեռ Apple–ի ստեղծած iOS–ը փակ արտոնագրով է տարածվում, որի պատճառով iOS–ը տեղադրվում է միայն Apple –ի կողմից արտադրվող հեռախոսներում: Իսկ Android համակարգը, ընդ որում՝ տարբեր ձևափոխումներով, հնարավոր է գտնել շատ տարբեր արտադրողների սարքերում, ընդ որում՝ ոչ միայն հեռախոսներում, այլև ժամացույցներում, հեռուստացույցներում, մեքենաներում և այլն:

Իսկ հիմա ներկայացնենք բաց կոդով ծրագրերի յուրահատկությունները.

- Ծրագրի կոդը ամբողջությամբ հասանելի է հանրությանը, որը խթանում է համագործակցությունը:
- Օգտատերերն ու մշակողները կարող են հարմարեցնել ծրագրային ապահովումը տարբեր կարիքների համար՝ ստեղծելով կիրառումների և բարեփոխումների լայն սպեկտոր: Փաստացի կամայական բաց կոդով ծրագիրը ոչ միայն կարելի է ձևափոխել, այլև դրանից ստեղծել բացարձակ նոր ծրագրեր:

Ինքնուրույն հետազոտության համար

Ավելի մանրամասն և խորությամբ հասկանալ ցանկացողները կարող են ինքնուրույն ուսումնասիրել բաց կոդով արտոնագրերի մի քանի տարբերակ: Տարածված արտոնագրերն են՝ GNU General Public License (GPL), MIT License և Apache License:

Բաց կոդով արտոնագիրը շատ մեծ ազդցություն է ունեցել ինչպես ծրագրային ապահովման զարգաման, այնպես էլ բազմաթիվ այլ ոլորտների վրա: Բաց կոդով ծրագրային ապահովումը խթանում է համայնքային համագործակցությունն ու գիտելիքների փոխանակումը: Այն հնարավորություն է տալիս օգտատերերին հարմարեցնելու ծրագրերն իրենց կարիքներին, բարձրացնելու թափանցիկությունը և ապահովելու ուսուցողական հնարավորություններ սկսնակ մշակողների համար: Բաց կոդով նախագծերը հաճախ ապահովում են նորարարություն և առաջարկում այլընտրանքներ թանկարժեք սեփականաշնորհային ծրագրերին: Թվային անվտանգության տեսանկյունից նույնպես սա կարևոր է, քանի որ կամայական բաց կոդով ծրագիր կարող է ուսումնասիրվել անկախ փորձագետների կողմից, որոնք կարող են փաստել, որ, օրինակ, ծրագրի մեջ չկան թաքնված «ծուղակներ», մարդկանց հետևելու գաղտնի գործիքներ և այլն:

4. Ազատ/վճարովի (Freemium) արտոնագիր

Freemium ծրագրային ապահովումն անվճար է հիմնական մակարդակում, սակայն վճարովի է ավելի բարդ գործառույթների համար:

Օրինակ՝ Spotify, Google Drive և Instagram: Բոլոր օրինակներում հարթակները առաջարկում են անվճար տարբերակներ, սակայն լրացուցիչ հնարավորությունների համար արդեն պահանջում են վճարովի բաժանորդագրություն:

Նման արտոնագրի հիմնական բնութագիրը

- Հիմնական տարբերակն անվճար է, սակայն լրացուցիչ գործառույթները պահանջում են բաժանորդագրություն:
- Հաճախ է օգտագործվում սկսնակ (stratup) ընկերությունների կողմից՝ ներգրավելու օգտատերերի:
- Օգտատերերը կարող են փորձարկել ծրագիրը և որոշել՝ ցանկանում են ներդրում կատարել, թե՝ ոչ՝ լրացուցիչ հնարավորություններ ստանալու համար:

Freemium մոդելը թույլ է տալիս օգտատերերին փորձարկել ծրագիրը նախքան վճարելը, որը մատչելի է դարձնում այն ավելի շատ մարդկանց համար: Սակայն կարող է առաջանալ որոշակի սահմանափակում, երբ կարևոր գործառույթները վճարովի են:

ՀԱՄԵՄԱՏՈՒԹՅՈՒՆ ԳՈՐԾԱՎԱՐ ՀԱՄԱԿԱՐԳԵՐԻ ՕՐԻՆԱԿՈՎ

Այսօր համակարգիչների մեծամասնության վրա օգտագործվում է երեք գործավար համակարգ: Երկուսը փակ կոդով են: *Առաջինը* Microsoft–ի արտադրած WIndows համակարգն է, որը տեղադրվում է գրեթե ցանկացած համակարգչի մեջ: *Երկրորդը* Apple ընկերության macOS համակարգն է, որը տեղադրվում է հենց նույն ընկերության արտադրած սարքերում: Երկուսն էլ վճարովի են: Պարզապես macOS–ի դեպքում առանձին վճար գոյություն չունի, այն արդեն իսկ ընդգրկված է համակարգչի գնի մեջ: *Երրորդը* բաց կոդով օպերացիոն համակարգերի ընտանիք է, որը հիմնված է Linux գործավար համակարգի վրա: Դրանցից ամենատարածվածը Ubuntu ընտանիքն է, կան Fedora, Kali Linux և այլն (հարկ է նշել, որ հեռախոսներում տեղադրվող Android համակարգը նույնպես հիմնված է Linux–ի վրա): Նշենք նաև, որ միշտ չէ, որ բաց կոդը նշանակում է անվճար: Այսպես, ***օրինակ***՝ Linux համակարգեր կան, որոնք վճար են պահանջում, ***օրինակ***՝ SUSE, Red Hat և այլն:

Նաև կարևոր է նշել, որ գրեթե բոլոր վճարովի ծրագրերի համար հնարավոր է գտնել բաց կոդով անվճար այլընտրանք: Այսպես, ***օրինակ***՝ Microsoft Office, որն իրենից ներկայացնում է ամենապահանջված Word, Excel և մի շարք այլ ծրագրերի փաթեթ, որը հաճախ օգտագործվում է «կոտրած» տարբերակով: Այն հնարավոր է փոխարինել բաց կոդով անվճար Libre Office փաթեթով: Կամ՝ ազատ/վճարովի (Freemium) տարբերակով Google Drive–ն ունի գրեթե նույն հնարավորությունները:

ԲԱՑ ԿՈԴԻ ՊԱՏՄՈՒԹՅՈՒՆԸ

Բաց կոդով ծրագրերի գաղափարը՝ երբ կոդը հասանելի է յուրաքանչյուրին դիտելու, խմբագրելու և տարածելու համար, ունի հետաքրքիր պատմություն, որն ազդել է թվային աշխարհի վրա:

1. Վաղ օրեր՝ համագործակցային սկիզբ

1950–1960–ականներին ծրագրային ապահովումը դեռևս առևտրային ապրանք չէր: Համակարգչային ոլորտի գիտնականներն ու հետազոտողները հաճախ բաց կիսում էին կոդը՝ բարելավելու և սովորելու նպատակով:

2. Փակ ծրագրային ապահովման վերելքը (1970–ականներ)

1970–ականներին ծրագրային ապահովման զարգացումը սկսեց փոփոխվել, քանի որ ընկերությունները տեսան ծրագրերի վաճառքի բիզնես հնարավորությունները: Ծրագրերը սկսեցին պաշտպանվել հատուկ արտոնագրերով, որոնք սահմանափակում էին օգտատերերին: Փաստացի, հենց գիտնականների ձեռքից ծրագրավորումն անցավ գործարաններին, շատ արագ արտոնագրերի մեծ մասը սկսեց ձևափոխվել բացից դեպի փակը, որը հանգեցրեց համայնքի կողմից զարգացման հնարավորությունների սահմանափակումների, տեղեկույթը շատ հաճախ այլևս չէր փոխանակվում մարդկանց միջև, քանի որ դառնում էր առևտրային գաղտնիք:

3. Ազատ ծրագրային ապահովման շարժում (1980–ականներ)

Այս սահմանափակումներին ի պատասխան՝ ծրագրավորող Ռիչարդ Սթոլմանը (Richard Stallman) հիմնեց Ազատ ծրագրային ապահովման շարժումը (Free Software Movement)՝ ստեղծելով մի կողմից՝ գաղափարային հիմքը, որը թույլ էր տալիս բաց, միասնական տեղեկատվական տեխնոլոգիաների զարգացումը, մյուս կողմից՝ բաց կոդով մի արտոնագիր, որը կոչեց GNU:

Ռիչարդ Սթոլմանը փաստացի դարձավ բաց կոդով ծրագրային ապահովման գաղափարախոսության հիմնադիր հայրերից մեկը: Նա համարում էր, որ մարդու ստեղծագործ կամքին չպետք է խանգարեն ընկերությունների բիզնես շահերը, տեղեկույթը, գիտելիքը պետք է տարածվեն բացեիբաց ու ազատ:

Ռիչարդ Սթոլման

4. Բաց կոդի ծնունդը (1990–ականներ)

1990–ականներին սկսվեց համացանցի զարգացումը: 1998–ին ստեղծվեց բաց կոդի նախաձեռնությունը՝ Open Source Initiative (OSI)–ը՝ նպաստելով բաց կոդի ծրա–գրերի տարածմանը:

5. Բաց կոդով մեծ նախագծեր (2000–ականներ)

2000–ականների սկզբին բաց կոդով ծրագրերը դարձան անփոխարինելի: Linux–ը, Apache–ը և MySQL–ը, դառնալով կարևորագույն ծրագրեր, հիմք դրեցին հա–մացանցի զարգացմանը:

6. Բաց կոդը այսօր. համագործակցային նոր շրջան

Այսօր գոյություն ունի ծրագրավորողների հարթակ՝ GitHub, որը թույլ է տալիս կիսվել սեփական ծրագրերով, համագործակցել միմյանց հետ: Փաստացի, սա կար–ծես մի սոցիալական ցանց լինի, որտեղ անհատ ծրագրավորողները, ծրագրավորող–ների խմբերը և համայնքները, մեծ ընկերությունները միմյանց հետ կիսվում են աշ–խատանքներով: GitHub–ը և նմանօրինակ հարթակները հեշտացրել են ծրագրավո–րողների համագործակցությունը: Microsoft, IBM և Google և այլ ընկերություններ ակ–

տիվորեն աջակցում են բաց կոդով նախագծերին՝ գնահատելով համագործակցության առավելությունները:

Ծրագրային ապահովումը մարդկային գործունեության մի հատված է: Իրականում արտոնագրերը, հեղինակային իրավունքը, գիտելիքի և տեղեկույթի տարածումը շատ ավելի լայն ոլորտ են: Եվ արժե այս ամենը հասկանալու համար ավելի ընդհանուր պատկեր ստանալ:

ԱՐՏՈՆԱԳՐԵՐ ԼԱՅՆ ԻՄԱՍՏՈՎ. ՀԵՂԻՆԱԿԱՅԻՆ ԻՐԱՎՈՒՆՔ ԵՎ CREATIVE COMMONS ԱՐՏՈՆԱԳԻՐ

Ծրագրային ապահովման և ստեղծագործությունների հեղինակային իրավունքները կարևոր դեր են խաղում ինչպես ստեղծագործողների, այնպես էլ օգտատերերի իրավունքների պաշտպանության գործում: Արտոնագրերն իրավական պայմանագրեր են, որոնք թույլ են տալիս սահմանել, թե ինչպես կարելի է օգտագործել, տարածել և փոփոխել տվյալ ստեղծագործությունը կամ ծրագիրը:

Հեղինակային իրավունք

Հեղինակային իրավունքն իրավական պաշտպանություն է, որը, անկախ ոլորտից, ստեղծագործողներին (լինի ծրագրավորող, թե կոմպոզիտոր) թույլ է տալիս վերահսկել իրենց ստեղծագործությունների օգտագործումը: Այն ապահովում է, որ միայն ստեղծագործողը կամ նրա թույլտվությամբ որոշակի անձինք կարող են օգտագործել, տարածել կամ փոփոխել ստեղծագործությունը:

- Հեղինակային իրավունքը թույլ է տալիս ստեղծագործողին բացառիկ իրավունք ունենալ իր ստեղծած նյութի նկատմամբ:
- Այն սովորաբար կիրառվում է գրական ստեղծագործությունների, երաժշտության, ֆիլմերի, ծրագրային ապահովման և այլ ստեղծագործությունների համար:
- Հեղինակային իրավունքը սովորաբար գործում է ստեղծագործողի ողջ կյանքի ընթացքում, ինչպես նաև որոշակի տարիներ մահից հետո (*օրինակ*՝ 50 կամ 70 տարի՝ կախված երկրից):

Հեղինակային իրավունքը կարևոր է ստեղծագործողների համար՝ հնարավորություն տալով նրանց ստանալու իրենց աշխատանքի համար վարձատրություն: Այն նաև թույլ է տալիս պաշտպանել ստեղծագործությունը արտոնագրի խախտումներից և անօրինական տարածումից:

Սակայն որոշ դեպքերում հեղինակային իրավունքը կարող է սահմանափակել հանրային հասանելիությունը և նվազեցնել համագործակցության հնարավորու-

թյունները: ***Օրինակ***՝ դասագրքերում հեղինակները հաճախ չեն կարողանում օգտագործել հետաքրքիր լուսանկարներ, քանի որ դրանք պաշտպանված են հեղինակային իրավունքով, իսկ դրանց գնելը կարող է կտրուկ բարձրացնել դասագրքի գինը:

Հեղինակային իրավունքի չափազանց կոշտ կիրառումը հանգեցրեց դրան հակադարձ շարժման, որի հետևանքով ստեղծվեց նոր արտոնագիր՝ Creative Commons (CC):

Creative Commons (CC) արտոնագրերի հավաքածուն (կա դրանց մի քանի տեսակ) թույլ է տալիս ստեղծագործողներին որոշել, թե ինչպես կարող են ուրիշներն ավելի ազատ օգտագործել իրենց աշխատանքը՝ պահպանելով որոշակի իրավունքներ և սահմանափակումներ:

CC արտոնագրերը թույլ են տալիս ստեղծագործողներին ընտրել իրենց ստեղծագործությունն օգտագործելու իրավունքների շրջանակը՝ տրամադրելու հանրությանն ազատ կամ որոշակի սահմանափակումներով:

Տարբեր տեսակի CC արտոնագրեր թույլ են տալիս տարբեր ազատություններ, ***օրինակ***՝ որոշները թույլ են տալիս միայն անհատական օգտագործման համար տարածում, մինչդեռ մյուսները թույլ են տալիս խմբագրում և փոփոխում: Այսինքն՝ հեղինակը կարող է իր լուսանկարն ազատորեն, անվճար օգտագործելու հնարավորություն տալ այլոց՝ միայն պահանջելով նշել, թե ով է հեղինակը: Մյուս տարբերակը թույլ է տալիս օգտագործել միայն ոչ կոմերցիոն դեպքերում: Այսինքն՝ որոշակի լուսանկար կարելի է տեղադրել անձնական բլոգում, սոցցանցում, բայց չի կարելի, օրինակ, սրճարանի գովազդում օգտագործել:

Creative Commons արտոնագրերը լայնացնում են ստեղծագործությունների հասանելիությունը հասարակության համար՝ ստեղծելով համագործակցային միջավայր: Հատկապես գիտական և կրթական ոլորտներում CC արտոնագրերը մեծ հնարավորություններ են տալիս մարդկանց և կազմակերպություններին՝ սովորելու և արարելու միջոցով բարելավելու հանրային հասանելիության շտեմարանները:

Հեղինակային իրավունքի և Creative Commons–ի տարբերությունները

Creative Commons արտոնագրերը հնարավորություն են տալիս ստեղծագործողներին որոշակի սահմանափակումներով տարածելու իրենց ստեղծագործությունը, մինչդեռ հեղինակային իրավունքը սովորաբար թույլ չի տալիս ազատ տարածում առանց ստեղծագործողի համաձայնության: CC արտոնագրերը ավելի շատ ուղղված են հասարակության հասանելիության ու համագործակցության խթանմանը՝ դառնալով կարևոր գործիքներ տեղեկատվական տեխնոլոգիաների և կրթական նախագծերի համար:

Creative Commons–ի օգտագործման օրինակներ

Creative Commons արտոնագրերը լայնորեն կիրառվում են տարբեր ոլորտներում՝ արվեստից և երաժշտությունից մինչև գիտություն և կրթություն: Բերենք օրինակներ:

1. Վիքիպեդիա

Վիքիպեդիա՝ ազատ առցանց հանրագիտարանը, օգտագործում է Creative Commons Attribution–ShareAlike (CC BY–SA) արտոնագիրը:

Վիքիպեդիա ազատ հանրա-գիտարան

Բոլոր օգտատերերը կարող են կարդալ, խմբագրել և տարածել Վիքիպեդիայի հոդվածները՝ պայմանով, որ նշում են հեղինակներին և տարածում են նյութը նույն արտոնագրի ներքո: Այսինքն՝ եթե օգտագործել ես Վիքիպեդիայից նյութ, նկար, ապա քո ստեղծածն էլ պետք է Creative Commons արտոնագրով տարածես: Սա խթանում է համագործակցությունը և գիտելիքների ազատ տարածումը ամբողջ աշխարհում:

2. Մշակութային հաստատություններ

Թանգարանները, գրադարանները և արխիվները հաճախ օգտագործում են Creative Commons արտոնագրեր՝ իրենց հավաքածուները թվայնացնելու և հանրությանը հասանելի դարձնելու համար:

Օրինակ՝ Եվրոպայի թանգարանների թվային հավաքածու Europeana–ն տրամադրում է միլիոնավոր մշակութային պատկերներ CC արտոնագրերի ներքո: Սա թույլ է տալիս ուսումնասիրողներին, ուսանողներին և արվեստագետներին օգտվել արժեքավոր նյութերից և օգտագործել իրենց աշխատանքներում: Ուսուցիչները հանգիստ կարող են օգտագործել դրանք՝ իրենց դասերն ավելի հետաքրքիր դարձնելու համար:

3. YouTube

YouTube–ը թույլ է տալիս ստեղծագործողներին տարածել իրենց տեսանյութերը CC BY արտոնագրի ներքո: Տեսանյութ տեղադրողն ինքը կարող է որոշել՝ ազատ արտոնագրով տարածել այն, թե՝ դնել հեղինակային իրավունքի սահմանափակումներ:

Սա հնարավորություն է տալիս այլ օգտատերերի ռեմիքսելու, վերաօգտագործելու և տարածելու այդ տեսանյութերը՝ նշելով հեղինակին: Սա խթանում է ստեղծագործականությունը և համագործակցությունը բովանդակության ստեղծման գործում:

Այս օրինակները ցույց են տալիս, թե ինչպես են Creative Commons արտոնագրերը, բաց կոդով ծրագրերն աջակցում գիտելիքների, արվեստի և մշակույթի ազատ տարածմանը, որն էլ իր հերթին նպաստում է ավելի բաց և համագործակցային հասարակության ձևավորմանը:

Ամփոփում

Համակարգչային ծրագրերի լիցենզիաները կարևոր բաղադրիչ են ոչ միայն ծրագրերի զարգացման գործընթացում, այլև իրավական տեսանկյունից: Յուրաքանչյուր լիցենզիա տարբերվում է իր իրավական պայմաններով, օգտագործման հնարավորություններով և տարածման սահմանափակումներով: Ազատ տարածվող ծրագրերը ազատ փոփոխության հնարավորություն են տալիս, կիսվող ծրագրերը թույլ են տալիս որոշակի ժամանակի ընթացքում փորձարկում և օգտագործում, իսկ կոմերցիոն ծրագրերը ապահովում են հեղինակի ամբողջական վերահսկողությունը՝ տրամադրելով միայն սահմանափակ լիցենզիա: Լիցենզիաների ճիշտ ընտրությունը կարևոր է ծրագրի ճիշտ օգտագործման և իրավական խնդիրներից խուսափելու համար:Ծրագրերի լիցենզիաների տարբեր տեսակների իմացությունը անհրաժեշտ է ինչպես ծրագրավորողներին, այնպես էլ օգտվողներին:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ծրագրային արտոնագրերի ի՞նչ տեսակներ կան:
2. Ի՞նչ է հեղինակային իրավունքը:
3. Ո՞րն է հեղինակային իրավունքի և *Creative Commons*–ի տարբերությունը:
4. Ի՞նչ տարբերություններ կան փակ արտոնագրի և բաց կոդով արտոնագրի միջև, և ինչպե՞ս են դրանք ազդում օգտատերերի վրա:

Գործնական աշխատանք

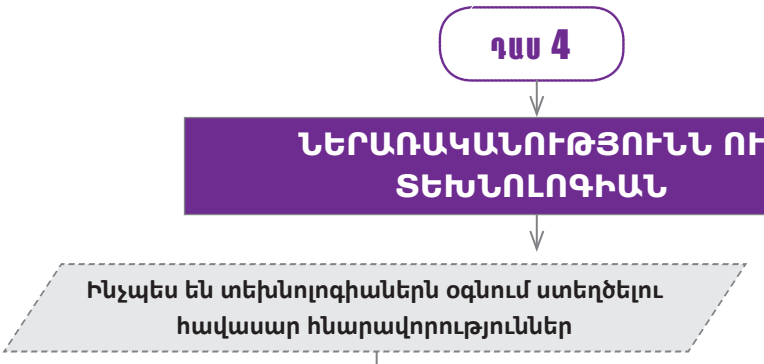
Ջույգերով կազմե՛ք բաց քննարկում բաց և փակ կոդով ծրագրային ապահովման դրական և բացասական կողմերի վերաբերյալ:

ԹԵՄԱ 4 ՆԵՐԱՌԱԿԱՆՈՒԹՅՈՒՆՆ ԱՊԱՀՈՎՈՂ ՏԵԽՆՈԼՈԳԻԱՆԵՐ

Տեխնոլոգիաները դարձել են հզոր գործիք աշխարհին ավելի ներառական դարձնելու համար՝ վերացնելով տարբեր տեսակի խոչընդոտները և ստեղծելով հավասար հնարավորություններ բոլորի համար: Տեխնոլոգիաները հզոր գործիք են հատկապես այն մարդկանց համար, ովքեր ունեն ֆիզիկական, լսողական, տեսողական կամ լեզվաբառարանական սահմանափակումներ: Դրանք թույլ են տալիս մարդկանց օգտագործել թվային տեխնոլոգիաներ առաջադիմելու և զարգանալու համար ՝ անկախ նրանց ունակություններից կամ առողջական վիճակից: Այս թեմայի շրջանակներում:

Ուսումնասիրելով այս թեման՝

- Կքննարկենք, թե ինչպես են տեխնոլոգիական զարգացումները նպաստում ավելի համարժեք և հասանելի միջավայրերի ստեղծմանը:
- Կնկարագրեք թվային գործիքներ, որոնք ստեղծված են լայն հասանելիություն ունենալու նպատակով:
- Կպարզեք, թե ինչպես են տեխնոլոգիաները աջակցում տարբեր խնդիրներ ունեցող սովորողներին:
- Կբացատրեք տեխնոլոգիական գործիքների կիրառումը աշխատանքի, կրթության և սոցիալական գործունեության մեջ՝ ներառելով ֆիզիկական սահմանափակումներով անձանց:



Ներառականություն նշանակում է ստեղծել և ապահովել այնպիսի միջավայր, որտեղ յուրաքանչյուր մարդ, անկախ իր կարողություններից, ծագումից, առողջական կամ այլ հանգամանքներից, ունենա հասանելիություն նույն հնարավորություններին, տեղեկատվությանը և կապերին:

Այժմ ներկայացնենք, թե տեխնոլոգիան ինչպես է նպաստում ներառականությանը կյանքի տարբեր ոլորտներում:

1. Հասանելիության բարելավում ֆիզիկական սահմանափակումներով անձանց համար

Տեխնոլոգիան էականորեն բարելավել է ֆիզիկական սահմանափակումներով անձանց հասանելիությունը՝ հնարավորություն տալով նրանց ավելի արդյունավետ գործածելու թվային գործիքները, վիրտուալ տարածքը:

• *Էկրանի ընթերցողներ և ձայնային օգնականներ:* Համակարգիչների և հեռախոսների համար գոյություն ունեն բազմաթիվ ծրագրեր, որոնք թույլ են տալիս առողջական խնդիրներ ունեցող մարդկանց օգտվելու սարքերից: Որպես ***օրինակ***՝ NVDA (NonVisual Desktop Access) ծրագիրը կարևոր է տեսողական խնդիրներ ունեցող անձանց համար: Դա և դրա նման բազմաթիվ այլ ծրագրեր էկրանի տեքստը վերափոխում են ձայնայինի՝ հնարավորություն տալով օգտատերերին օգտվելու կայքերից, կարդալու փաստաթղթեր և ուղարկելու հաղորդագրություններ: Ձայնային օգնականները, ինչպես, ***օրինակ***՝ հեռախոսներում տեղադրված Siri–ն և Google Assistant–ը, թույլ են տալիս օգտատերերին կատարել տարբեր գործառույթներ՝ առանց սարքին դիպչելու կամ այն տեսնելու:

• *Իրական ժամանակի ենթագրեր և տեքստի վերագրառման ծառայություններ*: Zoom–ը և նմանատիպ հարթակները իրական ժամանակի ենթագրերն անմիջապես գրառում են խոսքը, որը մեծ օգնություն է լսողության խնդիրներ ունեցող օգտատերերի համար: YouTube–ը և շատ այլ տեսանյութերի դիտման ծառայություններ նույնպես առաջարկում են ենթագրեր, իսկ Google Live Transcribe–ը տրամադրում է խոսակցությունների ենթագրեր՝ հեշտացնելով լսողության խնդիրներ ունեցող մարդկանց հաղորդակցումը: Ցավոք, նման ծրագրերը դեռ լիարժեք չեն հարմարեցված հայերենին, բայց առաջիկայում իրավիճակը միանշանակ կբարելավվի:

• *Հարմարեցված սարքեր*: Ֆիզիկական սահմանափակումներով անձանց համար նախատեսված միջոցները, ինչպիսիք են հայացքին հետևող տեխնոլոգիաները, օրինակ՝ Tobii Dynavox–ը, թույլ են տալիս ղեկավարել համակարգիչներն ու սմարթֆոնները: Այս սարքերը հնարավորություն են տալիս օգտատերերին կառավարելու սարքերը և մուտքագրելու գրվածք առանց ձեռքով հպվելու՝ ստեղծելով հնարավորություններ թե՛ աշխատանքի, թե՛ ինքնակրթվելու, թե՛ ժամանցի համար:

2. Ներառական կրթության խթանում

Կրթական տեխնոլոգիաները վերափոխում են ուսումնական միջավայրերը՝ աջակցելով տարբեր կարիքներով, տարբեր ծագմամբ և ուսումնական տարբեր ոճ նախընտրող սովորողներին:

Հեռավար և հիբրիդ ուսուցում: COVID–19 համավարակի ընթացքում Google Classroom և Microsoft Teams հարթակները կարևոր դարձան հեռավար կրթության համար՝ հատկապես հեռավոր կամ ծառայություններից զուրկ շրջաններում սովորողների համար: Այս հարթակները թույլ տվեցին առողջական խնդիրներ ունեցող կամ ֆիզիկական սահմանափակումներ ունեցող սովորողներին լիովին մասնակցելու դասերին տնից:

Լեզվական թարգմանչական գործիքներ, ինչպես Google Translate–ը: Google Translate–ն օգնում է հաղթահարելու լեզվական խոչընդոտները՝ թարգմանելով ուսումնական նյութերը կամ հնարավորություն տալով ուսուցիչներին և ոչ մայրենի լեզվով խոսող սովորողներին հաղորդակցվելու իրական ժամանակում: Այս գործիքը կարող է օգնել բազմալեզու համայնքներում՝ ներառելով այն սովորողներին, որոնք դժվարություն կունենան ուսումնառության ընթացքում:

3. Աշխատավայրի ներառականության խթանում

Տեխնոլոգիան թույլ է տալիս ստեղծել ավելի ներառական աշխատավայր՝ առաջարկելով գործիքներ, որոնք համապատասխանում են տարբեր կարողությունների տեր մարդկանց:

Հեռավար աշխատանքի հարթակներ: Slack, Zoom և Asana և նման հարթակները նպաստում են հեռավար համագործակցությանը: ***Օրինակ***՝ մանկահասակ երեխաների խնամող մայրերը կամ ֆիզիկական սահմանափակումներով անձինք կարող են արդյունավետ աշխատել տնից:

Օժանդակ տեխնոլոգիաներ աշխատավայրում: Արդեն նշված բազմաթիվ ծրագրեր, որոնք թույլ են տալիս ֆիզիկական սահմանափակումներով անձանց աշխատել փաստաթղթերի, տեսազանգերի միջոցով, այսօր փոխում են աշխատանքի որակը: Դեռ երբեք մարդկության պատմության մեջ ֆիզիկական սահմանափակումներով անձինք հնարավորություն չեն ունեցել այսքան դյուրին և արժանապատիվ ներգրավվելու խմբային աշխատանքներում, դուրս գալու հասարակական մեկուսացումից:

4. Տնտեսական բարելավում հասանելի տեխնոլոգիայի միջոցով

Տեխնոլոգիայի հասանելիությունը կարող է լինել թանկարժեք, սակայն նոր լուծումները դարձնում են սարքերն ու համացանցային հասանելիությունն ավելի մատչելի, որը նպաստում է թվային տարանջատման հաղթահարմանը: Ծանր ֆինանսական պայմաններում ապրող անձինք կարողանում են մատչելի կրթություն ստանալ, անվճար կամ ցածր վճարով օգտվել տեխնոլոգիական գործիքներից, բարելավել իրենց նյութական վիճակը և մտավոր զարգացումը:

Անվճար կրթական պաշարներ: Coursera, edX և YouTube հարթակներն առաջարկում են անվճար կամ ցածր վճարով բարձրորակ դասընթացներ, որոնք հնարավո–

րություն են տալիս աշխարհի տարբեր անկյուններից մարդկանց ձեռք բերելու նոր գիտելիքներ և հմտություններ:

Բաց կոդով անվճար ծրագրեր: LibreOffice–ի նման ծրագրերը և Linux գործավար համակարգը թույլ են տալիս օգտատերերին օգտվելու բարձրորակ գրասենյակային ծրագրերից և գործավար համակարգերից՝ առանց ֆինանսական լրացուցիչ ծախսերի: Բաց կոդով ծրագրերը հասանելի են համայնքային ներդրումների շնորհիվ և հաճախ ունեն ակտիվ օգտատերերի համայնքներ, որոնք աջակցում են նորամուծությունների և խնդիրների լուծմանը: Այս ծրագրերը կարևոր են նրանց համար, ովքեր ունեն սահմանափակ բյուջե և կարիք ունեն հուսալի գործիքների՝ իրենց աշխատանքն արդյունավետ կազմակերպելու համար, ***օրինակ***՝ ուսանողները, ուսուցիչները և փոքր բիզնես ունեցող գործարարները:

Ինչպես տեսնում ենք, մենք ապրում ենք աննախադեպ ժամանակաշրջանում, երբ միլիոնավոր մարդիկ, որոնք ունեն առողջական կամ ֆինանսական սահմանափակումներ, ստանում են հնարավորություն կրթվելու, զարգանալու, աշխատելու, ներգրավվելու հասարակական կյանքում:

Իհարկե, ներառականության զարգացման համար ոչ միայն ծրագրերի ու սարքերի առկայությունն է անհրաժեշտ, այլև կարևոր է, որ գիտելիքը, այդ գործիքների մասին տեղեկութը լայն տարածում ստանա, որպեսզի մարդիկ տեղեկանան այս ամենի մասին և կարողանան արդյունավետ կիրառել դրանք: Նաև խիստ կարևոր է, որպեսզի հասարակությունը, մենք բոլորս օգնենք միմյանց տեխնոլոգիաները բարի նպատակներով կիրառելու գործում:

Ամփոփում

Այս թեմայի ուսումնառությունը ցույց տվեց, որ տեխնոլոգիաները կարող են խթանել ներառականությունը տարբեր ոլորտներում՝ կրթությունից մինչև աշխատավայր ու հասարակություն: Թվային գործիքների ճիշտ օգտագործումը կարող է մեծապես բարելավել հասանելիությունը՝ թույլ տալով բոլորին, անկախ ֆիզիկական կամ այլ սահմանափակումներից, մասնակցել կրթական, աշխատանքային և սոցիալական գործունեությանը: Այդ իսկ պատճառով, կարևոր է, որ հասարակությունը կիրառի տեխնոլոգիաների հնարավորությունները՝ բարելավելու յուրաքանչյուրի կյանքը և ապահովելու նրանց հավասար հնարավորություններ:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Կրթական ոլորտի ներառականության խթանման ի՞նչ գործիքներ գիտեք:
2. Աշխատանքային ոլորտի ներառականության խթանման ի՞նչ հնարավորություններ են տալիս տեխնոլոգիաները:
3. Ինչպե՞ս է բարելավվել տնտեսական միջավայրը տեխնոլոգիաների օգնությամբ:
4. Ուսումնասիրիր մեկ թվային գործիք կամ տեխնոլոգիա, որը նախատեսված է հատուկ կարիքներ ունեցող մարդկանց համար: Ներկայացրու՝ ինչպես է այն նպաստում սոցիալական հավասարությանը:

ԹԵՄԱ 5

ԲԱՐԴ ՏՐԱՄԱԲԱՆԱԿԱՆ ՇՂԹԱՆԵՐ

Տրամաբանությունը տեղեկատվությունն արդյունավետ վերլուծելու և ճիշտ եզրակացություններ կատարելու կարևոր միջոց է: Թվային աշխարհում տվյալների մշակման կամ որոշումների ավտոմատացման խնդիրների լուծումները պահանջում են բարդ տրամաբանական շղթաների կառուցում: Այս շղթաները միավորում են մի քանի տրամաբանական գործողություններ՝ ԵՎ (AND), ԿԱՄ (OR), ՈՉ (NOT) և այլ օպերատորներ՝ ձևավորելով ավելի բարդ և ճշգրիտ պայմաններ: Տրամաբանական շղթաներն այսօր կիրառվում են ամենուր՝ որոնման համակարգերից մինչև ծրագրավորման ալգորիթմներ, թվային սխեմաներից մինչև արհեստական բանականության որոշումների կայացում: Բարդ տրամաբանական շղթաների ուսումնասիրումը զարգացնում է ոչ միայն տեխնոլոգիական մտածողությունը, այլև վերլուծական և քննադատական ունակությունները: Դրանք օգնում են հասկանալ, թե ինչպես կարող են մի քանի պարզ պայմաններ համադրվել ու ձևավորել մի ամբողջ համակարգ:

Ուսումնասիրելով այս թեման՝

- Կսովորեք կարդալ, գրել և բացատրել բարդ տրամաբանական արտահայտություններ՝ մի քանի պայմանների համադրությամբ:
- Կկարողանաք թվային սխեմաների միջոցով մոդելավորել տրամաբանական գործընթացներ, որոնք օգտագործվում են իրական համակարգերում:
- Կզարգացնեք քննադատական մտածողություն, որը անհրաժեշտ է պայմանական վերլուծություններ կատարելու և բանական որոշումներ կայացնելու համար:
- Կգիտակցեք, որ տրամաբանական ճշգրտությունը կոդավորման, տվյալների վերլուծության հիմքում ընկած հիմնարար հմտություն է:

ԴԱՍ 5

ԿԻՍԱԳՈՒՄԱՐԻՉ ԵՎ ԱՄԲՈՂՋԱԿԱՆ ԳՈՒՄԱՐԻՉ

10–րդ դասարանի դասընթացից արդեն քեզ ծանոթ է Բուլյան տրամաբանությունը, որի հիմքում ընկած է այն գաղափարը, որ պնդումը կամ ասույթն ունի երկու հնարավոր լուծում. այն կա՛մ ճիշտ է, կա՛մ սխալ, որևէ 3–րդ տարբերակ գոյություն չունի: Բուլյան տրամաբանությամբ է կառուցված տրամաբանական փականների

աշխատանքը, որոնց միջոցով էլ կազմակերպվում է մշակիչի (պրոցեսորի) աշխատանքը:

Պարզ տրամաբանական փականները օգտագործվում են տվյալների մշակման համար և կազմում են տրամաբանական շղթաների հիմքը: Ստորև ներկայացված են հիմնական պարզ տրամաբանական փականները.

Անվանումը	Գործառույթը	Փականը
AND (ԵՎ)	Ելքում ստացվում է 1, եթե մուտքերի բոլոր արժեքները 1 են:	
OR (ԿԱՄ)	Ելքում ստացվում է 1, եթե մուտքերից գոնե մեկի արժեքը 1 է:	
NOT (ՈՉ)	Փոփոխում է մուտքի արժեքը՝ 0–ն դարձնում է 1, 1–ը՝ 0:	
XOR (ԲԱՑԱՌՈՂ ԿԱՄ)	Ելքում ստացվում է 1, եթե մուտքերից միայն մեկի արժեքն է 1:	

Բարդ տրամաբանական շղթան կազմված է բազմաթիվ պարզ տրամաբանական փականներից (AND, OR, NOT, XOR և այլն):

Սարքավորումային մակարդակի թվաբանության տարրերը հիմնականում վերաբերում են տրամաբանական փականների միջոցով երկուական՝ 0 և 1 թվերի մշակմանը, ինչպիսիք են՝ գումարում, հանում, բազմապատկում և այլն: Այս մակարդակում թվաբանությունն իրականացվում է էլեկտրոնային սարքավորումների, *օրինակ*՝ կենտրոնական մշակիչների (պրոցեսորների) միջոցով:

Սարքավորումային մակարդակի թվաբանության մեջ հիմնականում կիրառվում են.

1. Երկուական թվեր

Սարքավորումային մակարդակում բոլոր թվերը ներկայացվում են երկուական համակարգով՝ 0–ներով և 1–երով: Համակարգիչները երկուական թվեր են օգտագործում, քանի որ այդ թվերը հեշտությամբ կարելի է ներկայացնել էլեկտրական ազդանշաններով, որտեղ 0–ն համապատասխանում է ցածր լարմանը, իսկ 1–ը՝ բարձր լարմանը:

2. Տրամաբանական փականներ և շղթաներ

Թվաբանական գործողություններն իրականացվում են տրամաբանական փականների միջոցով:

3. Կիսագումարիչ և ամբողջական գումարիչ

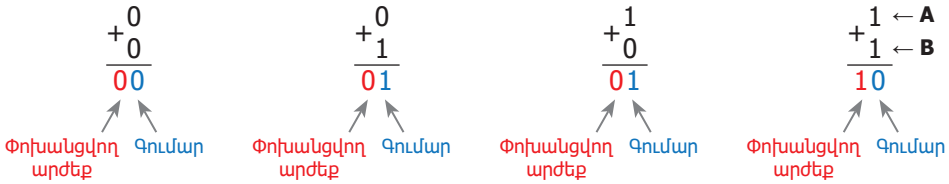
Կիսագումարիչները և ամբողջական գումարիչները ապահովում են երկուական թվերի գումարումը:

ԿԻՍԱԳՈՒՄԱՐԻՉ

Կիսագումարիչը տրամաբանական բարդ շղթա է, որն օգտագործվում է երկու երկուական թվերի գումարման համար: Մուտքում ունենալով երկուական արժեքները՝ ելքում ստացվում են գումարը և փոխանցվող արժեքը:

Վերհիշենք, թե ինչպես է կատարվում երկուական թվերի գումարումը:

Մեկ բիթ երկուական թվերի գումարի հաշվարկն ունի հետևյալ տեսքը.



Կարմիրով նշված արժեքները գումարման արդյունքում փոխանցվող արժեքներն են: Ինչպես երևում է, մուտքում երկու 1 ունենալու դեպքում է փոխանցվող արժեքը ստացվում 1:

Գործողության արդյունքում ստացված գումարները համապատասխանում են XOR տրամաբանական փականի ելքային արժեքներին:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Փոխանցվող արժեքները համապատասխանում են AND տրամաբանական փականի արժեքներին:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

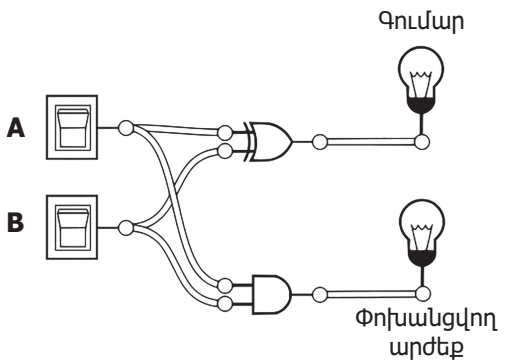
Ուստի կարող ենք ասել, որ կիսագումարիչը բաղկացած է երկու փականից.

- Գումար՝ A XOR B:
- Փոխանցվող արժեք՝ A AND B:

Կիսագումարիչի **իսկության աղյուսակը** կունենա այսպիսի տեսք.

A	B	A XOR B (գումար)	A AND B (փոխանցվող արժեք)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Ստորև բերված սխեմայում ցուցադրված է կիսագումարիչի տրամաբանական շղթան, որը ստեղծվել է https://logic.ly/demo/ հարթակում:



Կիսագումարիչի հիմնական նպատակը երկու՝ մեկ բիթ երկուական թվերի գումարումն է՝ հաշվի չառնելով նախորդ քայլից փոխանցված արժեքը: Ուստի կիսագումարիչը միայնակ չի կարող հանդես գալ, և դրա հետևանքով կիրառվում է ամբողջական գումարիչ:

ԱՄԲՈՂՋԱԿԱՆ ԳՈՒՄԱՐԻՉ

Ամբողջական գումարիչը կիրառվում է երկու բիթերի գումարման համար՝ հաշվի առնելով նաև գումարման նախորդ կարգից ստացված փոխանցված արժեքը: Ամբողջական գումարիչն ունի երեք մուտք և երկու ելք: Այն բազմաբիթանոց գումարիչների և այլ թվաբանական գործողություններ իրականացնող շղթաների հիմք է:

Ամբողջական գումարիչը գումարում է ոչ միայն A և B մուտքային բիթերը, այլև հաշվի է առնում նախորդ կարգից փոխանցված արժեքը (C*in*):

Օրինակ՝
$$\begin{array}{r} 11 \\ +\ 01 \\ \hline 100 \end{array}$$

Քայլ 1. 1 + 1 = 10, 0–ն գրվում է, իսկ 1–ը փոխանցվում է հաջորդ կարգ:

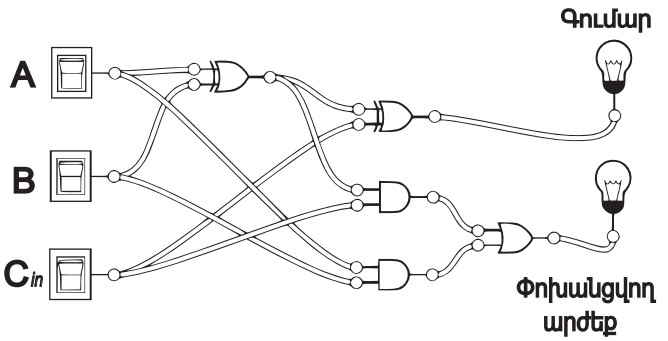
Քայլ 2. 1 + 0 = 1, որին ավելացնում ենք նախորդ կարգից փոխանցված արժեքը՝ 1–ը: Ստացվում է՝ 1 + 0 + 1 = 10:

Հետևաբար ամբողջական գումարիչը հաշվի է առնում երեք մուտքային բիթերը և տրամադրում է երկու ելքեր՝ գումար և փոխանցվող արժեք.

Ամբողջական գումարիչի **իսկության աղյուսակն** ունի հետևյալ տեսքը.

A	**B**	**C***in*	**Գումար** **A** XOR **B** XOR **C***in*	**Փոխանցվող արժեք** (**A** AND **B**) + ((**A** XOR **B**) AND **C***in*)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Ստորև բերված սխեմայում ցուցադրված է ամբողջական գումարիչի շղթան https://logic.ly/demo/ կայքում:

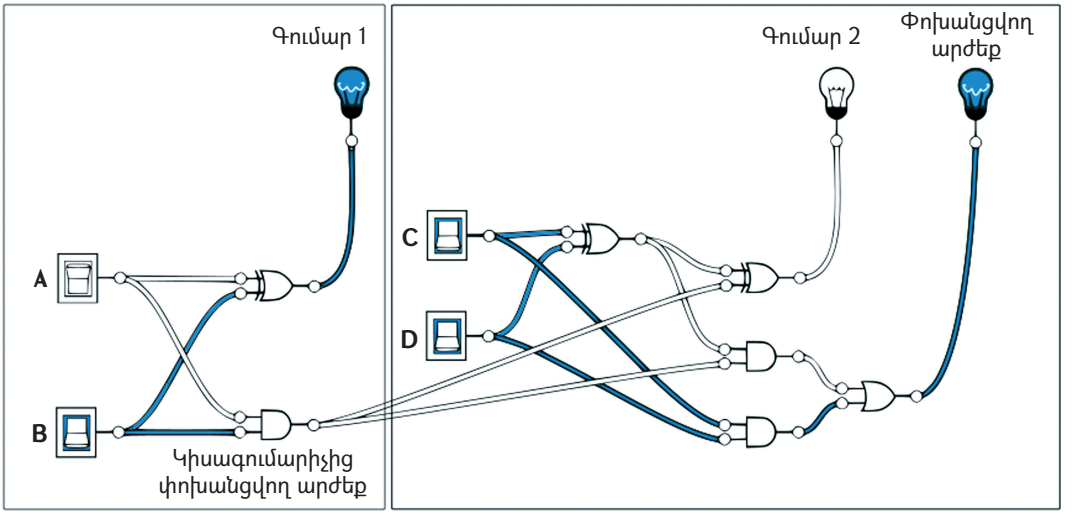


Խնդիր: Կառուցենք երկու բիթանոց ամբողջական գումարիչի սխեման հետևյալ երկուական թվերի գումարի համար.

$$\begin{array}{r} 10 \\ +\ 11 \\ \hline 101 \end{array}$$

Դրա համար կատարենք հետևյալ քայլերը.

1. Կառուցենք կիսագումարիչ և ամբողջական գումարիչ:
2. Միացնենք դրանք այնպես, որ կիսագումարիչը փոխանցի իր փոխանցվող արժեքը ամբողջական գումարիչին:
3. Կառուցենք այդ շղթայի իսկության աղյուսակը:



Կիսագումարիչ **Ամբողջական գումարիչ**

Իսկության աղյուսակը կունենա հետևյալ տեսքը.

A	B	Գումար 1	Կիսագումարիչից փոխանցվող արժեք	C	D	Գումար 2	Փոխանցվող արժեք
0	1	**1**	0	1	1	**0**	**1**

Վերը բերված օրինակում տրված շղթան կաշխատի բոլոր երկու բիթանոց երկուական թվերի գումարման համար, կտարբերվեն միայն իսկության աղյուսակները:

Ամփոփում

Բարդ տրամաբանական շղթաների կառուցման սկզբունքների ընկալումը կարևոր է համակարգչային գիտությունների հիմունքները հասկանալու համար, քանի որ տրամաբանական փականներն ու դրանց միջոցով իրականացվող գործողությունները հանդիսանում են թվաբանական հաշվողական համակարգերի ներքին կառուցվածքի հիմք: Կիսագումարիչները և ամբողջական գումարիչները հնարավորություն են տալիս համակարգիչներին իրականացնել երկուական թվերի գումարումը, ինչը կարևոր է ինչպես փոքրիկ միկրոսարքերի, այնպես էլ բարդ համակարգերի համար:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Որո՞նք են պարզ տրամաբանական փականները:
2. Ո՞ր շղթաներին ենք անվանում բարդ տրամաբանական շղթաներ:
3. Նկարագրե՛ք կիսագումարիչի աշխատանքի սկզբունքը:
4. Նկարագրե՛ք ամբողջական գումարիչի աշխատանքի սկզբունքը:
5. Կազմե՛ք հետևյալ մեկ բիթանոց գումարների կիսագումարիչների իսկության աղյուսակը և տրամաբական շղթան.

   ա) 1 + 0
   բ) 0 + 1
   գ) 1 + 1
6. Կազմե՛ք հետևյալ երկու բիթանոց գումարների իսկության աղյուսակները և տրամաբանական շղթանները:

   ա) 10 + 01
   բ) 01 + 01
   գ) 11 + 11
7. Կազմե՛ք հետևյալ երեք բիթանոց գումարի իսկության աղյուսակը և տրամաբանական շղթան.

   010 + 110

ԹԵՄԱ 6

ՑԱՆՑԻ ՀԱՂՈՐԴԱԿՑՄԱՆ ՀԻՄՈՒՆՔՆԵՐ

Ցանցի արդյունավետությունը անխուսափելիորեն ազդում է մեր ամենօրյա աշխատանքի, ուսման և հաղորդակցման կազմակերպման վրա: Ցանցի հաղորդակցությունը, որը ներառում է տվյալների փոխանցումը տարբեր հանգույցների միջոցով, կարևոր դեր ունի թվային աշխարհում: Այն պահանջում է, որ տվյալները փոխանցվեն արագ, ճշգրիտ և առանց ուշացումների կամ կորուստների:

Ուսումնասիրելով այս թեման՝

- Կսովորեք, թե ինչպես են ցանցի թողունակությունն ու հապաղումը ազդում տվյալների փոխանցման արագության և ճշգրտության վրա:
- Կիմանաք, թե որոնք են թողունակության և հապաղման հետ կապված ցանցի հիմնական խնդիրները, ինչպես հայտնաբերել դրանք:
- Կտիրապետեք այն հրամաններին, որոնք օգտագործվում են ցանցի խնդիրների հայտնաբերման համար:
- Կկարողանաք վերլուծել ստացված արդյունքները Ping և Tracert հրամանները կիրառելիս:

ԴԱՍ 6

ՑԱՆՑԻ ԱՐԴՅՈՒՆԱՎԵՏՈՒԹՅՈՒՆ

Ցանցի հաղորդակցումը տարբեր փոխկապակցված ցանցերի միջոցով տվյալների փոխանցման գործընթացն է: Ցանցերի համար կարևոր բնութագրիչ է **ցանցի արդյունավետությունը**: Այն ցույց է տալիս, թե ցանցում տվյալները որքան արագ, ճշգրիտ և առանց կորուստների կամ ուշացումների են փոխանցվում: Ցանցի արդյունավետության վրա ազդում է երկու հիմնական գործոն՝ թողունակությունը և հապաղումը:

ՑԱՆՑԻ ԹՈՂՈՒՆԱԿՈՒԹՅՈՒՆ

- **Թողունակությունը** մեկ վայրկյանում ցանցում փոխանցվող տվյալների քանակն է: Այն սովորաբար չափվում է **բիթ/վայրկյանով**, *օրինակ՝* **Մբիթ/վ** կամ

Գբիթ/վ: Այն ցույց է տալիս ցանցի՝ մեծածավալ տվյալների փոխանցման հնարավորությունը:

Թողունակության հետ կապված խնդիրներ կարող են առաջանալ մի շարք պատճառներով: Նշենք դրանցից մի քանիսը:

Ցանցի ծանրաբեռնվածություն: Երբ շատ օգտատերեր կամ սարքեր միաժամանակ օգտագործում են ցանցը: *Օրինակ*՝ բազմաթիվ սարքեր կարող են միաժամանակ օգտվել ինտերնետից՝ մեծ ֆայլեր ներբեռնելու կամ տեսակոնֆերանսներ վարելու համար:

Սարքավորումների թողունակության սահմանափակում: Երթուղիչները, ցանցի այլ սարքավորումները կարող են ունենալ սահմանափակ թողունակություն: *Օրինակ*՝ եթե ցանցը օգտագործում է 100 Մբիթ/վ թողունակությամբ երթուղիչ, բայց ցանցի ընդհանուր բեռնվածությունը գերազանցում է 100 Մբիթ/վ–ը:

Ցանցի կապի հեռավորություն: Մեծ հեռավորությունների վրա գտնվող ցանցերի հանգույցների միջև տվյալների փոխանցումը կարող է դժվարանալ:

Անլար ցանցերի խնդիրներ: Wi–Fi կապերում թողունակության վրա կարող են ազդել այնպիսի արտաքին գործոններ, ինչպիսիք են շենքի կառուցվածքը կամ Wi–Fi կապի շատ սարքերի միաժամանակյա օգտագործումը:

Սարքավորումների կամ ծրագրային ապահովման սխալներ: Ցանցային սարքավորումների ծրագրային սխալ կարգավորումները կամ սարքավորումների մաշվածությունը կարող են սահմանափակել ցանցի թողունակությունը:

Անհամապատասխանության խնդիրներ: Երբ ցանցում տարբեր սարքեր ունեն տարբեր թողունակություններ: *Օրինակ*՝ որոշ սարքեր աջակցում են 1 Գբիթ/վ, իսկ մյուսները՝ միայն 100 Մբիթ/վ, ապա ցանցի ընդհանուր թողունակությունը կարող է նվազել:

ՑԱՆՑԻ ՀԱՊԱՂՈՒՄ

Ցանցի հապաղումը այն ժամանակահատվածն է, որը պահանջվում է փաթեթները ցանցի միջոցով մի հանգույցից մյուսին փոխանցելու համար: Բարձր հապաղումը կարող է առաջացնել կապի դանդաղում, որը հատկապես նկատելի է իրական ժամանակում տվյալների փոխանակման դեպքում, *օրինակ*՝ ձայնային զանգերի, տեսազանգերի, առցանց խաղերի և հեռավոր սերվերների հետ աշխատելու ընթացքում:

Ցանցի հապաղումներ կարող են առաջանալ մի քանի պատճառով.

Ֆիզիկական հեռավորություն: Տվյալների փոխանցումը մեծ հեռավորություննե–րի վրա ավելի շատ ժամանակ է պահանջում:

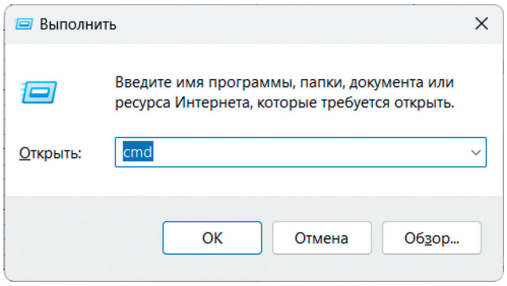
Ցանցի ծանրաբեռնվածություն: Երբ ցանցում շատ օգտատերեր կամ սարքեր են միացված և օգտագործում են մեծ քանակությամբ տվյալներ, դա կարող է «խցա–նում» առաջացնել և մեծացնել հապաղումը:

Սարքավորումների որակը: Ցանցի սարքավորումների վատ որակը կարող է դանդաղեցնել ցանցի աշխատանքը:

Վատ որակի համացանցային մատակարարում: Համացանցի մատակարարը կարող է տեխնիկական խնդիրներներ ունենալ իր ենթակառուցվածքում, որն էլ մե–ծացնում է հապաղումը:

Ցանցի թողունակության և հապաղման խնդիրների հայտնաբերումը կարող է իրականացվել մի շարք գործիքներով և հրամաններով, որոնք թույլ են տալիս մշտա–դիտարկել ցանցը, վերլուծել և գնահատել ցանցի արդյունավետությունը:

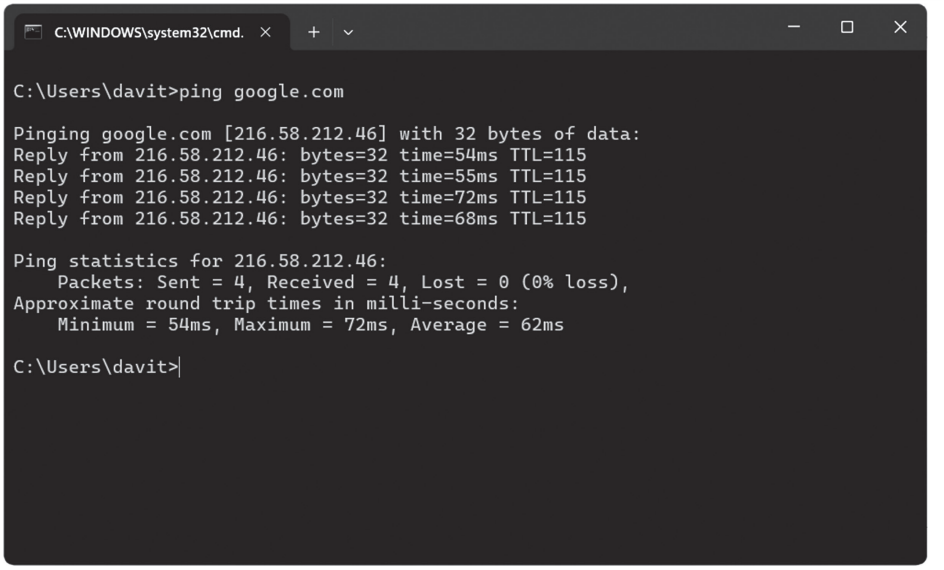
Ցանցի խնդիրները ստուգվում են համակարգչի տերմինալի միջոցով, որը բացե–լու համար ստեղնաշարից միաժամանակ սեղմում ենք Windows + R ստեղները: Բաց–ված պատուհանում ներմուծում ենք cmd հրահանգը և սեղմում OK կոճակը:



Թողունակության և հապաղման խնդիրները հայտնաբերելու գործիքներ

Ping հրամանը օգտագործվում է ցանցի հանգույցների միջև կապի ստուգման և հապաղման չափման համար:

Օրինակ՝ տերմինալում ներմուծելով ping google.com հրահանգը՝ կտեղեկա–նանք, թե որքան ժամանակ է պահանջվում տվյալների փաթեթը համակարգչից google.com հասցեին ուղարկելու և պատասխան ստանալու համար:



Վերլուծենք ստացված արդյունքը.

1. Reply from 216.58.212.46

Նշանակում է, որ համակարգիչը կապ է հաստատել տվյալ IP հասցեի հետ և փաթեթը հաջողությամբ հասել է նպատակակետին և ստացվել է պատասխան:

Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

Ցույց է տալիս, որ ուղարկվել է 4 փաթեթ և 4 պատասխան է ստացվել, կորուստը՝ 0 է:

2. bytes=32

Սա ցույց է տալիս, թե որքան տվյալ է ուղարկվել մեկ փաթեթի մեջ: Տվյալ դեպքում՝ 32 բայթ:

3. time=61ms

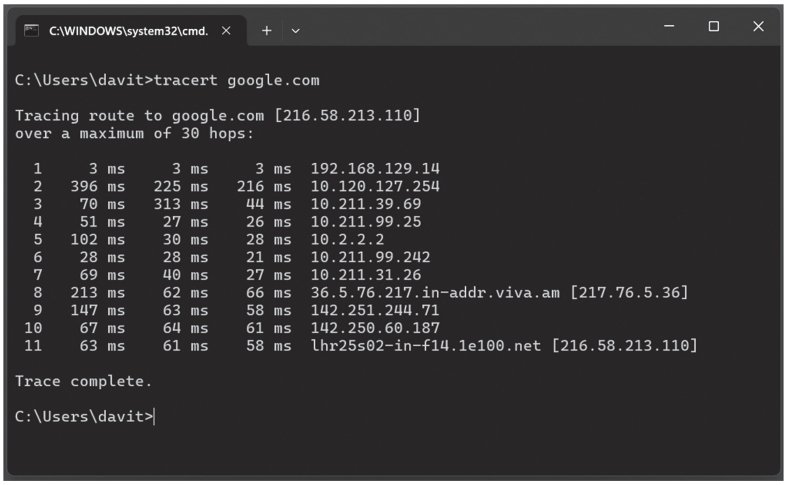
Սա միլիվայրկյաններով չափվող այն ժամանակն է, որն անհրաժեշտ է փաթեթի ուղարկման և պատասխանը ստանալու համար: Այս տվյալը ցույց է տալիս ցանցի հապաղումը.

- **Ավելի փոքր ժամանակը** ցույց է տալիս, որ կապն ավելի արագ է:
- **Մեծ ժամանակը** նշանակում է, որ կապը դանդաղ է, կամ ցանցում կան հապաղումներ:

4. TTL=56.

TTL–ը ցույց է տալիս, թե քանի հանգույցներով է անցել փաթեթը նպատակակետին հասնելու համար: Այն օգնում է վերլուծելու ցանցի երթուղին:

Հաջորդ հրամանը **Tracert**–ն է, որը ցույց է տալիս, թե տվյալները քանի հանգույցով են անցնում մինչև իրենց նպատակակետին հասնելը: Արդյունքներից կարող ենք բացահայտել, թե որտեղ են առաջանում թողունակության խնդիրները և հապաղումները: Տերմինալում ներմուծելով **tracert google.com** հրահանգը՝ կտեսնենք, թե մեր սարքից փաթեթները google.com հասցե հասնելու համար քանի հանգույցով են անցնում, և որքան է հապաղումը:



Կատարենք արդյունքների վերլուծություն.

Հանգույց 1

– 192.168.129.14

– Ժամանակը 3 ms:

– Սա առաջին հանգույցն է՝ ձեր տեղական ցանցի սարքավորումը: Հապաղումը շատ փոքր է, որը ցույց է տալիս, որ ներքին ցանցում խնդիր չկա:

Հանգույց 2

– 10.120.127.254

– Ժամանակը 396 ms, 225 ms, և 216 ms:

– Այստեղ հապաղումը շատ մեծ է, որը կարող է ցույց տալ ծանրաբեռնվածություն կամ ցանցի արդյունավետության խնդիր: Սա կարող է կապված լինել ձեզ համացանցային ծառայություն մատուցողի հետ:

Հանգույց 3

– 10.211.39.69

– Ժամանակը 70 ms, 313 ms, և 44 ms:

– Ժամանակի փոփոխությունները մեծ են, որը կարող է ցույց տալ ցանցի փոփո–խական ծանրաբեռնվածություն:

Հանգույց 4

– 10.211.99.25

– Ժամանակը 51 ms, 27 ms, և 26 ms:

– Հապաղումը փոքրացել է և մոտ է նախկին հանգույցներին, որը ցույց է տալիս որոշակի կայունացում:

Հանգույց 5

– 10.2.2.2

– Ժամանակը 102 ms , 30 ms , և 28 ms :

– Այս հանգույցում ժամանակը կրկին աճել է, որը կարող է ցանցի ծանրաբեռն–վածության նշան լինել:

Հանգույց 6

– 10.211.99.242

– Ժամանակը 28 ms, 28 ms, և 21 ms:

– Ժամանակի տարբերությունը փոքր է, որը նշանակում է, որ ցանցը գործում է համեմատաբար կայուն:

Հանգույց 7

– 10.211.31.26

– Ժամանակը 69 ms, 40 ms, և 27 ms:

– Ժամանակի տարբերությունները կրկին մեծանում են, որը կարող է ցույց տալ փոքր հապաղում:

Հանգույց 8

– 36.5.76.217.in–addr.viva.am [217.76.5.36]

– Ժամանակը 213 ms , 62 ms , և 66 ms :

– Սա համացանցային հանգույցի հասցե է և կարող է լինել միջազգային ցանցի հանգույց: Հապաղումը մեծ է, ինչը կարող է հանդիպել միջազգային ցանցի կապերի դեպքում:

Հանգույց 9

– 142.250.244.71

– Ժամանակը 147 ms , 63 ms , և 58 ms :

– Սա Google–ի ցանցի ենթակառուցվածքի հասցե է: Ժամանակը կայունացել է՝ ցածր հապաղումով:

Հանգույց 10

– 142.250.60.187

– ժամանակը 67 ms , 64 ms , և 61 ms :

– Սա ևս մի հանգույց է Google–ի ենթակառուցվածքում, որտեղ կապը լավ վիճակում է:

Հանգույց 11

– 1hr25s02–in–f14.1e100.net [216.58.213.110]

– ժամանակը 63 ms , 61 ms , և 58 ms :

– Վերջնական հանգույցը՝ Google–ի սերվերը, որտեղ կապը կայուն է, իսկ հապաղումը՝ ցածր:

Տերմինալի միջոցով կատարվող հրահանգները պահանջում են ավելի մասնագիտացված մոտեցում, որը ոչ բոլորին է հասանելի: Սակայն կան բազմաթիվ բջջային և համակարգչային գործիքներ, որոնք ավելի պարզ են օգտագործման համար և ցուցադրում են թողունակության վիճակագրությունը՝ գրաֆիկական միջերեսի միջոցով:

Նման գործիքներից են՝ SolarWinds Bandwidth Analyzer, Paessler PRTG Network Monitor, Nagios և այլն:

Ցանցի թողունակությունն ու հապաղումը փոխկապակցված են, բայց դրանք տարբեր գործոններ են, որոնք միասին ազդում են ցանցի արդյունավետության վրա: Թողունակությունը չափում է, թե որքան տվյալ կարող է փոխանցվել միավոր ժամանակում, իսկ հապաղումը՝ որքան արագ են տվյալները հասնում նպատակակետին: Բարձր թողունակությունը չի նշանակում ցածր հապաղում, և ցածր հապաղումը չի երաշխավորում մեծ տվյալների արագ փոխանցում: Հաճախ ցանցի լավ աշխատանքը պահանջում է և՛ բարձր թողունակություն, և՛ ցածր հապաղում՝ ապահովելով արագ և հուսալի կապ:

Ամփոփում

Ցանցի արդյունավետությունը ապահովելու համար կարևոր է հասկանալ, թե ինչ գործոններ են ազդում տվյալների փոխանցման արագության և ճշգրտության վրա: Ցանցի թողունակությունը և հապաղումը երկու կարևոր չափանիշներ են, որոնք ներկայացնում են տվյալների փոխանցման արագությունը: Ցանցի լավ աշխատանքը պահանջում է ոչ միայն բարձր թողունակություն, այլ նաև ցածր հապաղում՝ այդպիսով ապահովելով արագ, վստահելի և արդյունավետ կապ:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ է ցանցի թողունակությունը:
2. Ի՞նչ է ցանցի հապաղումը:
3. Թվարկե՛ք ցանցի թողունակության և հապաղման խնդիրների մի քանի պատճառներ:
4. Ինչպե՞ս են փոխկապակցված թողունակությունը և հապաղումը:
5. Ի՞նչ գործիքների միջոցով է հնարավոր ստուգել ցանցի թողունակությունը և հապաղումը:

Գործնական աշխատանք

- *Օգտագործելով ping և tracert գործիքները՝ չափե՛ք հետևյալ հանգույցների թողունակությունը և հապաղումը.*

  kznakhimnadram.blogspot.com

  chatgpt.com

  escs.am

  youtube.com

Վերլուծե՛ք տվյալները՝ դուրս բերելով ցանցի խնդիրներ ունեցող հանգույցները: Բացատրե՛ք արդյունքները:

ԹԵՄԱ 7

ԲԱՐԴ ՏՎՅԱԼՆԵՐ, ՊԱՐԶ ՆԵՐԿԱՅԱՑՈՒՄ

Ժամանակակից տեղեկատվական տեխնոլոգիաները հնարավորություն են տալիս հավաքագրել, պահպանել և վերլուծել մեծածավալ տվյալներ՝ տարբեր ոլորտներում արդյունավետ որոշումներ կայացնելու համար: Մեծ տվյալների վերլուծությունը լայնորեն կիրառվում է բիզնեսում, առողջապահությունում, կառավարությունում և այլ համակարգերում: Այս թեմայում կուսումնասիրենք, թե ինչպես կարելի է մեծածավալ տվյալների հետ աշխատել Microsoft Excel ծրագրի միջոցով: Ուսումնասիրությունը կենտրոնանալու է այնպիսի գործիքների վրա, որոնք օգնում են տվյալները զտել, կարգավորել, կրկնությունները հայտնաբերել և ներկայացնել արդյունավետորեն:

Ուսումնասիրելով այս թեման՝

- Կնկարագրեք մեծածավալ տվյալների կիրառությունը տարբեր ոլորտներում:
- Կօգտագործեք Excel–ի տվյալների զտման Filter գործիքը:
- Կկիրառեք կարգավորման գործիքները՝ տվյալները աճման, նվազման կամ այբբենական կարգով դասավորելու նպատակով:
- Կկարողանաք հայտնաբերել և անհրաժեշտության դեպքում հեռացնել կրկնվող տվյալները:
- Կվերլուծեք տվյալները՝ ելնելով դրանց պարզեցված և խմբավորված ներկայացումից:

ԳԼՈՒԽ 7

ՄԵԾԱԾԱՎԱԼ ՏՎՅԱԼՆԵՐՆ ԱՂՅՈՒՍԱԿՈՒՄ

Աշխատանք մեծածավալ տվյալների հետ

Ջարգացող տեղեկատվական տեխնոլոգիաները թույլ են տալիս չափազանց մեծ մասշտաբների հասնող տվյալների հավաքագրում, պահպանում և վերլուծություն: Դրանք մեծածավալ տվյալներ են, որոնք կարող են կիրառվել կազմակերպություններից տեղեկություններ ձեռք բերելու և իրենց գործունեության վերաբերյալ որոշումներ կայացնելու համար:

Օրինակ՝ բիզնեսը կարող է տվյալների վերլուծության շնորհիվ մշակել հաճախորդների նախասիրությունների վրա հիմնված բիզնես ռազմավարություններ: Առողջապա-

հական համակարգերի մեծ տվյալները կարող են օգտագործվել հիվանդությունների ընդհանուր ախտանիշեր գտնելու և արդյունավետ բուժման եղանակներ առաջարկելու համար: Կառավարությունները կարող են օգտագործել երթևեկության տվյալները՝ նոր ճանապարհներ նախագծելու կամ երթևեկությունը կանոնակարգելու համար:

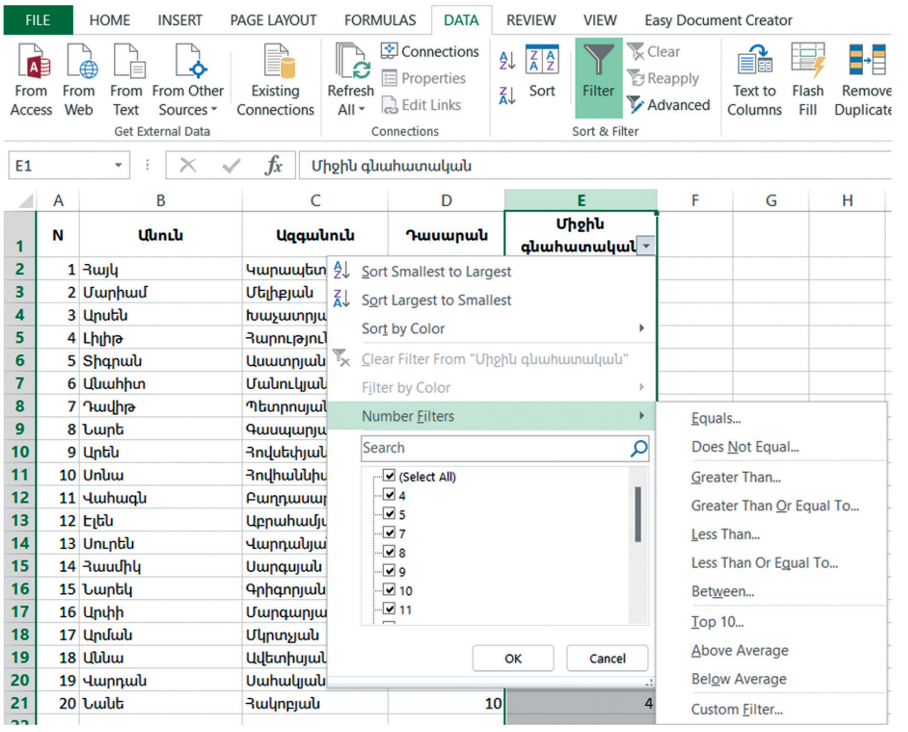
Մեծածավալ տվյալներ կարող են ստացվել տարբեր աղբյուրներից, *օրինակ*՝ հարցումներից, վաճառքի տվյալներից և այլն:

Օգտակարությանը զուգահեռ կան նաև մեծածավալ տվյալների հետ աշխատելու, վերլուծություններ կատարելու դժվարություններ: Մենք կուսումնասիրենք մեծածավալ տվյալների հետ աշխատելու էլեկտրոնային աղյուսակների որոշ հնարավորություններ, ինչպիսիք են՝

- **տվյալների զտումը,**
- **կարգավորումը,**
- **կրկնությունների հայտնաբերումը:**

Զտում

Զտման միջոցով կարելի է ըստ սահմանված պայմանների դուրս բերել անհրաժեշտ տվյալները: Microsoft Excel ծրագրում ընտրված տվյալների զտման համար նախատեսված է Data հրամանային ներդիրի Filter գործիքը.

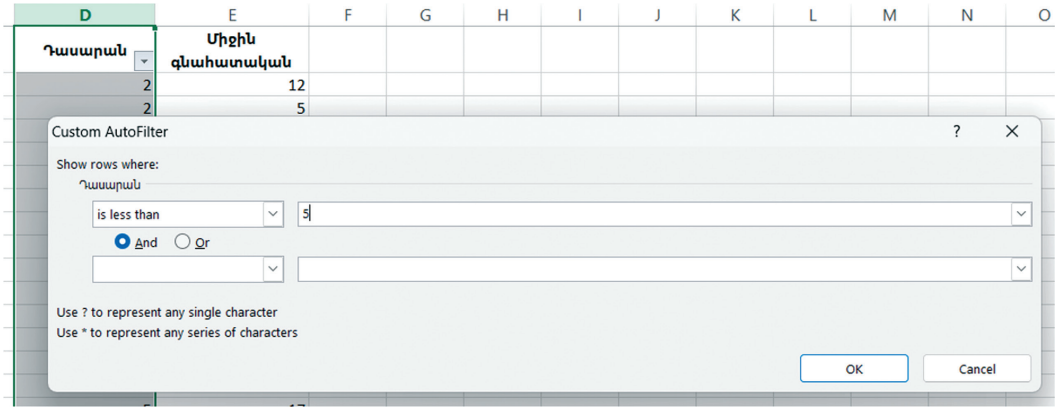


Filter գործիքը ունի տվյալների զտման տարբեր հնարավորություններ: Օրինակ՝ թվային արժեքների համար առաջարկում է զտման հետևյալ տարբերակները.

Equals...	Հավասար է...
Does Not Equal...	Հավասար չէ...
Greater Than...	Մեծ է, քան...
Greater Than Or Equal To...	Մեծ է կամ հավասար...
Less Than...	Փոքր է, քան...
Less Than Or Equal To...	Փոքր է կամ հավասար ...
Between...	Միջև...
Top 10...	Լավագույն տասնյակ...
Above Average	Միջինից բարձր...
Below Average	Միջինից ցածր...
Custom Filter...	Հատուկ զտիչ

Բերենք օրինակ, որտեղ տարբեր դասարաններում սովորողների ցուցակում տվյալները զտվում են ***Less Than...*** տարբերակով այնպես, որ ցուցադրվեն միայն տարրական դասարաններում սովորողների տվյալները:

Հրամանն ընտրելուց հետո բացված պատուհանում ներմուծում ենք անհրաժեշտ արժեքը՝ 5–ը:



Զտումից հետո կցուցադրվեն միայն 5–ից փոքր դասարանները, այսինքն՝ այն տվյալները, որոնք մեզ անհրաժեշտ են:

D1 | Դասարան

	A	B	C	D	E	F
1	N	Անուն	Ազգանուն	Դասարան	Միջին գնահատական	
2	1	Հայկ	Կարապետյան	2	12	
3	2	Մարիամ	Մելիքյան	2	5	
13	12	Էլեն	Աբրահամյան	3	16	
14	13	Սուրեն	Վարդանյան	4	19	
15	14	Հասմիկ	Սարգսյան	4	18	
24						

Կարգավորում

Կարգավորումը թույլ է տալիս տվյալները կարգավորել ըստ նախընտրած չափանիշի: Microsoft Excel ծրագրում ընտրված թվային տվյալները կարելի է կարգավորել աճման կամ նվազման կարգով, իսկ տեքստային տվյալները այբբենական կարգով՝ Data հրամանային ներդիրի հետևյալ գործիքների օգնությամբ.

Ընդ որում, տվյալ գործիքները կարգավորում են ոչ միայն թվային տվյալները:

Նշված տվյալների կարգավորման ժամանակ գործիքն առաջարկում է 2 տարբերակ.

	A	B	C	D	E	F	G	H	I
1	N	Անուն	Ազգանուն	Դասարան	Միջին գնահատական				
2	1	Հայկ	Կարապ						
3	2	Մարիամ	Մելիքյ						
4	3	Արսեն	Խաչատ						
5	4	Լիլիթ	Հարությ						
6	5	Տիգրան	Ասատր						
7	6	Անահիտ	Մանուկ						
8	7	Դավիթ	Պետրոս						
9	8	Նարե	Գասպա						
10	9	Արեն	Հովսեփ						
11	10	Սոնա	Հովհաննիսյան	9	20				
12	11	Վահագն	Բաղդասարյան	6	14				
13	12	Էլեն	Աբրահամյան	3	16				
14	13	Սուրեն	Վարդանյան	4	19				
15	14	Հասմիկ	Սարգսյան	4	18				
16	15	Նարեկ	Գրիգորյան	5	17				
17	16	Արփի	Մարգարյան	7	8				
18	17	Արման	Մկրտչյան	7	9				
19	18	Աննա	Ավետիսյան	7	11				
20	19	Վարդան	Սահակյան	10	10				
21	20	Նանե	Հակոբյան	10	4				
22									

Sort Warning

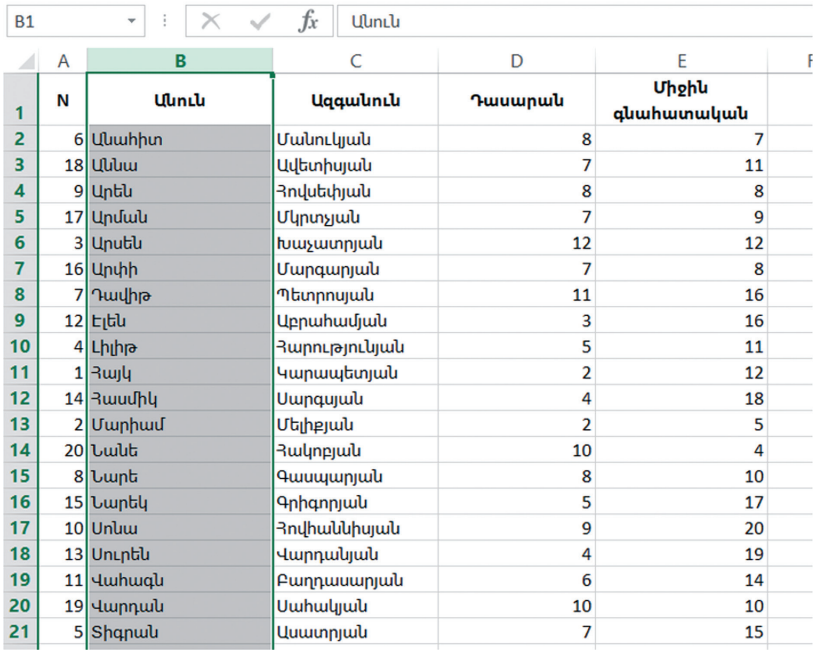
Microsoft Excel found data next to your selection. Since you have not selected this data, it will not be sorted.

What do you want to do?

- Expand the selection
- Continue with the current selection

Sort | Cancel

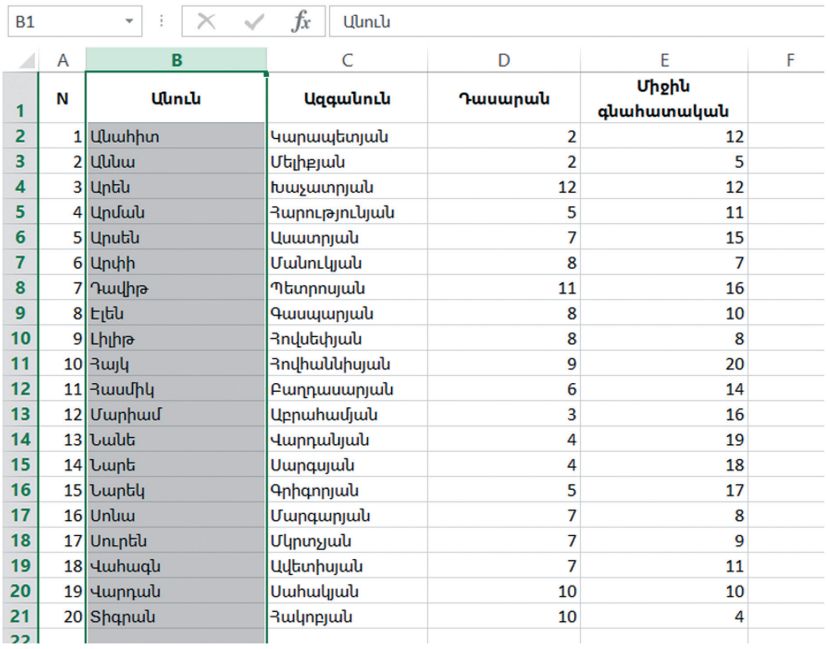
- **Expand the selection** — երբ տվյալների կարգավորման ժամանակ կատարվող փոփոխությունը տարածվում է ողջ աղյուսակի վրա:



B1 | Անուն

	A	B	C	D	E
1	N	Անուն	Ազգանուն	Դասարան	Միջին գնահատական
2	6	Անահիտ	Մանուկյան	8	7
3	18	Աննա	Ավետիսյան	7	11
4	9	Արեն	Հովսեփյան	8	8
5	17	Արման	Մկրտչյան	7	9
6	3	Արսեն	Խաչատրյան	12	12
7	16	Արփի	Մարգարյան	7	8
8	7	Դավիթ	Պետրոսյան	11	16
9	12	Էլեն	Աբրահամյան	3	16
10	4	Լիլիթ	Հարությունյան	5	11
11	1	Հայկ	Կարապետյան	2	12
12	14	Հասմիկ	Սարգսյան	4	18
13	2	Մարիամ	Մելիքյան	2	5
14	20	Նանե	Հակոբյան	10	4
15	8	Նարե	Գասպարյան	8	10
16	15	Նարեկ	Գրիգորյան	5	17
17	10	Սոնա	Հովհաննիսյան	9	20
18	13	Սուրեն	Վարդանյան	4	19
19	11	Վահագն	Բաղդասարյան	6	14
20	19	Վարդան	Սահակյան	10	10
21	5	Տիգրան	Ասատրյան	7	15

- **Continue with the current selection** — երբ տվյալների կարգավորման ժամանակ կատարվող փոփոխությունը տարածվում է միայն նշված սյան վրա:



B1 | Անուն

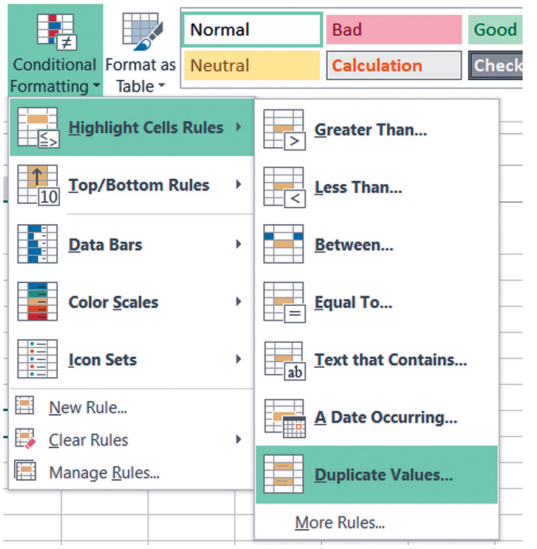
	A	B	C	D	E
1	N	Անուն	Ազգանուն	Դասարան	Միջին գնահատական
2	1	Անահիտ	Կարապետյան	2	12
3	2	Աննա	Մելիքյան	2	5
4	3	Արեն	Խաչատրյան	12	12
5	4	Արման	Հարությունյան	5	11
6	5	Արսեն	Ասատրյան	7	15
7	6	Արփի	Մանուկյան	8	7
8	7	Դավիթ	Պետրոսյան	11	16
9	8	Էլեն	Գասպարյան	8	10
10	9	Լիլիթ	Հովսեփյան	8	8
11	10	Հայկ	Հովհաննիսյան	9	20
12	11	Հասմիկ	Բաղդասարյան	6	14
13	12	Մարիամ	Աբրահամյան	3	16
14	13	Նանե	Վարդանյան	4	19
15	14	Նարե	Սարգսյան	4	18
16	15	Նարեկ	Գրիգորյան	5	17
17	16	Սոնա	Մարգարյան	7	8
18	17	Սուրեն	Մկրտչյան	7	9
19	18	Վահագն	Ավետիսյան	7	11
20	19	Վարդան	Սահակյան	10	10
21	20	Տիգրան	Հակոբյան	10	4

Կրկնությունների հայտնաբերում, հեռացում

Մեծածավալ տվյալների մեջ հաճախ հանդիպող խնդիրներից է տվյալների կրկնությունը: Դա կարող է բերել ավելորդ ծանրաբեռնվածության և վերլուծություննե–րի անճշտության: Ուստի կարևոր է գտնել և հեռացնել այդ կրկնությունները:

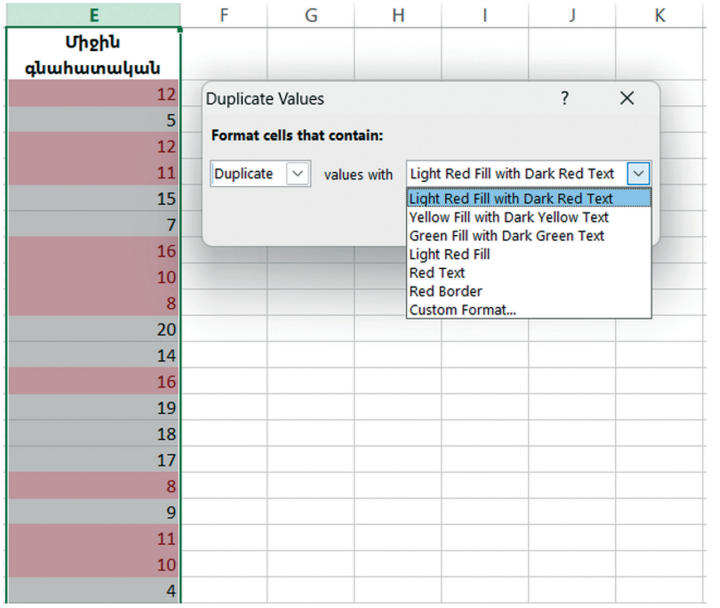
Data հրամանային ներ–դիրի **Remove Duplicates** գործիքը օգնում է հայտնա–բերելու կրկնությունները և ջնջել դրանք:

Սակայն միշտ չէ, որ կրկնվող տվյալ–ները անհրաժեշտ է ջնջել: Երբեմն դրանք անհրաժեշտ է գտնել և առանձ–նացնել՝ հետագայում մշակելու համար:

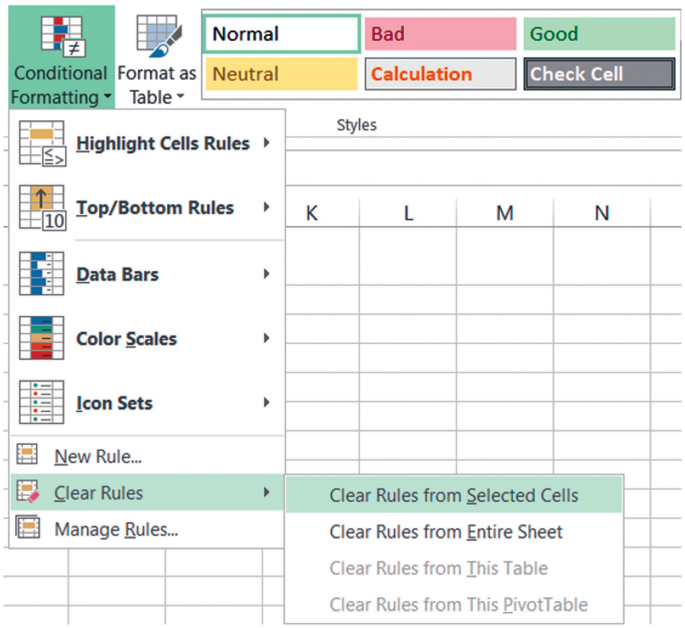


Home հրամանային ներդիրի **Conditional Formatting** հրամանի **Highlight Cells Rulers** խմբի **Dublicate Values** գործիքի օգնությամբ հնարա–վոր է ընտրված բջիջների համար գտնել և որոշակի գույնով նշել կրկնվող տվյալները:

Բերված օրինակում նշված են բոլոր այն գնահատականները, որոնք առնվազն մեկ անգամ կրկնվում են:



Home հրամանային ներդիրի **Conditional Formatting** հրամանացանկի **Clear Rules** գործիքով կարելի է հեռացնել կրկնություններ սահմանելը:



Տվյալների ներկայացում

Մեծածավալ տվյալների արդյունավետ ներկայացումը մեծ նշանակություն ունի վերլուծության ընթացքում: Հաճախ տվյալները հավաքագրվում և ներկայացվում են աղյուսակի տեսքով, սակայն նման աղյուսակները կարող են պարունակել հարյուրավոր կամ հազարավոր տողեր և սյուներ: Հետևաբար առաջանում է ծրագրի պատուհանում դրանց արդյունավետ ցուցադրելու անհրաժեշտություն:

Data հրամանային ներդիրի **Group** գործիքը նախատեսված է հենց այդ նպատակի համար: Այն խմբավորում է տվյալներն ըստ տողերի կամ սյուների և ստեղծում ենթացանկեր:

Դիտարկենք խանութում առկա ապրանքների ցուցակը: Դա բավականին մեծ տվյալների բազա է, որի հետ աշխատելու համար կարևոր է խմբավորել այդ տվյալները:

	A	B	C	D	E	F
1	**Տեսակը**	**Անվանումը**	**Ինքնարժեք**	**Վաճառքի գին**	**Եկամուտ**	**Մնացորդ**
2	**Հացաբուլկեղեն**	Լավաշ	100	150	50	20
3		Մատնաքաշ	120	180	60	15
4		Տարեկանի հաց	150	230	80	12
5		Կարկանդակ	180	250	70	8
6		Բուլկի	90	140	50	20
7	**Կաթնամթերք**	Կաթ	200	300	100	50
8		Մածուն	150	220	70	40
9		Թթվասեր	180	270	90	35
10		Կաթնաշոռ	220	320	100	25
11		Պանիր	200	280	80	20
12	**Բանջարեղեն**	Կարտոֆիլ	150	200	50	25
13		Կաղամբ	100	200	100	20
14		Գազար	200	350	150	10
15		Վարունգ	250	400	150	8

Խմբավորման նպատակով անհրաժեշտ է ընտրել խմբավորման ենթակա տողերը, *օրինակ*՝ 3–ից 6–րդ տողերը, և ընտրել Group հրամանը: Նման եղանակով խմբավորելով բոլոր ենթաբաժինները՝ կունենանք հետևյալ տեսքը.

1 2		A	B	C	D	E	F	G
	1	**Տեսակը**	**Անվանումը**	**Ինքնարժեք**	**Վաճառքի գին**	**Եկամուտ**	**Մնացորդ**	
	2	**Հացաբուլկեղեն**	Լավաշ	100	150	50	20	
+	7	**Կաթնամթերք**	Կաթ	200	300	100	50	
+	12	**Բանջարեղեն**	Կարտոֆիլ	150	200	50	25	
+	16							

Աղյուսակի ձախ մասում ավելացած + կամ − նշանների միջոցով կարելի է բացել խմբավորված ենթացանկը կամ նորից փակել: Այս կերպ մենք հնարավորություն ենք ունենում ցուցադրելու միայն տվյալ պահին անհրաժեշտ տվյալները:

1 2		A	B	C	D	E	F	G
	1	**Տեսակը**	**Անվանումը**	**Ինքնարժեք**	**Վաճառքի գին**	**Եկամուտ**	**Մնացորդ**	
	2	**Հացաբուլկեղեն**	Լավաշ	100	150	50	20	
+	7	**Կաթնամթերք**	Կաթ	200	300	100	50	
·	8		Մածուն	150	220	70	40	
·	9		Թթվասեր	180	270	90	35	
·	10		Կաթնաշոռ	220	320	100	25	
·	11		Պանիր	200	280	80	20	
−	12	**Բանջարեղեն**	Կարտոֆիլ	150	200	50	25	
+	16							

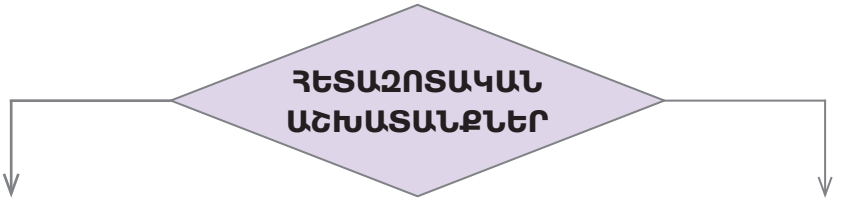
Մեծածավալ տվյալների հետ աշխատելու վերը թվարկված հնարավորությունները առաջին քայլերն են՝ տվյալների համակարգման, մշակման և վերլուծությունների իրականացման արդյունավետության համար:

Ամփոփում

Ժամանակակից աշխարհում տեղեկատվության ծավալն անընդհատ աճում է: Հասարակական, պետական, բիզնես և գիտական ոլորտներում կիրառվող տեղեկատվական տեխնոլոգիաները հնարավորություն են տալիս հավաքել, պահպանել և մշակել հսկայական քանակությամբ տվյալներ: Այսպիսի մեծածավալ տվյալների հետ արդյունավետ աշխատելու հմտությունները դառնում են պարտադիր ցանկացած մասնագիտական և ուսումնական ոլորտում:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ եք հասկանում՝ մեծածավալ տվյալներ ասելով:
2. Ինչո՞ւ է կարևոր տվյալների հավաքագրումը:
3. Ո՞ր ոլորտներում և ինչպե՞ս են կիրառվում տվյալները:
4. Ինչպե՞ս է իրականացվում տվյալների զտումը Microsoft Excel էլեկտրոնային աղյուսակում:
5. Ինչպե՞ս է իրականացվում տվյալների կարգավորումը Microsoft Excel էլեկտրոնային աղյուսակում:
6. Ինչպե՞ս է իրականացվում տվյալներում կրկնությունների հայտնաբերումը Microsoft Excel էլեկտրոնային աղյուսակում:
7. Ինչպե՞ս է իրականացվում տվյալների խմբավորումը Excel էլեկտրոնային աղյուսակում:



Ուսումնասիրեք Filter գործիքի հնարավորությունները տեքստային և ամսաթվային տվյալների համար: Կազմեք աղյուսակ՝ ներկայացնելով այդ հնարավորությունները:

Հետազոտական հարցեր.

– Ի՞նչ հնարավորություններ է տալիս Filter գործիքը տարբեր տվյալների համար:

– Ի՞նչ տարբերություններ կան թվային, տեքստային և ամսաթվային տվյալների ֆիլտրացման մեխանիզմների միջև:

– Ինչպե՞ս կարող է Filter-ը օգնել տվյալների վերլուծության արդյունավետությանը դպրոցական, համայնքային կամ անձնական նախագծերում:

Հետազոտական աշխատանքի գնահատման ռուբրիկ

N	Չափանիշ	Նկարագրություն	Միավոր
1	Ներածություն և թեմայի ըմբռնում	Բացատրված է, թե ինչ է Filter–ը, ինչ խնդիր է լուծում, ինչով է կարևոր տվյալների վերլուծության մեջ:	2 միավոր
2	Տեսական ուսումնասիրություն	Ուսումնասիրած են Filter գործիքի գործառույթները՝ տեքստային և ամսաթվային տվյալների դեպքում, ներկայացված են օրինակներ:	2 միավոր
3	Գործնական աշխատանք և աղյուսակ	Աշակերտը ստեղծել է փորձնական աղյուսակ և կիրառել տարբեր ֆիլտրեր՝ ճիշտ ձևով: Ներկայացված է համեմատական աղյուսակ:	3 միավոր
4	Վերլուծություն և եզրակացություն	Կատարել է տվյալների համեմատություն, քննարկել Filter–ի արդյունավետությունը, ներկայացրել հետևություններ:	2 միավոր
5	Ձևավորում և գրավոր մշակման որակ	Աշխատանքը կոկիկ է, գրագետ, տրամաբանական կառուցվածքով, բովանդակությունը համապատասխանում է առաջադրանքին:	1 միավոր

Գործնական աշխատանք

Microsoft Excel–ում կազմե՛ք հետևյալ աղյուսակը և լրացրե՛ք այն պատահական տվյալներով:

	A	B	C	D	E
1	**N**	**Անուն**	**Ազգանուն**	**Դասարան**	**Միջին գնահատական**
2					
3					
4					
5					
6					
7					
8					
9					

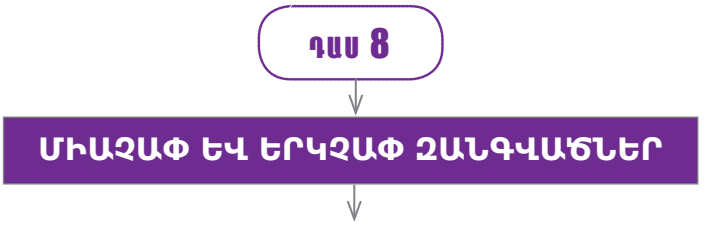
- *Կատարե՛ք ցուցակի կարգավորում:*
- *Ջոման միջոցով առանձնացրե՛ք դպրոցի գերազանց առաջադիմությամբ սովորողների ցուցակը:*
- *Խմբավորե՛ք տվյալներն ըստ դասարանների:*

ԹԵՄԱ 8 ՄԻԱՉԱՓ ԵՎ ԵՐԿՉԱՓ ՁԱՆԳՎԱԾՆԵՐ

Ծրագրավորման մեջ հաճախ աշխատում ենք ոչ միայն մեկ փոփոխականի, այլ տվյալների ամբողջ խմբերի հետ: Օրինակ՝ սովորողների գնահատականները, ջեր–մաստիճանի տվյալները, քննությունների միավորները և այլն: Այսպիսի տեղեկա–տվությունը պահպանելու և մշակելու համար կիրառվում են զանգվածները: Զանգված–ները հնարավորություն են տալիս նույն տիպի տվյալների հետ աշխատել ավելի հարմար ձևով:

Ուսումնասիրելով այս թեման՝

- Կսովորեք, թե ինչ է զանգվածը և ինչու է այն օգտակար ծրագրավորման մեջ:
- Կկարողանաք սահմանել և կիրառել միաչափ ու երկչափ զանգվածներ:
- Կհասկանաք, թե ինչպես կարելի է զանգվածի տարրերին դիմել դասիչներով:
- Կկարողանաք գրել պարզ ծրագրեր, որոնք կիրառում են զանգվածներ՝ տվյալների պահպանման ու մշակման համար:
- Կսովորեք արտածել զանգվածների տվյալները ցիկլերի միջոցով:



Ինչպես արդեն գիտեք, զանգվածը նույնատիպ տվյալների հավաքածու է, որը զբաղեցնում է հիշողության հաջորդական անընդհատ տարածք: Զանգվածները լի–նում են միաչափ, երկչափ, բազմաչափ: Զանգվածներում յուրաքանչյուր տարր ունի սեփական համարը, որը կոչվում է դասիչ, և C++–ում այդ տարրերը համարակալվում են՝ սկսած 0–ից:

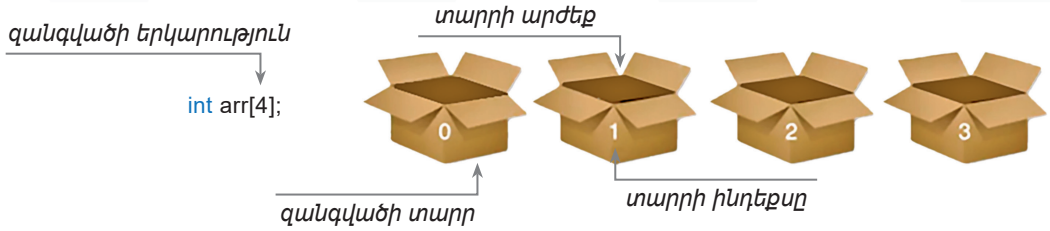
Միաչափ զանգված

C++–ում միաչափ զանգված կարող ենք հայտարարել հետևյալ կերպ.

<Փոփոխականի տիպ> <զանգվածի անուն>[զանգվածի երկարություն]

Օրինակ՝ double A[5];

int arr[4];



Նկարում ցույց է տրված ամբողջ տիպի չորս տարրերից բաղկացած զանգվածի հայտարարումը:

Զանգվածի տարրերը կարելի է սկզբնարժեքավորել անմիջապես հայտարարելու ընթացում:

Օրինակ՝

int arr[4] = {12, 6, 4, 2}; // հայտարարել ենք զանգված և սկզբնարժեքավորել:

Զանգվածի տարրերին դիմելու համար գրում ենք զանգվածի անունը և նշում դասիչը: Բերված օրինակում զանգվածի տարրերին կարող ենք դիմել այսպես՝ arr[0], arr[1], arr[2], arr[3]:

Զանգվածի որոշակի տարրի արժեքը կարող ենք փոփոխել՝ կիրառելով նրա դասիչը:

Օրինակ՝

int arr[5] = {10, 20, 30, 40, 50};

arr[2] = 35; // arr[2]–ի 30 արժեքը կփոխարինվի 35–ով:

Դիտարկենք դեպքեր, երբ տվյալները ներկայացվում են միաչափ զանգվածներով:

- **Սովորողների գնահատականներ**

Ենթադրենք՝ ցանկանում եք պահպանել մեկ առարկայի համար սովորողների գնահատականները: Այդ տվյալները կարելի է ներկայացնել grades անվանումով ամբողջ տիպի միաչափ զանգվածով:

Օրինակ՝

int grades[10] = {9, 8, 6, 7, 10, 9, 9, 5, 4, 5};

Զանգվածի յուրաքանչյուր տարր հասանելի է մեկ դասիչով, *օրինակ՝* grades[0] = 9, grades[1] = 8 և այլն:

- **Շաբաթվա օրերի ջերմաստիճանները**

Մեկ շաբաթվա ընթացքում օրվա միջին ջերմաստիճանները կարելի է ներկայացնել իրական տիպի միաչափ զանգվածով:

Օրինակ՝

float temperatures[7] = {19.5, 18.1, 21.2, 19.0, 22.5, 23.1, 24.3};

Դիտարկենք հետևյալ խնդիրը.

Հայտարարել միաչափ զանգված, որի յուրաքանչյուր տարր հարկավոր է մեծացնել 1–ով: Արտածել նոր արժեքներով զանգվածը:

Online GDB հարթակ

Ծրագիրը իրականացնենք https://www.onlinegdb.com/ հարթակում:

```
#include<iostream>
using namespace std;
int main()
{
    int n; // զանգվածի չափս
    int i; // ցիկլում կիրառվող հաշվիչ
    cout<<"Մուտքագրեք զանգվածի չափը ";
    cin>>n;
    int arr[n]; // Հայտարարում են միաչափ զանգված
    cout<<"Մուտքագրեք զանգվածի տարրերը :"<<endl;
    for(i=0;i<n;++i)
    {
        cin>>arr[i];
    }
        // Յուրաքանչյուր տարրի արժեքին ավելացնում ենք 1
    for(i=0;i<n;++i)
    {
        arr[i]=arr[i]+1;
    }
       // Արտածում ենք տարրերի նոր արժեքները
    for(i=0;i<n;++i)
    {
        cout<<arr[i]<<" ";
    }
}
```

```
Մուտքագրեք զանգվածի չափը 3
Մուտքագրեք զանգվ՞ծի տարրերը :
12 8 94
13 9 95

...Program finished with exit code 0
Press ENTER to exit console.
```

Երկչափ զանգված

C++–ում երկչափ զանգվածները հայտարարում ենք այսպես.

<Փոփոխականի տիպ> <զանգվածի անուն>[տողերի քանակ] [սյուների քանակ].

int numbers[3][2];

Երկչափ զանգվածը կարելի է հայտարարել՝ տալով սկզբնական արժեքներ:

Օրինակ՝

```
int numbers[3][2] = {
    {1, 2},
    {4, 5},
    {7, 8}
};
```

Եթե միաչափ զանգվածները նման են վանդակավոր տետրի մի տողին, ապա երկչափ զանգվածները համեմատում ենք տետրի էջի հետ:

Տողեր \ Սյուներ	0	1	2	3
0	12	0	7	9
1	3	27	0	7
2	15	21	5	6

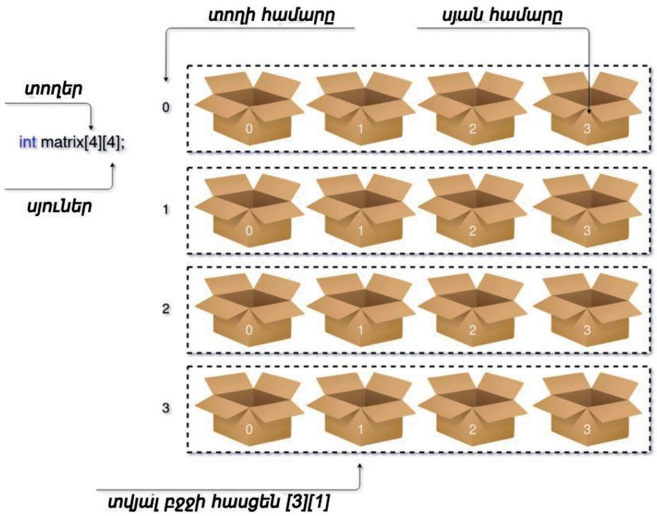
Այսպիսով՝ երկչափ զանգվածները կարող են ներկայացվել որպես աղյուսակ: Զանգվածի տարրի առաջին դասիչը տողի համարն է, իսկ երկրորդը՝ սյան համարը:

Օրինակ՝ A[3][4] զանգվածը բաղկացած կլինի 12 տարրից և կարող է ներկայացվել այսպես.

A[0][0]	A[0][1]	A[0][2]	A[0][3]
A[1][0]	A[1][1]	A[1][2]	A[1][3]
A[2][0]	A[2][1]	A[2][2]	A[2][3]

Երկչափ զանգվածի համարակալումը սկսվում է (0,0)–ից, որը ձախ վերևի անկյունն է, որին հաջորդում է (0,1) վանդակը՝ առաջին տողի երկրորդ վանդակը, և այդպես շարունակ:

Այսպիսով՝ համարակալում ենք **[տող][սյուն]** հերթականությամբ:



Երկչափ զանգվածի տարրերին դիմում են հետևյալ կերպ.

<Զանգվածի անուն> [տողի համար] [սյան համար]:

Օրինակ՝

numbers[0][0], numbers[2][1] և այլն:

Երկչափ զանգվածը անվանում են նաև մատրից:

Դիտարկենք օրինակ, երբ տվյալները ներկայացվում են երկչափ զանգվածով:

- **Սովորողների գնահատականներ տարբեր առարկաներից**

Եթե ցանկանում ենք պահպանել սովորողների գնահատականները տարբեր առարկաներից, ապա կարող ենք օգտագործել երկչափ զանգված:

Տողերը համապատասխանում են սովորողներին, իսկ սյուները՝ առարկաներին:

```
#include <iostream>
using namespace std;
int main()
{
int grades[3][3] = {
    {9, 8, 10}, // Սովորող 1–ի գնահատականները
    {6, 4, 7}, // Սովորող 2–ի գնահատականները
    {5, 6, 7} // Սովորող 3–ի գնահատականները
};

cout<< grades[2][0]; //Արտածում է 3–րդ սովորողի 1–ին առարկայի գնահատականը:
}
```

Այստեղ grades[0][0] տարրի արժեքը 9 է, որն առաջին սովորողի առաջին առարկայի գնահատականն է, իսկ grades[1][2] տարրի արժեքը 7 է՝ երկրորդ սովորողի երրորդ առարկայի գնահատականը:

Դիտարկենք հետևյալ խնդիրը.

Արտածել տրված երկչափ զանգվածի բոլոր տարրերը:

```
#include <iostream>
using namespace std;

int main()
{
   int rows = 3, columns = 2;
   int numbers[rows][columns] = { {1, 2}, {3, 4}, {5, 6} };
```

```
    for(int i=0; i < rows; i++)
    {
        for(int j=0; j < columns; j++)
        {
            cout << numbers[i][j] << "\t";
        }
        cout << endl;
    }
}
```

Ամփոփում

Այս թեման ցույց տվեց զանգվածների կիրառման կարևորությունը՝ երբ աշխատում ենք բազմաթիվ, փոխկապակցված տվյալների հետ: Միաչափ զանգվածները հարմար են գծային կառուցվածքով տվյալների համար, իսկ երկչափ զանգվածները՝ աղյուսակային տեսքով ներկայացված տվյալների համար: Նման կառուցվածքները հեշտացնում են տվյալների պահպանումը, ընթերցումը և վերլուծությունը: Ծրագրավորման խնդիրների լուծման ժամանակ զանգվածների ճիշտ կիրառումը մեծացնում է կոդի արդյունավետությունն ու պարզությունը:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ է զանգվածը: Ե՞րբ կարելի է կիրառել զանգվածներ:
2. Ի՞նչ տեսակի զանգվածներ կան:
3. Նկարագրե՛ք դեպքեր, երբ տվյալները ներկայացվում են միաչափ զանգվածով:
4. Նկարագրե՛ք դեպքեր, երբ տվյալները ներկայացվում են երկչափ զանգվածով:
5. Ինչպե՞ս է հայտարարվում միաչափ զանգվածը *C++*–ում:
6. Ինչպե՞ս է հայտարարվում երկչափ զանգվածը *C++*–ում:
7. Երկչափ զանգվածում ի՞նչ են ցույց տալիս առաջին և երկրորդ դասիչները:
8. Ինչպե՞ս դիմել երկչափ զանգվածի տարրերին:

Գործնական աշխատանք

1. Գրե՛ք ծրագիր, որը կհայտարարի տրված երկչափ զանգվածը և կարտածի 3–րդ տողի առաջին տարրը:

1	2	3
4	5	6
7	8	9
10	11	12

2. Գրե՛ք ծրագիր, որը կհայտարարի տրված 3×3 չափանի երկչափ զանգվածը և կարտածի առաջին և վերջին տարրերը:

17	25	3
4	0	6
21	8	–17

3. Տրված է հետևյալ ծրագրային հատվածը.

```
int matrix[2][2] = {{1, 2}, {3, 4}};
for(int i = 0; i < 2; i++) {
  for(int j = 0; j < 2; j++) {
```

```
        cout << matrix[i][j] << " ";
    }
    cout << endl;
}
```

- *Ինչպիսի՞ զանգված է հայտարարված:*
- *Ներկայացնել զանգվածը աղյուսակի տեսքով:*
- *Տվյալ ծրագրային կոդի արդյունքում ի՞նչ կարտածվի էկրանին:*

4. Գրե՛ք ծրագիր, որը կհաշվի և կարտածի տրված երկչափ զանգվածի յուրաքանչյուր տողի տարրերի գումարը:

1	2	3
4	5	6
7	8	9

Ստուգում. ելքային տվյալներ՝ 6, 15, 24

5. Գրե՛ք ծրագիր, որը կհաշվի և կարտածի տրված երկչափ զանգվածի յուրաքանչյուր սյան տարրերի գումարը:

1	2	3
4	5	6
7	8	9

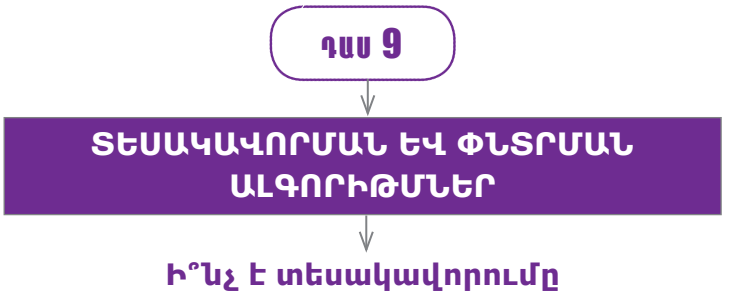
Ստուգում. ելքային տվյալներ՝ 12, 15, 18

ԹԵՄԱ 9 ՏԵՍԱԿԱՎՈՐՄԱՆ ԵՎ ՓՆՏՐՄԱՆ ԱԼԳՈՐԻԹՄՆԵՐ

Տվյալների տեսակավորման և որոնման ալգորիթմները կարևոր դեր են խաղում տարբեր ոլորտներում, հատկապես տվյալների մշակման և վերլուծության մեջ: Հաշվի առնելով, որ մեր միջավայրում մշտապես հանդիպում ենք մեծ ծավալի տվյալներ, դրանց ճիշտ տեսակավորումը և արդյունավետ որոնումը թույլ են տալիս հեշտությամբ իրականացնել տվյալների մշակման գործընթացները: Այդ նպատակով մշակվել են բազմաթիվ տեսակավորման և փնտրման ալգորիթմներ, որոնք տարբեր արդյունավետություն ունեն՝ կախված տվյալների քանակից, կառուցվածքից և օգտագործման նպատակից: Օրինակ՝ փոքր քանակությամբ տվյալների համար բավական են պարզ տեսակավորման ալգորիթմները, իսկ մեծածավալ տվյալների դեպքում անհրաժեշտ է ընտրել արագ գործող ալգորիթմներ: Այս թեմայում կսովորենք տարածված տեսակավորման և փնտրման եղանակները, դրանց տարբերությունները և կիրառման դեպքերը:

Ուսումնասիրելով այս թեման՝

- Կսովորեք, թե ինչ նպատակով են օգտագործվում տեսակավորման և փնտրման ալգորիթմները:
- Կկարողանաք գրել ծրագրեր, որոնք տեսակավորում են զանգվածի տարրերը կամ փնտրում են որոշակի արժեք տվյալների շարքում:
- Կսովորեք կիրառել հիմնական ալգորիթմներ և ծրագրային կոդով իրականացնել դրանք:



Ի՞նչ է տեսակավորումը

Տեսակավորումը զանգվածի տարրերի դասավորումն է ըստ որևէ կարգի:

Օրինակ՝ խանութում առևտուր անելիս բոլորս էլ նկատել ենք, որ ապրանքները դասավորված են ըստ տեսակի. մի ցուցափեղկում մսամթերքն է, մյուսում՝ կաթնամթերքը, երրորդում՝ քաղցրավենիքը, իսկ չորրորդում՝ միրգն ու բանջարեղենը:

Տվյալների տեսակավորումը կարևոր է նաև թվային աշխարհում: Առօրյա կյանքում բոլորս էլ առնչվում ենք նման տվյալների տեսակավորման հետ:

Օրինակ՝ առցանց առևտուր անելիս անհրաժեշտ ապրանքը փնտրելու համար մենք ընտրում ենք ապրանքի դասավորության որևէ պայման՝ այն, ինչը տվյալ պահին կարևոր է մեզ համար: Եթե կարևոր է, որ ապարանքը լինի հնարավորինս էժան, ապա առցանց խանութի կայքում ընտրում ենք ապրանքի դասավորվածության էժանից թանկ տարբերակը:

Ենթադրենք՝ բառարանում կա ավելի քան 10000 բառ: Բայց անհրաժեշտ բառը գտնելը մեզնից խլում է ընդամենը 1–2 րոպե: Դրա պատճառն այն է, որ բառարանում բառերը դասավորված են այբբենական կարգով, և մենք հստակ գիտենք, թե որ բառը որտեղ է պետք փնտրել:

Էլեկտրոնային բառարանները ևս կառուցված են նույն սկզբունքով, ինչպես թղթայինը: Ու եթե թղթային բառարանում բառը մենք ենք փնտրում, ապա էլեկտրոնային բառարանում մեր փոխարեն փնտրում է համակարգչային ծրագիրը: Որպեսզի համակարգչային ծրագիրը բառը շուտ գտնի, անհրաժեշտ է, որ էլեկտրոնային բառարանում բառերը նույնպես դասավորված լինեն:

Տեսակավորված տեղեկույթի առավելությունները

Արագ որոնում

Տեսակավորված տեղեկույթի հետ աշխատանքը կատարվում է արագ, քանի որ պահանջված տվյալը հայտնաբերվում է արագ:

Օրինակ՝ չտեսակավորված էլեկտրոնային բառարանում բառը փնտրելու համար պետք է հերթով դիտարկել բոլոր բառերը, մինչև փնտրվող բառին հանդիպելը:

Այբբենական կարգով տեսակավորված էլեկտրոնային բառարանում բառը փնտրելու համար առաջին հերթին դիտարկվում է ցանկի մեջտեղում գտնվող բառը: Եթե փնտրվող բառի առաջին տառը այբուբենում ավելի առաջ է, քան ցանկի մեջտեղում գտնվողինը, ապա ցանկի երկրորդ հատվածը այլևս չի դիտարկվում, և բառի հետագա փնտրումը կատարվում է նույն սկզբունքով առաջին հատվածում: Այդպիսով զգալիորեն կրճատվում է դիտարկումների քանակը:

Առավելագույն և նվազագույն արժեքների արագ դուրսբերում

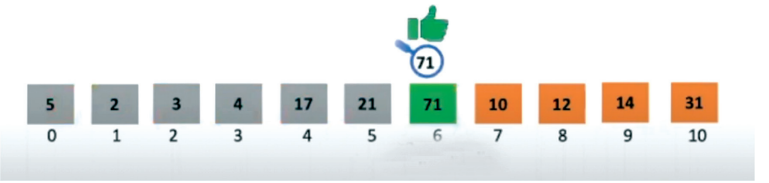
Օրինակ՝ ունենք թվերի բազմություն, և անհրաժեշտ է գտնել այդ բազմության ամենամեծ թիվը:

Չտեսակավորված թվային բազմության դեպքում պետք է հերթով դիտարկել բոլոր թվերը, համեմատել մյուսների հետ՝ ամենամեծը գտնելու համար:

Ըստ աճման կարգի տեսակավորված բազմությունում ամենամեծ թիվը շարքի ամենավերջին թիվն է:

Ինչպես երևում է թվարկված օրինակներից, տեսակավորված տվյալներով աշխատելն ավելի հեշտ է:

Գծային որոնում



Գծային որոնումը որոնման ամենապարզ ալգորիթմներից մեկն է, որն օգտագործվում է զանգվածում տարրը գտնելու համար: Այն աշխատում է հետևյալ սկզբունքով՝ տրված զանգվածի տարրերը մեկ առ մեկ համեմատվում են որոնվող արժեքի հետ:

Գծային որոնման ալգորիթմի քայլերը

1. Որոնումը սկսել զանգվածի առաջին տարրից, համեմատել այն որոնվող արժեքի հետ:
2. Շարունակել համեմատումը: Հերթով անցնել զանգվածի բոլոր տարրերով, մինչև գտնվի համապատասխան տարրը:

Գծային որոնման ալգորիթմը իրականացնենք C++–ում

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {2, 4, 0, 1, 9}; // Հայտարարում ենք զանգված
    int x = 4;     // Որոնվող արժեքը
    bool k = false;   //Հայտարարում ենք տրամաբանական տիպի փոփխական
    int n = sizeof(arr) / sizeof(arr[0]);   // Հաշվում ենք զանգվածի չափը

    for (int i = 0; i < n; i++)      // Դիտարկում ենք զանգվածի տարրերը
    {
        if (arr[i] == x)       // Համեմատում ենք զանգվածի տարրը որոնվող արժեքի հետ
        {
            cout<<" Տարրի ինդեքսը զանգվածում = "<< i << endl; // Եթե տարրը գտնվել է, արտածում ենք դրա դասիչը:

            k = true;    // Փոփոխականին վերագրում ենք true արժեք
        }
    }

    if (k == false)      // Ստուգում ենք, թե արդյո՞ք զանգվածում որոնվող արժեքը բացակայում է
    {
      cout<<" Տրված արժեքը զանգվածում չի գտնվել"; // Արտածում են արժեքի բացակայության մասին հաղորդագրություն

    }
}
```

```
input
Տարրի ինդեքսը զանգվածում = 1


...Program finished with exit code 0
Press ENTER to exit console.
```

Ինչպես է աշխատում կոդը

Տող 6. int array[] ── Մենք հայտարարում ենք ամբողջ թվային զանգված՝ 5 տարրերով: Այս զանգվածի արժեքներն են՝ {2, 4, 0, 1, 9}:

Տող 9. sizeof(array) / sizeof(array[0]) – Կոդի հատվածը հաշվում է զանգվածի չափը:

sizeof(array) ── Ցույց է տալիս int ամբողջ թվային տիպի 5–տարրանոց զանգվածի ընդհանուր չափը, որը կլինի 20 բայթ, քանի որ int տիպը զբաղեցնում է 4 բայթ հիշողության ծավալ:

sizeof(array[0]) ── Վերադարձնում է զանգվածի առաջին տարրի չափը բայթերով, այսինքն՝ 4:

sizeof(array) / sizeof(array[0]) հաշվարկի արդյունքում ստանում ենք 20 բայթ / 4 բայթ = 5 բայթ: 5–ը մեր զանգվածի չափն է:

Տող 13. if պայմանը ճիշտ լինելը նշանակում է, որ զանգվածում որոնվող արժեքը գտնվել է: Այս դեպքում **Տող 15–ում** արտածվում է գտնված տարրի դասիչը և տրամաբանական **k** փոփոխականը **Տող 16–ում** ընդունում է **true** արժեքը:

Այն պարագայում, երբ զանգվածում որոնվող արժեքը չգտնվի, տրամաբանական **k** փոփոխականի արժեքը կմնա **Տող 8–ում** վերագրված սկզբնական **false** արժեքը**:**

Տող 20. if պայմանը ճիշտ լինելը նշանակում է, որ տրված արժեքը զանգվածում չի գտնվել և այդ մասին **Տող 22–ում** արտածվում է հաղորդագրություն:

Տեսակավորման ալգորիթմներ

Բազմությունները համակարգչում կարող են ներկայացված լինել տարբեր կառուցվածքներով: Ամենահաճախ հանդիպող կառուցվածքը զանգվածն է: Դիտարկենք զանգվածների տեսակավորման մի քանի ալգորիթմներ:

Պղպջակի մեթոդը

Դիտարկենք հետևյալ խնդիրը.

Խնդիր: Դասավորել տրված ամբողջ թվերի զանգվածը աճման կարգով:

Տեսակավորման պղպջակի մեթոդը ամենահայտնի ու ամենատարածված մեթոդն է:

Այս մեթոդով յուրաքանչյուր քայլում համեմատվում են զանգվածի տարրերը զույգ առ զույգ:

Քայլ 1. Համեմատվում է զանգվածի առաջին զույգը, այսինքն՝ 1–ին և 2–րդ տարրերը:

Քայլ 2. Համեմատվում է զանգվածի 2–րդ զույգը, այսինքն՝ 2–րդ և 3–րդ տարրերը:

....

Ալգորիթմը շարունակվում է այնքան ժամանակ, քանի դեռ բոլոր զույգերը չեն դիտարկվել:

Ամեն համեմատության արդյունքում ստուգվում է հետևյալ պայմանը.

Եթե զույգի առաջին տարրը մեծ է երկրորդից, ապա տարրերը տեղերով փոխվում են:

Եթե զույգի առաջին տարրը փոքր է կամ հավասար է երկրորդին, ապա տարրերը մնում են իրենց տեղերում:

Պղպջակի մեթոդով զանգվածի տեսակավորման ալգորիթմի կոդը հետևյալն է.

```
#include <iostream>
#include <algorithm> //Կցում ենք գրադարան swap ֆունկցիան կիրառելու համար
using namespace std;
int main() {
    int array[] = {5, 2, 9, 1, 5, 6};  // Հայտարարում ենք զանգված

    int size = sizeof(array) / sizeof(array[0]); // Հաշվում ենք զանգվածի չափը

    for (int i = 1; i < size; i++)   // Ցիկլ՝ սկսած զանգվածի երկրորդ տարրից
    {
        if (array[i] < array[i - 1])  // Համեմատում ենք զանգվածի հարևան տարրերը
        {
            swap(array[i], array[i - 1]); //Փոխում ենք տեղերով, եթե ձախի արժեքը մեծ է
        }
    }
    // Տպում ենք փոփոխված զանգվածը
    for (int i = 0; i < size; i++)
    {
        cout << array[i] << " ";
    }
}
```

```
input
2 5 1 5 6 9

...Program finished with exit code 0
Press ENTER to exit console.
```

Ինչպես է աշխատում կոդը

Տող 5. int array[]: Մենք հայտարարում ենք ամբողջ թվային զանգված՝ 6 տարրերով: Այս զանգվածի արժեքներն են՝ {5, 2, 9, 1, 5, 6}:

Տող 7. sizeof(array) / sizeof(array[0]) — Կոդի հատվածը հաշվում է զանգվածի չափը:

sizeof(array) — Ցույց է տալիս int ամբողջ թվային տիպի 6–տարրանոց զանգվածի ընդհանուր չափը, որը կլինի 24 բայթ, քանի որ int տիպը զբաղեցնում է 4 բայթ հիշողության ծավալ:

sizeof(array[0]) — Վերադարձնում է զանգվածի առաջին տարրի չափը բայթերով, այսինքն՝ 4:

sizeof(array) / sizeof(array[0]) հաշվարկի արդյունքում ստանում ենք՝ 24 բայթ / 4 բայթ = 6 բայթ: 6–ը մեր զանգվածի չափն է:

Տող 9. Դիտարկում ենք զանգվածի բոլոր տարրերը՝ սկսած երկրորդ տարրից:

Տող 11. Ստուգում ենք՝ արդյոք տվյալ տարրը փոքր է նախորդից:

Տող 13. Եթե տվյալ տարրը փոքր է նախորդից, ապա տարրերը տեղերով փոխում ենք swap(array[i], array[i – 1]) ֆունկցիայի միջոցով:

Տող 19. Տեղափոխման գործընթացի ավարտից հետո արտածում ենք զանգվածը:

Բերված ալգորիթմում տեղափոխությունը կատարվում է միայն մեկ անգամ: Պետք է կրկնել ալգորիթմը այնքան անգամ, մինչև զանգվածի բոլոր տարրերը դասավորվեն աճման կարգով:

```
1  #include <iostream>
2  #include <algorithm> //Կցում ենք գրադարան swap ֆունկցիան կիրառելու համար
3  using namespace std;
4  int main()
5  {
6      int array[] = {5, 2, 9, 1, 5, 6};  // Հայտարարում ենք զանգված
7
8      bool t = true;  //Հայտարարում ենք տրամաբանական տիպի փոփոխական
9
10     int size = sizeof(array) / sizeof(array[0]); // Հաշվում ենք զանգվածի չափը
11     while (t == true)
12     {
13         t = false;
14         for (int i = 1; i < size; i++)   // Ցիկլ՝ սկսած զանգվածի երկրորդ տարրից
15         {
16             if (array[i] < array[i - 1])  // Համեմատում ենք զանգվածի հարևան տարրերը
17             {
18                 swap(array[i], array[i - 1]); //Փոխում ենք տեղերով, եթե ձախի արժեքը մեծ է
19                 t = true;
20             }
21         }
22     }
23     // Պատում ենք փոփոխված զանգվածը
24     for (int i = 0; i < size; i++)
25     {
26         cout << array[i] << " ";
27     }
28 }
```

```
input
1 2 5 5 6 9

...Program finished with exit code 0
Press ENTER to exit console.
```

Տող 11. While նախապայմանի ցիկլը կրկնվում է այնքան ժամանակ, քանի դեռ ***t = true***, այսինքն՝ շարունակվում են համեմատվել թվերը:

Տող 19. Եթե համեմատման արդյունքնում կատարվում է տեղափոխություն, ապա *t*–ն ընդունում է true արժեք, այսինքն՝ ***t = true***, և ***while*** ցիկլը շարունակվում է: Այն դեպքում, երբ այլևս համեմատման արդյունքում տեղափոխություն չի կատարվում, այսինքն՝ **Տող 16–ի** *if* պայմանը տեղի չունի, նշանակում է՝ զանգվածը դասավորված է աճման կարգով, և անհրաժեշտ է ընդհատել while ցիկլը:

Տող 13. t փոփոխականին վերագրած false արժեքը՝ ***t = false,*** ապահովում է, որ *if* պայմանը տեղի չունենալու արդյունքում **Տող 11–ի** while ցիկլը ընդհատվի:

Ամփոփում

Այս թեման ներկայացրեց տեսակավորման և փնտրման ալգորիթմների կարևորությունը տվյալների մշակման գործընթացում: Տեսակավորումը թույլ է տալիս կազմակերպել տվյալներն այնպես, որ դրանց հետ աշխատելը լինի ավելի արագ և արդյունավետ: Փնտրման ալգորիթմները օգնում են արագ գտնել անհրաժեշտ տվյալներ խնայելով ժամանակ և ռեսուրս: Սովորելով և կիրառելով այս ալգորիթմները՝ դուք կկարողանաք ստեղծել ավելի հզոր և արդյունավետ ծրագրեր՝ պատրաստ լինելով նաև աշխատել բարդ տվյալային կառուցվածքների:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ինչո՞ւ են անհրաժեշտ տեսակավորման ալգորիթմները:
2. Ձանգվածի տարրերի ինչպիսի՞ դասավորվածության դեպքում տեսակավորման ալգորիթմը կաշխատի առավել արդյունավետ:
3. Նկարագրե՛ք տեսակավորման պղպջակների մեթոդը:
4. Նկարագրե՛ք գծային որոնման ալգորիթմը:

Գործնական աշխատանք

1. *Տրված է հետևյալ զանգվածը՝ {25, 34, 36, 24, 56, 99, 23}: Տրված զանգվածում գծային որոնման ալգորիթմի օգնությամբ արտածե՛ք x = 24 արժեքն ունեցող տարրի նախորդ և հաջորդ տարրերը:*

   Ստուգում. ելքային տվյալներ՝ 36, 56

2. *Տեսակավորման պղպջակի մեթոդով դասավորե՛ք ամբողջ թվերի {0, –5, 12, 47, –6} զանգվածը նվազման կարգով:*

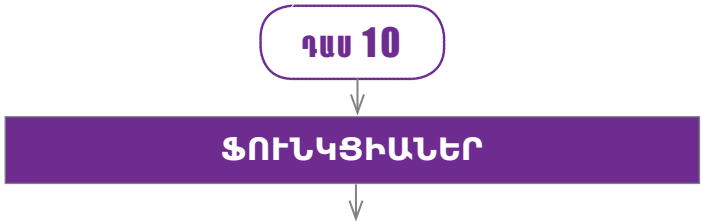
   Ստուգում. ելքային տվյալներ՝ 47, 12, 0, –5, –6

ԹԵՄԱ 10 ՖՈՒՆԿՑԻԱՆԵՐ ԵՎ ԸՆԹԱՑԱԿԱՐԳԵՐ

Ծրագրավորման մեջ ֆունկցիաները օգնում են կազմակերպել կոդը՝ բաժանելով այն տրամաբանական հատվածների: Դրանք թույլ են տալիս միևնույն գործողությունը օգտագործել բազմիցս՝ առանց կրկնօրինակելու կոդը: Ֆունկցիայի հայտարարումը, կանչը և արգումենտների միջոցով տվյալների փոխանցումը կարևոր հիմք են կառուցվածքային ծրագրավորման համար: Այս դասում կսովորենք, թե ինչպես հայտարարել և կանչել ֆունկցիա, ինչպես նաև ինչ է նշանակում արգումենտ փոխանցել արժեքով և ինչ հետևանքներ կարող են լինել դրանից:

Ուսումնասիրելով այս թեման՝

- Կսովորեք, թե ինչպես հայտարարել ֆունկցիա՝ սահմանելով անունը, պարա–մետրերը և գործողությունները:
- Կկարողանաք կազմակերպել ֆունկցիայի կանչը՝ անհրաժեշտ տվյալներ փոխանցելով:
- Կհասկանաք, թե ինչ է արժեքով փոխանցում. երբ ֆունկցիան ստանում է արգումենտի պատճենը, և փոփոխությունները չեն ազդում սկզբնական փոփոխականի վրա:



Ֆունկցիան ծրագրային կոդի հատված է, որն իրականացնում է որոշակի գործո–ղություն և որպես արդյունք վերադարձնում է մեկ արժեք:

Տարբեր խնդիրներ ծրագրավորելու ընթացքում հաճախ ստիպված ենք լինում կատարել նույն գործողությունները: Օրինակ՝ տեսակավորել զանգվածը, գտնել մե–ծագույն կամ փոքրագույն արժեքները և այլն: Եթե ամեն անգամ հրամաններ գրենք կրկնվող գործողությունների համար, ծրագիրը կստացվի ծավալուն և ոչ ընթեռնելի:

Նման իրավիճակներում օգնության են գալիս ֆունկցիաները: Դրանք թույլ են տալիս մեկ անգամ նկարագրել անհարժեշտ գործողությունները, այնուհետև կիրառել դրանք բազմաթիվ անգամ:

Ֆունկցիաների շնորհիվ կոդը դառնում է ավելի պարզ, հակիրճ և հասկանալի:

Ծրագրավորման լեզուներում գոյություն ունեն պատրաստի ֆունկցիաների հավաքածուներ, որոնք ներառված են տարբեր գրադարաններում: Ըստ անհրաժեշտության կարելի է կիրառել այդ ֆունկցիաները՝ դրանք ներառող գրադարանները կցելով ծրագրին:

Ֆունկցիաների կիրառումը ծրագրում ներառում է հետևյալ քայլերը.

- ֆունկցիայի հայտարարում,
- ֆունկցիայի սահմանում,
- ֆունկցիայի կանչ:

Ֆունկցիայի հայտարարումը կատարվում է հետևյալ եղանակով.

Տիպ <Ֆունցիայի անուն> (ֆունկցիայի պարամետրեր)

Ֆունկցիան կարող է պարունակել մեկից ավելի պարամետրեր, որոնք միմյանցից առանձնացվում են ստորակետերով: Ֆունկցիան կարող է նաև չպարունակել պարամետրեր:

Ֆունկցիայի պարամետրերը կարելի է օգտագործել միայն ֆունկցիայի մարմնում: Ֆունկցիայից դուրս այդ պարամետրերը անհասանելի են:

Ֆունկցիայի հայտարարման օրինակ է՝ int Summa(int a, int b)

Ֆունկցիայի սահմանումը ֆունկցիայի հիմնական գործողությունների նկարագրությունն է և գրվում է ֆունկցիան հայտարարելուց հետո: Այն ֆունկցիայի մարմինն է:

Տիպ <Ֆունկցիայի անուն> (պարամետրեր)

```
{
    Գործողություններ;
    return արժեք;
}
```

} Ֆունկցիայի մարմին

Օրինակ.

```
int Summa(int a, int b) {
    cout<< a + b;
}
```

Ֆունկցիայի կանչը իրականացվում է ծրագրի հիմնական կոդում: Ֆունկցիան կարելի է կանչել այնքան անգամ, որքան անհրաժեշտ է: Ֆունկցիայի կանչն իրականացվում է հետևյալ կերպ.

Ֆունկցիայի անուն (արգումենտներ);

Արգումենտները ֆունկցիայի կանչի ժամանակ պարամետրերի փոխարեն տրվող արժեքներն են, ընդ որում՝ արգումներնտների քանակը համընկնում է ֆունկցիայի հայտարարման ժամանակ տրված պարամետրերի քանակին:

Օրինակ՝ վերը հայտարարված և սահմանված ֆունկցիայի կանչը իրականացվում է այսպես.

```
int main () {
    Summa(25,12);
}
```

Ֆունկցիաների արգումենտները և վերադարձվող արժեքները

Հայտարարված ֆունկցիաների աշխատանքը երևում է միայն այդ ֆունկցիաների կանչի ժամանակ: Ընդ որում՝ ֆունկցիաները անհրաժեշտ է հայտարարել դրանց կանչից առաջ:

Ֆունկցիայի հայտարարման ժամանակ տրվող տիպը ցույց է տալիս ֆունկցիայի կատարման արդյունքում վերադարձվող արժեքի տիպը: Այն դեպքում, երբ ֆունկցիան արժեք չի վերադարձնում, այն կհայտարարենք **void** տիպի:

Օրինակ՝

```
void Hello(string name)
{
    cout<< "Բարև " << name;
}
```

Նման ֆունկցիայի կանչի իրականացվում է հետևյալ կերպ.

```
1
2  #include <iostream>
3  using namespace std;
4
5  void Hello(string name)
6  {
7      cout<< "Բարև " << name;// ֆունկցիայի հայտարարում
8  }
9
10 int main() {
11
12     Hello("Հայկ");// ֆունկցիայի կանչ
13 }
14
```

```
Բարև Հայկ

...Program finished with exit code 0
Press ENTER to exit console.
```

Բերված օօրինակում *name*–ը *Hello* ֆունկցիայի պարամետրն է, իսկ «Հայկ»–ը ֆունկցիայի արգումենտն է:

Ֆունկցիայի վերադարձվող արժեքն այն արդյունքն է, որը վերադարձնում է ֆունկցիան կանչի կատարման արդյունքում: Ֆունկցիայի արժեքը վերադարձնելու համար օգտագործվում է **return** հրամանը: Երբ ֆունկցիան հանդիպում է **return** հրամանին, ապա նրա կատարումը դադարում է:

Օրինակ՝

```
int myFunction ()
{
    return 42;  // վերադարձնում է 42 արժեքը
}
```

Բերված օրինակում ֆունկցիան վերադարձնում է ամբողջ տիպի արժեք, քանի որ myFunction() ֆունկցիան հայտարարված է int տիպի:

Ֆունկցիայի մարմնում օգտագործվող return օպերատորը կարող է վերադարձնել փոփոխական, հաշվարկի արդյունք կամ հաստատուն մեծություն:

Բերված օրինակում վերադարձվում է հաշվարկի արդյունք:

```
double sum(double a, double b) {
    return a + b;  // վերադարձնում է a+b-ի արժեքը
}
```

Եթե ֆունկցիան ոչինչ չի վերադարձնում, ապա **return** հրամանը կարող է բացակայել:

Ֆունկցիայի կանչը կարելի է կազմակերպել՝ նշելով ֆունկցիայի անունը և արգումենտները:

Օրինակ՝

Հայտարարել ֆունկցիա, որը հաշվում է երկու իրական տիպի թվերի գումարը: Արտածել ֆունկցիայի կանչի արդյունքը:

```
1  #include <iostream>
2  using namespace std;
3
4  double sum(double a, double b)
5  {
6      return a+b;
7  }
8  int main()
9  {
10     double n = 2.3, k=15.1;
11     cout << sum(n,k);// Արտածում ենք ֆունկցիայի կանչի արդյունքը
12 }
13
```

```
input
17.4

...Program finished with exit code 0
Press ENTER to exit console.
```

Տող 4. Հայտարարում ենք double տիպի sum ֆունկցիան, որն ունի երկու պարա–մետր՝ a և b:

Տող 5–7. Նկարագրում ենք ֆունկցիայի մարմինը:

Տող 11. Կազմակերպում ենք ֆունկցիայի կանչ n և k արգումենտներով՝ sum(n,k), և արտածում այդ կանչի արդյունքը: Այսինքն՝ արտածում ենք n և k փոփոխականների գումարը:

Այժմ դիտարկենք ֆունկցիաների կիրառմամբ մի քանի խնդիրներ:

Խնդիր 1. Գրել ֆունկցիա, որը ողջունում է այցելուին:

```
#include <iostream>
using namespace std;

void myFunction() // ֆունկցիայի հայտարարում և սահմանում
{
    cout <<"Ողջույն, հարգելի այցելու:";
}
int main()
{
    myFunction(); //ֆունկցիայի կանչ
}
```

```
Ողջույն, հարգելի այցելու:

...Program finished with exit code 0
Press ENTER to exit console.
```

Խնդիր 2. Գրել ֆունկցիա, որը կարտածի ընտանիքի անդամների անունը, ազգանունը, տարիքը:

```
#include <iostream>
using namespace std;

void myFunction(string name, int age)//ֆունկցիայի հայտարարում և սահմանում
{
  cout << name << " Սարգսյան " << age << " տարեկան " << endl;
}
int main()
{
  myFunction("Նարինե", 3); //ֆունկցիայի կանչ
  myFunction("Կարեն", 14); //ֆունկցիայի կանչ
  myFunction("Աննա", 30);  //ֆունկցիայի կանչ
}
```

```
input
Նարինե Սարգսյան 3 տարեկան
Կարեն Սարգսյան 14 տարեկան
Աննա Սարգսյան 30 տարեկան


...Program finished with exit code 0
Press ENTER to exit console.
```

Խնդիր 3. **Գրել ծրագիր, որը հաշվում է ներմուծված երկու թվերից փոքրագույնը:**

```
1  #include <iostream>
2  using namespace std;
3
4  int Min(int a, int b)
5  {
6      if (a < b)
7      {
8          return a;  // եթե a-ն փոքր է b-ից, վերադարձնում ենք a-ն
9      }
10     else
11     {
12         return b;  // հակառակ դեպքում վերադարձնում ենք b-ն
13     }
14 }
15 int main()
16 {
17     int num1, num2;
18     cout<<"Մուտքագրիր իրարից տարբեր երկու թիվ՝ ";
19     cin>>num1;
20     cin>>num2;
21
22     // Կանչում ենք Min ֆունկցիան
23     cout << "Երկու թվերից փոքրագույնն է՝ " << Min(num1, num2) << endl;
24 }
25
```

```
input
Մուտքագրիր իրարից տարբեր երկու թիվ՝ 5 7
Երկու թվերից փոքրագույնն է՝ 5


...Program finished with exit code 0
Press ENTER to exit console.
```

Ինչպես երևում է տող 4–ի ֆունկցիայի հայտարարումից, ֆունկցիան ունի երկու պարամետր, և տող 23–ի կանչի ժամանակ նրան տրվել է երկու արգումենտ:

Խնդիր 4. Ֆունկցիայի կիրառմամբ գրել ծրագիր, որը հաշվում է ներմուծված երկու թվերից մեծագույնի քառակուսին և կրկնապատիկը:

```cpp
#include <iostream>
using namespace std;

int Max(int x, int y)
{
    if (x >= y)
    {
        return x;
    }
    else
    {
        return y;
    }
}
int main()
{
    int a, b;
    cout << "Ներմուծեք երկու ամբողջ թիվ ";
    cin >> a;
    cin >> b;

    int m = Max(a,b); //Ֆունկցիայի կանչ
    cout << m * m << endl;
    cout << 2 * m << endl;
}
```

```
input
Ներմուծեք երկու ամբողջ թիվ 5 2
25
10


...Program finished with exit code 0
Press ENTER to exit console.
```

Ամփոփում

Այս թեման ներկայացրեց ֆունկցիաների հիմունքները՝ դրանց կառուցումը և կիրառումը ծրագրավորման մեջ: Սովորեցիք, թե ինչպես կարելի է կազմակերպել կոդը ավելի հստակ և օգտագործելի ձևով՝ ֆունկցիաների օգնությամբ: Նաև քննարկվեց արժեքով փոխանցման սկզբունքը, որը կարևոր է տվյալների պահպանման և կառավարման տեսանկյունից: Այս գիտելիքները անհրաժեշտ են ավելի բարդ կառուցվածքներ ստեղծելու համար և հիմք են հանդիսանում ծրագրավորման հաջորդ քայլերի համար:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Նկարագրե՛ք, թե ինչ է ֆունկցիան, և ի՞նչ նպատակներով են օգտագործվում ֆունկցիաները:
2. Ի՞նչ քայլերով է իրականացվում ֆունկցիան:
3. *C++*–ում ինչպե՞ս են հայտարարում և սահմանում ֆունկցիան:
4. *C++*–ում ինչպե՞ս են իրականացնում ֆունկցիայի կանչը:
5. Ո՞րն է ֆունկցիայի պարամետրի և արգումենտի տարբերությունը:
6. Ի՞նչ է իրենից ներկայացնում ֆունկցիայի վերադարձվող արժեքը:
7. Ինչպե՞ս է հայտարարվում ֆունկցիան, որը որևէ արժեք չի վերադարձնում:

Գործնական աշխատանք

1. *Գրել ծրագիր՝ կիրառելով ֆունկցիա, որը մուտքում կստանա 4 արժեք և կվերադարձնի դրանց միջին թվաբանականը:*

Ստուգում. Մուտքային տվյալներ 1՝ 2,4,5,1, ելքային տվյալ 1՝ 3:

Մուտքային տվյալ 2, 4, 6, 3, ելքանին տվյալ 2՝ 3,75:

2. *Ձևափոխել խնդիր 2–ն այնպես, որ հնարավոր լինի ստեղնաշարից ներմուծել անունները և տարիքը, այնուհետև կանչել ֆունկցիան այդ արժեքների համար:*

3. *Գրել ծրագիր, որում ֆունկցիան վերադարձնում է երկու թվերից մեծագույնը:*

Ստուգում. Մուտքային տվյալներ 7, 18, ելքային տվյալ՝ 18:

4. *Գրել ծրագիր, որում ֆունկցիան կհաշվի տրված թվի խորանարդը:*

Ստուգում. Մուտքային տվյալ՝ 5, ելքային տվյալ՝ 125:

5. *Ուսումնասիրել տրված ծրագրային կոդը: Ի՞նչ կարտածվի տրված կոդի կատարման արդյունքում:*

```
#include <iostream>
using namespace std;

double average(int arr[], int size)
{
    double  sum = 0;
    for (int i = 0; i < size; i++)
    {
        sum =sum+arr[i];
    }
    return sum / size;
}
int main()
{
    int n;
    cout << "Ներմուծի՛ր տարրերի քանակը՝ ";
    cin >> n;
    int arr[n];
    cout << "Ներմուծի՛ր զանգվածի տարրերը՝ "<<endl;
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    cout << average(arr, n) << endl;
}
```

ԹԵՄԱ 11

ՌԵԿՈՒՐՍԻԱ

Ծրագրավորման մեջ ռեկուրսիան մի միջոց է, որի օգնությամբ ֆունկցիան կարող է ինքն իրեն կանչել՝ խնդիրը լուծելով ավելի փոքր հատվածներով: Թեպետ սկզբում այդ գաղափարը կարող է բարդ թվալ, սակայն ռեկուրսիան շատ արդյունավետ է հատկապես այն խնդիրների դեպքում, որոնք ունեն կրկնվող կառուցվածք: Բնության մեջ էլ կարելի է հանդիպել ռեկուրսիվ ձևերի, օրինակ՝ ծառի ճյուղավորումները կամ ձյան փաթիլների երկրաչափական ձևերը: Այս թեմայում կուսումնասիրենք, թե ինչ է ռեկուրսիան, ինչպես է այն արտահայտվում թե՛ բնության մեջ, թե՛ ծրագրավորման մեջ, և ինչպես կարող ենք կառուցել ռեկուրսիվ ֆունկցիաներ:

Ուսումնասիրելով այս թեման՝

- Կսովորեք, թե ինչ է ռեկուրսիան և որտեղ է այն հանդիպում բնության և տեխնոլոգիայի մեջ:
- Կկարողանաք բացատրել, թե երբ է ռեկուրսիայի օգտագործումն արդյունավետ:
- Կսովորեք գրել ռեկուրսիվ ֆունկցիաներ:
- Կկարողանաք հետևել ռեկուրսիվ ալգորիթմների ընթացքին և հասկանալ դրանց տրամաբանությունը:

ԴԱՍ 11

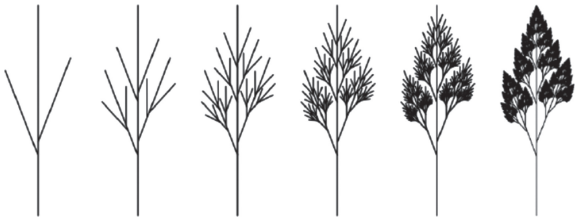
ՌԵԿՈՒՐՍԻԱՅԻ ԳԱՂԱՓԱՐԸ: ՌԵԿՈՒՐՍԻԱՆ ԲՆՈՒԹՅԱՆ ՄԵՋ

Ռեկուրսիան մի մեթոդ է, երբ գործընթացն ինքն իրեն կանչում կամ կրկնում է, մինչև որոշակի պայմանների բավարարումը:

Ծրագրավորման և մաթեմատիկայի մեջ ռեկուրսիան ֆունկցիա է, որը կանչում է ինքն իրեն: Ռեկուրսիան լայնորեն հանդիպում է նաև բնության մեջ, որտեղ ռեկուրսիվ գործընթացները արտահայտված են ինքնանման և կրկնվող ձևերով, երբ մի կառուցվածքը կրկնվում է ավելի փոքր մասշտաբներով:

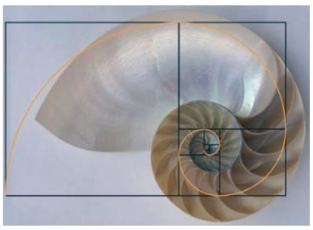
ՌԵԿՈՒՐՍԻԱՅԻ ՕՐԻՆԱԿՆԵՐ ԲՆՈՒԹՅԱՆ ՄԵՋ

Ծառերին նայելով՝ կտեսնենք, որ մեծ ճյուղերը բաժանվում են փոքրերի, իսկ փոքրերը՝ է՛լ ավելի փոքրերի: Այսպիսով՝ ճյուղավորումը կրկնվում է տարբեր մասշտաբներով՝ ցուցադրելով ռեկուրսիա:



ԴՆԹ–ի մոլեկուլը նույնպես ունի ռեկուրսիվ կառուցվածք. այն կազմված է կրկնվող նուկլեոտիդային հաջորդականությունից, որոնք կրկնվում են ամբողջ մոլեկուլում: ԴՆԹ–ի գենետիկական կոդի փոխանցումը նոր բջիջներին կարող է դիտվել որպես ռեկուրսիվ գործընթաց, որտեղ նույն կոդը կրկնվում է նոր բջիջներում:

Ֆիբոնաչիի հաջորդականությունը՝ 1, 1, 2, 3, 5, 8, 13..., որը ստացվում է ռեկուրսիայով, հաճախ հանդիպում է բնության մեջ: Օրինակ՝ ծաղիկների թերթիկների դասավորությունը կամ կակտուսների փշերի դիրքերը: Բազմաթիվ բույսեր ունեն ծաղիկներ, որոնց թերթիկների քանակը համապատասխանում է Ֆիբոնաչիի թվերին: Նույնը կարելի է ասել արևածաղկի սերմերի մասին, որոնք դասավորված են յուրահատուկ պատկերներով՝ հիմնված Ֆիբոնաչիի հաջորդականության վրա: Խխունջի պատյանը փաթաթված է Ֆիբոնաչիի պարույրի վրա, որտեղ յուրաքանչյուր նոր շրջան ավելի մեծ է նախորդից՝ համաձայն հաջորդականության կանոնների:



Բջիջների բաժանումը նույնպես կարելի է դիտարկել որպես ռեկուրսիվ գործընթաց, որտեղ ամեն նոր բջիջ կրկնօրինակում է իր գենետիկական տեղեկույթը՝ կրկին և կրկին ստեղծելով նոր բջիջներ: Այս գործընթացը շատ կարևոր է բազմացման և աճի համար:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ է ռեկուրսիան
2. Բերե՛ք բնության մեջ հանդիպող ռեկուրսիայի օրինակներ:
3. Ֆիբոնաչիի թվերը ինչպե՞ս են արտահայտվում բնության մեջ:
4. Համացանցից կամ այլ աղբյուրներից որոնե՛ք և գտե՛ք բնության մեջ ռեկուրսիայի գաղափարն արտահայտող օրինակներ:

ԴԱՍ 12

ՌԵԿՈՒՐՍԻՎ ԱԼԳՈՐԻԹՄՆԵՐԻ ԾՐԱԳՐԱՅԻՆ ԻՐԱԿԱՆԱՑՈՒՄԸ

Ռեկուրսիան լայնորեն կիրառվում է տարբեր ալգորիթմներում, հատկապես այն խնդիրներում, որտեղ խնդիրը կարելի է բաժանել ավելի փոքր ենթախնդիրների: Ծրագրավորման մեջ ռեկուրսիան ամենից հաճախ վերաբերում է ֆունկցիային, որը կանչում է ինքն իրեն: Ռեկուրսիվ ֆունկցիաների կարևոր բաղադրիչներ են բազային դեպքը և ռեկուրսիվ կանչը:

Բազային դեպքն այն դեպքն է, երբ խնդիրը կարող է լուծվել առանց ռեկուրսիայի, այսինքն՝ դա ռեկուրսիայի դադարեցման պայմանն է: Եթե ռեկուրսիան չի հասնում բազային դեպքին, ապա կստացվի անվերջ գործընթաց:

Ռեկուրսիվ կանչն այն դեպքն է, երբ ֆունկցիան կանչում է ինքն իրեն՝ խնդիրներն ավելի փոքր մասերի բաժանելու միջոցով լուծելու համար: Յուրաքանչյուր ռեկուրսիվ կանչ խնդրի լուծումը մոտեցնում է բազային դեպքին:

Ռեկուրսիվ ֆունկցիաների միջոցով հաշվարկում են թվի ֆակտորիալը, Ֆիբոնաչիի հաջորդականության n–րդ անդամը և այլն:

Օրինակ 1. Կիրառելով ռեկուրսիվ ֆունկցիա՝ գտնենք n–ից փոքր բոլոր բնական կենտ թվերի գումարը:

```cpp
#include <iostream>
using namespace std;
int sumOdd(int n)
{
    // Բազային դեպք. եթե n <= 0, վերադարձնում է 0
    if (n <= 0)
    {
    return 0;
    }
    // եթե n-ը կենտ է, այն ավելացնում ենք ռեկուրսիվ ֆունկցիայի արդյունքին
    if (n % 2 == 1)
    {
        return n + sumOdd(n - 2);
    }
    else
    {
      // եթե n-ը զույգ է, ֆունկցիան կանչում ենք n-1-ի համար
        return sumOdd(n - 1);
    }
}
int main()
{
    int n;
    cout << "Մուտքագրիր թիվը՝ ";
    cin >> n;
    cout << n << " -ից փոքր կենտ թվերի գումարն է՝ " << sumOdd(n-1) << endl;
}
```

```
input
Մուտքագրիր թիվը՝ 10
10 -ից փոքր կենտ թվ◆ի գումարն է՝ 25


...Program finished with exit code 0
Press ENTER to exit console.
```

Տող 3. Հայտարարում ենք sumOdd ֆունկցիա, որը որպես պարամետր ընդունում է մեկ ամբողջ թիվ՝ n–ը :

Ֆունկցիայում նկարագրված են բազային և ռեկուրսիվ դեպքերը:

Տող 6. Բազային դեպք: Եթե n <= 0, ապա սա նշանակում է, որ n թիվը բացասական է, կամ հավասար է 0–ի, որը չի բավարարում խնդրի պայմանին: Այս դեպքում ֆունկցիան վերադարձնում է 0 արժեք, և ռեկուրսիան դադարում է:

Տող 11. Եթե n–ը կենտ է, այսինքն՝ n % 2 == 1, ապա ունենք ռեկուրսիվ դեպք:

Տող 13. Ռեկուրսիվ դեպք: Սա նշանակում է, որ n–ին գումարվում է n–ին նախորդող կենտ թիվը, որը ստանալու համար կանչվում է sumOdd(n – 2) ֆունկցիան:

Տող 18. Հակառակ դեպքում, եթե n–ը զույգ է, ապա ռեկուրսիան կանչվում է n–ին նախորդող կենտ թվի համար, այսինքն՝ sumOdd(n – 1):

Տող 21. main() ֆունկցիան: Հիմնական ծրագրում մուտքագրվում է n թիվը, որից հետո հաշվվում է այդ թվից փոքր կենտ թվերի գումարը՝ օգտագործելով sumOdd(n–1) ֆունկցիայի կանչը: Արտածվում է արդյունքը:

Օրինակ 1. Եթե մուտքագրենք 10 թիվը, ապա ֆունկցիան կհաշվի՝ 9 + 7 + 5 + 3 + 1 = 25:

Օրինակ 2. Կիրառելով ռեկուրսիվ ֆունկցիա՝ հաշվել 1–ից N բնական թվերի գումարը:

```
1  #include <iostream>
2  using namespace std;
3  int Sum(int N)  // Հայտարարում ենք գումար հաշվելու Sum ֆունկցիա
4  {
5      if (N == 1) // ֆունկցիայի բազային դեպք, երբ N-ը ընդունում է ամենափոքր թույլատրելի արժեքը
6      {
7          return 1;
8      }
9      else
10     {
11         return N + Sum(N - 1); // Ռեկուրսիվ կանչի կազմակերպում
12     }
13 }
14 int main()
15 {
16     int N;
17     cout << "Մուտքագրեք թիվը՝ ";
18     cin >> N;
19     cout << "Գումարը 1-ից " << N << " -ը հավասար է " << Sum(N) << endl; // ֆունկցիայի կանչ
20 }
```

```
input
Մուտքագրեք թիվը՝ 100
Գումարը 1-ից 100 -ը հավասար է 5050


...Program finished with exit code 0
Press ENTER to exit console.
```

Ռեկուրսիայի առավելությունները և թերությունները

Ռեկուրսիան խնդրի լուծման կարևոր մեթոդ է, որը հնարավորություն է տալիս լուծելու բարդ խնդիրներ ավելի պարզ և համակարգված ձևով: Այնուամենայնիվ, ռեկուրսիան պետք է կիրառվի ճիշտ իրավիճակներում՝ ուշադրություն դարձնելով արդյունավետությանը և ռեսուրսների օգտագործմանը: Ռեկուրսիան շատ հարմար է պարզ խնդիրների և ռեկուրսիվ բնույթի խնդիրների համար՝ ապահովելով պարզ ու ընթեռնելի կոդ: Ռեկուրսիան հաճախ կարող է անարդյունավետ լինել, պահանջել մեծ հիշողություն և առաջացնել կրկնակի հաշվարկներ, որը կսահմանափակի դրա կիրառումը մեծաքանակ տվյալներ պարունակող խնդիրների լուծման ժամանակ:

Ամփոփում

Այս թեմայի ընթացքում ծանոթացաք ռեկուրսիայի հիմունքներին՝ թե՛ տեսական, թե՛ գործնական մակարդակով: Ռեկուրսիան օգտակար գործիք է հատկապես այն խնդիրների համար, որոնք կարելի է բաժանել նույնատիպ փոքր խնդիրների: Այն կիրառելի է ոչ միայն ծրագրավորման մեջ, այլ նաև բնության մեջ հանդիպող բազմաթիվ կառուցվածքներում: Հիմնական դեպքի ճիշտ օգտագործմամբ հնարավոր է վերահսկել ռեկուրսիայի ավարտը և ապահովել ծրագրի ճիշտ աշխատանքը: Այս թեմայի ուսումնասիրությունը կօգնի ձեզ մտածել խնդիրների լուծման կառուցվածքային և ստեղծարար եղանակներով:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Թվարկե՛ք ռեկուրսիայի առավելություններն ու թերությունները:

Գործնական աշխատանք

2. ***Կիրառելով ռեկուրսիվ ֆունկցիա՝ հաշվել n–ից փոքր բոլոր զույգ թվերի գումարը:***

Ստուգում. Մուտքային տվյալ՝ 13, ելքային տվյալ՝ 42:

3. ***Կիրառելով ռեկուրսիվ ֆունկցիա՝ հաշվել 1–ից n թվերի արտադրյալը:***

Ստուգում. Մուտքային տվյալ՝ 6, ելքային տվյալ՝ 720:

ԹԵՄԱ 12

ԴԱՍԵՐ

Ծրագրավորման մեջ արդյունավետությունը և կոդի կազմակերպվածությունը մեծ նշանակություն ունեն: Մեծածավալ խնդիրներ լուծելու համար անհրաժեշտ է ծրագրերը կառուցել պարզ և վերաօգտագործելի ձևով: Այդ նպատակով ստեղծվել է օբյեկտ կողմնորոշված ծրագրավորման (ՕԿԾ) մոտեցումը, որն առավելություններ է տալիս ինչպես կոդի կազմակերպման, այնպես էլ դրա հետագա սպասարկման ու ընդլայնման հարցում: Օբյեկտ կողմնորոշված ծրագրավորման հիմքում ընկած է «դաս» և «օբյեկտ» հասկացությունները, որոնք թույլ են տալիս իրական աշխարհում եղած երևույթները պատկերել ծրագրավորման լեզվով:

Ուսումնասիրելով այս թեման՝

- Կբացատրեք, թե ինչ է օբյեկտ կողմնորոշված ծրագրավորումը և որն է դրա հիմնական առավելությունը՝ համեմատած այլ մոտեցումների հետ:
- Կսովորեք նկարագրել դասի կառուցվածքը՝ հաշվի առնելով հատկություն–ները և մեթոդները:
- Կգրեք պարզ ծրագրեր, որոնք օգտագործում են դասեր՝ ստեղծելով օբյեկտներ և կիրառելով մեթոդներ:
- Կծանոթանաք ժառանգականության գաղափարին և կտեսնեք, թե ինչպես է այն կիրառվում տարբեր դասերի միջև:
- Կգրեք պարզ օրինակներ, որոնք ցույց են տալիս, թե ինչպես կարելի է ստեղծել ժառանգություն ունեցող դասեր՝ օգտագործելով C++ լեզուն:

ԴԱՍ 13

ՕԲՅԵԿՏ–ԿՈՂՄՆՈՐՈՇՎԱԾ ԾՐԱԳՐԱՎՈՐՈՒՄ: ԴԱՍԵՐ

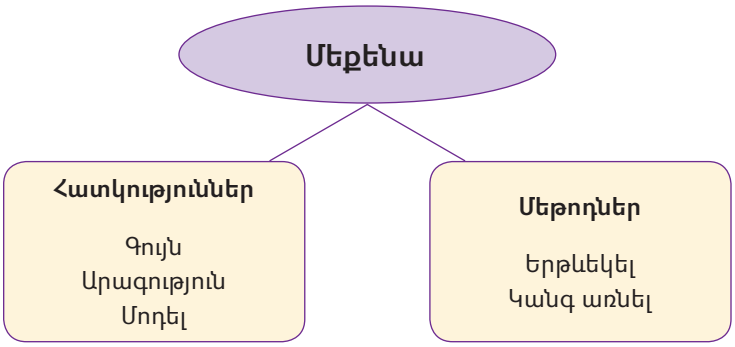
Օբյեկտ–կողմնորոշված ծրագրավորումը (ՕԿԾ) մոտեցում է, երբ ծրագիրը դի–տարկվում է որպես միմյանց հետ փոխազդող օբյեկտների խումբ: Յուրաքանչյուր օբյեկտ ունի իր սեփական հատկությունները և մեթոդները:

Հատկություններ: Օբյեկտի նկարագրություններն են, որոնք որոշում են օբյեկտի վիճակը:

Օրինակ՝ «մեքենա» օբյեկտի հատկությունները կարող են լինել *գույնը, արագությունը, մոդելը*:

Գործողություններ կամ մեթոդներ: Օբյեկտի կատարած գործողություններն են, որոնք սահմանում են նրա վարքագիծը:

Օրինակ՝ «մեքենա» օբյեկտի գործողությունները կարող են լինել *երթևեկելը, կանգ առնելը*:

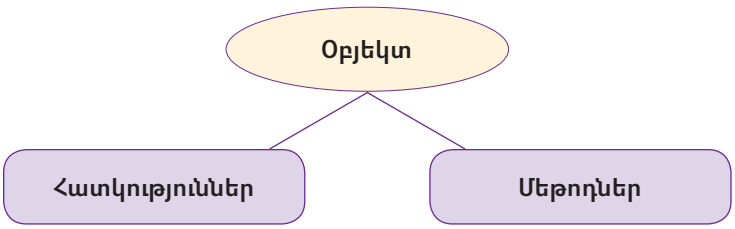


Պատկերացրե՛ք, որ ստեղծում եք համակարգչային խաղ, որտեղ կան տարբեր կենդանիներ, օրինակ՝ կատու կամ շուն: Օբյեկտ–կողմնորոշված ծրագրավորման մեջ յուրաքանչյուր կենդանի ներկայացվում է որպես «օբյեկտ»: Այդ կենդանիներն ունեն տարբեր հատկություններ, օրինակ՝ անուն, տարիք, տեսակ և այլն: Նրանք կարող են տարբեր գործողություններ կատարել, օրինակ՝ քայլել, հաչել, ցատկել, որոնք օբյեկտի մեթոդներն են:

Օբյեկտ–կողմնորոշված ծրագրավորման հիմնական գաղափարներն են դասերը, օբյեկտները, մեթոդները:

1. Դասեր (Classes)

Դասը ծրագրում ներկայացված ձևանմուշ է, որը նկարագրում է, թե ինչպես պետք է կառուցվի օբյեկտը: Դասը թույլ է տալիս սահմանել, թե ինչ հատկություններ կունենա և ինչ գործողություններ կկատարի տվյալ օբյեկտը:



2. Օբյեկտներ (Objects)

Օբյեկտը կարելի է դիտարկել որպես որոշակի կառուցվածքով փոփոխական, որը ստեղծվում է «դաս» կոչվող ձևանմուշների հիման վրա:

3. Մեթոդներ

Մեթոդները գործնականում այն գործողություններն են, որոնք օբյեկտը կարող է կատարել: Օրինակ՝ քայլելը կամ հաչելը մեթոդներ են:

Այսպիսով՝ օբյեկտ–կողմնորոշված ծրագրավորումը մոտեցում է, որը թույլ է տալիս ավելի կառավարելի և ընդլայնված ծրագրեր ստեղծել: Օբյեկտ–կողմնորոշված ծրագրավորումը ծրագրերը դարձնում է ավելի կազմակերպված և ընթեռնելի, հնարավորություն է տալիս բազմակի օգտագործելու ստեղծված դասերը: Ծրագիրը բաժանվում է փոքր, իրարից անկախ միավորների, որոնք հեշտ է կառավարել, թեստավորել և փոփոխել:

Ընդհանրացնելով կարող ենք ասել, որ օբյեկտ–կողմնորոշված ծրագրավորումը մի մոտեցում է, որի դեպքում ծրագիրը ներկայացվում է որպես օբյեկտների համախումբ, որտեղ ամեն մի օբյեկտ պատկանում է որևէ դասի:

ԴԱՍԵՐ

Դասը նախատեսված է որոշակի տեսակի օբյեկտ նկարագրելու համար: Այն օբյեկտի ստեղծման ձևանմուշ է, իսկ օբյեկտը դասն իրականացնողն է:

C++ ծրագրավորման լեզվում դաս սահմանելու համար օգտագործում են **class** առանցքային բառը, որին հաջորդում է դասի անունը: Ընդհանուր կառուցվածքն այսպիսին է.

```
class <դասի անուն>
{
    // հատկությունների ցանկ
    // մեթոդների ցանկ
};
```

Օրինակ՝ `class Person { };`

Տվյալ օրինակում սահմանված է Person անունով դասը: Որպես կանոն՝ դասերի անունները գրվում են մեծատառով: Այն դատարկ է, չի պարունակում որևէ բաղադրիչ: Դասը սահմանելուց հետո այն կարող ենք օգտագործել ծրագրում:

Հետևյալ օրինակում main() ֆունկցիայում ստեղծվում է Person դասի մեկ օբյեկտ՝ person:

```
#include <iostream>
class Person
{

};
int main()
{
    Person person;
}
```

Դասի հատկությունները փոփոխականներ են, որոնք ընդունում են որոշակի արժեքներ: Մեթոդները և հատկությունները դասում նկարագրվում են հետևյալ կերպ.

```
class Person
{
    int age; // age հատկություն, որը նկարագրում է տարիքը
    string name; // name հատկություն, որը նկարագրում է անունը

    void print() { // հայտարարում ենք մեթոդ, որը տպում է name հատկությունը
        cout<<name;
    }
};
```

Ծրագրում նկարագրված օբյեկտի համար դասի հատկություններին և մեթոդներին դիմում են կետի միջոցով:

Օրինակ՝

```
int main()
{
    Person person;

    person.age = 20; // Դիմում ենք դասի age հատկությանը
    person.print();  // Դիմում ենք դասի print մեթոդին
}
```

Դաս սահմանելիս սովորաբար դասի անդամները հայտարարվում են private կամ public բաժնում: C++–ում **private** և **public** հասկացությունները սահմանում են դասի անդամների հասանելիությունը:

Դասի public անդամները հասանելի են դասից դուրս ցանկացած տեղից:

Դաս private անդամները հասանելի չեն դասից դուրս: Դրանք կարող են օգտագործվել միայն դասի ներսում, այսինքն՝ միայն դասի մեթոդներն ունեն թույլտվություն փոխազդելու կամ փոփոխելու այդ տվյալները:

Դասի private բաժնում հայտարարված մեծություններն անվանում են փակ, իսկ public բաժնում հայտարարվածները՝ բաց:

Դասի սահմանման մեջ կարող են իրար հաջորդող բազմաթիվ public և private բաժիններ ներառվել:

Դիտարկենք հետևյալ խնդիրը, որոնցում նկարագրված է **Person** անունով դասը: Այն պարունակում է մարդու *անուն* և *տարիք* հատկությունները, ինչպես նաև այդ տվյալները արտածելու համար նախատեսված մեթոդ:

```cpp
#include <iostream>
using namespace std;

// Person դասի սահմանում
class Person
{
    public:
        string name; // name հատկության հայտարարում
        int age; // age հատկության հայտարարում

        // Անձի տվյալները արտածող մեթոդի նկարագրում
        void Print()
        {
            cout << "Անունը՝ " << name << " , Տարիքը՝ " << age << endl;
        }
};
int main()
{
    //Հայտարարում ենք Person դասի օբյեկտ
    Person person1;
    // Տալիս ենք արժեքներ անվանը և տարիքին
    person1.name = "Արարատ";
    person1.age = 28;
    // Արտածում ենք տվյալները մեթոդի օգնությամբ:
    person1.Print();
}
```

```
input
Անունը՝ Արարատ , Տարիքը՝ 28


...Program finished with exit code 0
Press ENTER to exit console.
```

Ինչպես է աշխատում կոդը

Տող 5. Սահմանել ենք **Person** դասը, որը պարունակում է երկու հատկություն՝ **name**, որը նկարագրում է մարդու անունը, և **age**, որը նկարագրում է մարդու տարիքը:

Տող 12. Նկարագրում ենք **Person** դասի **Print()** մեթոդը, որը արտածում է անձի անունն ու տարիքը:

Տող 20. Հայտարարում ենք **Person** դասին պատկանող **person1** օբյեկտը:

Տող 22–23. Սահմանում ենք **person1**–ի անունը որպես **«Արարատ»**, իսկ տարիքը՝ **30**:

Տող 25. person1.Print() մեթոդի կիրառմամբ տպում ենք անձի անունը և տարիքը:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ է օբյեկտ–կողմնորոշված ծրագրավորումը:
2. Ի՞նչ է իրենից ներկայացնում դասը: Բերե՛ք օրինակներ:
3. Ի՞նչ է իրենից ներկայացնում օբյեկտը:
4. Ինչպե՞ս է հայտարարվում դասը *C++* ծրագրավորման լեզվում:
5. Ինչպե՞ս դիմել դասի բաղադրիչներին հիմնական ծրագրում:
6. Գրե՛ք կոդ, որը կստեղծի Car անունով դաս՝ մեքենայի մակնիշ, մոդել, արտադրության տարի և այլ հատկություններով: Ստեղծե՛ք մեթոդ, որը կարտածի այդ տվյալները:
7. Ստեղծե՛ք Student անունով դաս, որը կունենա սովորողի անուն, դասարան և գնահատական հատկությունները: Ստեղծե՛ք մեթոդ, որը կարտածի սովորողի գնահատականը:
8. Ստեղծե՛ք Book անունով դաս, որը կունենա գրքի անուն, հեղինակ և գին հատկությունները: Ստեղծե՛ք մեթոդ, որը կարտածի գրքի և հեղինակի անունները:

ԴԱՍ 14

ԺԱՌԱՆԳՄԱՆ ԳԱՂԱՓԱՐԸ

Ժառանգումը օբյեկտ–կողմնորոշված ծրագրավորման հիմնական հասկացություններից է: Այն թույլ է տալիս մի դասից ստանալ նոր դաս՝ օգտագործելով արդեն գոյություն ունեցող դասի հատկություններն ու ֆունկցիաները: Ժառանգումը հնարավորություն է տալիս օգտագործելու արդեն գրված կոդը՝ չկրկնելով այն:

Այս գաղափարը կարելի է համեմատել իրական աշխարհի ժառանգման հետ, որտեղ երեխաները ժառանգում են որոշ հատկություններ իրենց ծնողներից, օրինակ՝ աչքերի կամ մազերի գույնը:

Ժառանգման հիմնական գաղափարներն են.

- Հիմնական դաս, որը փոխանցում է իր հատկությունները:
- Ժառանգված դաս, որը ժառանգում է հատկությունը հիմնական դասից:

C++–ում ժառանգումը հայտարարվում է հետևյալ ձևով.

```
class <ժառանգված դաս> : <ժառանգման տիպ> <հիմնական դաս>
{
// Տվյալներ և մեթոդներ ժառանգված դասում
};
```

Օրինակ՝

```
1  #include <iostream>
2  class Person // Հիմնական դաս
3  {
4      public:
5      string name;
6      int age;
7  }
8
9  class Employee : public Person // ժառանգված դաս
10 {
11 public:
12     string company; // ժառանգված դասի հատկություն
13 };
```

Տող 9. Հայտարարված է **Employee** ժառանգված դասը, որը Տող 2–ի **Person** դասից ժառանգում է մարդու տարիքը և անունը:

Տող 12. Նկարագրված է ժառանգված դասի company հատկությունը, որը ցույց է տալիս մարդու աշխատանքի վայրը:

Տվյալ օրինակում **Employee** դասը կունենա **name**, **age**, **company** հատկությունները:

Դիտարկենք հետևյալ օրինակը.

Ենթադրենք՝ ունենք Animal դասը, որը նկարագրում է բոլոր կենդանիներին՝ անուն և տարիք հատկություններով: Բայց կան որոշ կենդանիներ, որոնք ունեն իրենց յուրահատուկ հատկությունները, օրինակ՝ շները, որոնք ունեն ցեղատեսակ: Այդ դեպքում կարող ենք ստեղծել Dog դաս, որը ժառանգում է Animal դասի ընդհանուր հատկությունները՝ անունը և տարիքը, և ավելացնել նոր հատկություն՝ ցեղատեսակ:

```cpp
#include <iostream>
using namespace std;
//Animal հիմնական դասի նկարագրություն
class Animal
{
    public:
        string name;  // Կենդանու անուն հատկություն
        int age;      // Կենդանու տարիք հատկություն
};
// Dog դաս, որը ժառանգված է Animal դասից
class Dog : public Animal
{
    public:
       string breed;  // Շան ցեղատեսակ հատկություն
};
int main() {
    Dog myDog;              // Հայտարարում ենք Dog դասի myDog օբյեկտ
    myDog.name = "Դիկ";    //Տալիս ենք արժեք myDog օբյեկտի անվանը
    myDog.age = 2;          //Տալիս ենք արժեք myDog օբյեկտի տարիքին
    myDog.breed = "Հասկի"; //Տալիս ենք արժեք myDog օբյեկտի ցեղատեսակին

    // Տպում ենք շան բոլոր տվյալները
    cout<< myDog.name << endl << myDog.age << endl << myDog.breed;
}
```

```
input
Դիկ
2
Հասկի

...Program finished with exit code 0
Press ENTER to exit console.
```

Տող 4. **Animal** դասը հիմնական դասն է, որն ունի *կենդանու անուն* և *տարիք* հատկությունները:

Տող 11. **Dog** դասը նոր դաս է, որը ժառանգում է **Animal** դասի բոլոր հատկությունները՝ անունը և տարիքը: Բացի դրանցից՝ այն ունի սեփական հատկությունը՝ շան ցեղատեսակը:

Տող 18–19. Այս տողերը ցույց են տալիս ժառանգականության կիրառումը: **Dog** դասը ստանում է **Animal** դասի **name** և **age** հատկությունները, չնայած իր մեջ դրանք նկարագրված չեն:

Այս օրինակով ցույց է տրվում, որ ժառանգված **Dog** դասը կարող է օգտագործել հիմնական **Animal** դասի հատկությունները՝ առանց դրանք նորից սահմանելու:

Ժառանգման առավելությունները

1. *Վերաօգտագործում*: Կոդը կարելի է վերաօգտագործել՝ չկրկնելով այն:
2. *Կազմակերպվածություն*: ժառանգման միջոցով կարող ենք տարանջատել օբյեկտի ընդհանուր և հատուկ հատկությունները:
3. *Կոդի պարզեցում*: ժառանգումը թույլ է տալիս ավելի կոկիկ և տրամաբանական կառուցվածք ստանալ:

ժառանգումն օգնում է դասակարգելու կոդը՝ կոդի տարբեր մասերը միմյանցից անկախ պահելու և ավելի արդյունավետ լուծումներ ստեղծելու համար:

Ամփոփում

Այս թեմայի ուսումնասիրության արդյունքում պարզ դարձավ, որ օբյեկտ կողմնորոշված ծրագրավորումը (ՕԿԾ) թույլ է տալիս կառուցել ավելի ճկուն, կարդացվող և վերահսկելի ծրագրեր: Դասերի և օբյեկտների կիրառումը հնարավորություն է տալիս ծրագրերը կառուցել իրական աշխարհի օրինակով, իսկ ժառանգականության շնորհիվ հնարավոր է վերաօգտագործել և ընդլայնել արդեն ստեղծված կոդը: Դասերի ճիշտ կառուցումը և ՕԿԾ-ի սկզբունքների կիրառումը ապահովում են ծրագրերի կազմակերպվածություն և ավելի հեշտ սպասարկում ապագայում:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ է ժառանգումը:
2. Ժառանգումը ներկայացրե՛ք իրական կյանքի օրինակով:
3. Ինչպե՞ս է հայտարարվում ժառանգումը *C++*–ում:

Գործնական աշխատանք

Ներմուծե՛ք հետևյալ ծրագիրը և գործարկե՛ք այն:

Ներկայացրե՛ք, թե՝

- *որն է հիմնական դասը,*
- *որոնք են ժառանգված դասերը,*
- *որ հատկություններն են ժառանգվել,*
- *որ հատկությունն են ավելացվել ժառանգվող դասերում:*

```
#include <iostream>
using namespace std;
class Person
{
    public:
        string name;
        int age;
        void displayInfo()
        {
            cout << "Անունը՝ " << name << ", Տարիքը՝ " << age << endl;
        }
};
class Teacher : public Person
{
    public:
        string subject;
        void printTeacher()
       {
            cout <<"Անունը՝ "<<name<<", Տարիքը՝ " <<age<<", Առարկան՝ "<<subject<<endl;
        }
};
class Student : public Person
{
    public:
        int grade;
        void printStudent()
        {
            cout <<"Անունը՝ "<<name<<", Տարիքը՝ "<<age<<", Դասարանը՝ "<<grade<<endl;
        }
};
int main()
{
    Teacher teacher1;
    teacher1.name = "Կարեն Սահակյան";
    teacher1.age = 32;
    teacher1.subject = "ԹԳՀԳ";
    Student student1;
    student1.name = "Արամ Աբգարյան";
    student1.age = 16;
    student1.grade = 11;
    teacher1.printTeacher();
    student1.printStudent();
}
```

ԹԵՄԱ 13

ՍԽԱԼՆԵՐ ԵՎ ԴՐԱՆՑ ՏԵՍԱԿՆԵՐԸ

Ծրագրավորումը նման է հանելուկ լուծելուն. նույնիսկ փոքր վրիպումները կարող են խաթարել ամբողջ համակարգի աշխատանքը: Սխալները ծրագրավորման գործընթացի անբաժան մասն են, և սկսնակից մինչև փորձառու ծրագրավորող թույլ են տալիս սխալներ: Սակայն կարևոր չէ միայն սխալը գտնելը, այլ նաև՝ հասկանալ դրա բնույթը և արդյունավետորեն ուղղել այն: Սխալների վերլուծությունը զարգացնում է տրամաբանական մտածողությունը, ուշադրությունը և համակարգային մոտեցումը:

Ուսումնասիրելով այս թեման՝

- Կսովորեք տարբերակել սխալների հիմնական տեսակները՝ շարահյուսական, տրամաբանական և աշխատանքի ընթացքի սխալներ:
- Կկարողանաք վերլուծել սխալի առաջացման պատճառները:
- Կկիրառեք ձեր գիտելիքները՝ սխալը ուղղելու և ծրագրի արդյունավետությունն ապահովելու համար:
- Կզարգացնեք քննադատական մտածողություն և համառություն՝ ծրագրում առաջացած խնդիրները լուծելիս:

Ծրագրային սխալը ծրագրում կամ ծրագրային կոդում առկա թերություն է, որը կարող է հանգեցնել սխալ արդյունքի կամ ծրագրի անսպասելի վարքի: Սխալների պատճառով ծրագիրը չի տալիս այն արդյունքը, ինչ որ նախատեսված է:

Ծրագրային սխալների ուղղումը ծրագրավորման գործընթացի կարևոր մասն է: Այդ ուղղման գործընթացը հայտնի է որպես վրիպազերծում կամ **debugging**, որը ներառում է սխալների հայտնաբերում և շտկում՝ ապահովելով ծրագրի ճշգրիտ և կայուն աշխատանքը:

C++ ծրագրավորման լեզվում սխալները հայտնաբերվում են՝

- *թարգմանելու ընթացքում,*
- *կատարելու ընթացքում:*

Թարգմանական սխալներ

Թարգմանական սխալները հայտնաբերվում են ծրագրի կոմպիլյացիայի ժամանակ և հիմնականում կապված են շարահյուսության և տվյալների տիպերի հետ: Եթե կոմպիլյատորը հայտնաբերում է որևէ թարգմանական սխալ, ապա այն չի կարող ստեղծել ծրագրի կատարման ֆայլը:

Թարգմանական սխալները լինում են՝

- շարահյուսական,
- տիպային,
- հայտարարման և այլն:

Շարահյուսական սխալներ

C++–ում շարահյուսական սխալները կապված են ծրագրավորման լեզվի կանոնների խախտման հետ, որի արդյունքում կոմպիլյատորը չի կարողանում կարդալ և հասկանալ կոդը: Դրանք հիմնականում առաջանում են այն դեպքում, երբ կոդում չի պահպանվում ծրագրավորման լեզվի ճիշտ շարահյուսությունը, ինչպիսիք են կետադրական նշանների բացակայությունը, սխալ օպերատորների օգտագործումը, փոփոխականների կամ ֆունկցիաների սխալ անունների կիրառումը և այլն: Այսպիսի սխալները հայտնաբերվում են թարգմանչի կողմից, և ծրագրին թույլ չեն տալիս թարգմանել մինչև սխալի ուղղումը:

Հաճախ հանդիպող շարահյուսական սխալներն են՝

- կետ–ստորակետերի (;) բացակայությունը,
- չփակված փակագծերը՝ {}, (), [], կամ փակագծերի անհամապատասխանությունը,
- կրկնակի կամ սխալ հայտարարված անունները:

C++–ում նույն անունով փոփոխականների կրկնակի հայտարարումը հանգեցնում է սխալի:

Օրինակ 1.

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int x = 10;
7      int x = 20;
8  }
```

input

Compilation failed due to following error(s).

```
main.cpp: In function 'int main()':
main.cpp:15:9: error: redeclaration of 'int x'
   15 |     int x = 20;
      |         ^
main.cpp:14:9: note: 'int x' previously declared here
   14 |     int x = 10;
      |         ^
```

Տիպային սխալներ

Տիպային սխալներն առաջանում են, երբ մեկ տիպի փոփոխականին վերագրվում է մեկ այլ տիպի արժեք կամ փոփոխական:

Օրինակ 2.

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      // տողային արժեքի վերագրում թվային ամբողջ տիպի x փոփոխականին
7      int x = "Թվային գրագիտություն";
8  }
```

input

Compilation failed due to following error(s).

```
main.cpp: In function ‘int main()’:
main.cpp:7:13: error: invalid conversion from ‘const char*’ to ‘int’ [-fpermissive]
    7 |     int x = "Թվային գրագիտություն";
      |             ^~~~~~~~~~~~~~~~~~~~~~
      |             |
      |             const char*
```

Հայտարարման սխալներ

Հայտարարման սխալները C++–ում առաջանում են, երբ փոփոխականը, ֆունկցիան կամ որևէ այլ տարր օգտագործվում է նախքան դրա հայտարարումը: Այսպիսի սխալները կանխելու համար անհրաժեշտ է ճիշտ հերթականությամբ հայտարարել կոդի տարրերը:

Օրինակ 3. Փոփոխականի հայտարարման սխալ՝ երբ փոփոխականը օգտագործվում է նախքան հայտարարումը.

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << x; // x փոփոխականի կիրառում նախքան հայտարարումը:
7      int x = 10;
8  }
```

input

Compilation failed due to following error(s).

```
main.cpp: In function ‘int main()’:
main.cpp:6:13: error: ‘x’ was not declared in this scope
    6 |     cout << x;
      |             ^
```

Օրինակ 4. Ֆունկցիայի հայտարարման սխալ՝ երբ կանչում ենք ֆունկցիան մինչև այն հայտարարելը կամ սահմանելը.

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      printMessage(); //printMessage ֆունկցիան կիրառվել է մինչև հայտարարվելը:
7      return 0;
8  }
9
10 void printMessage()
11 {
12     cout << "ես սիրում եմ կոդ գրել" << endl;
13 }
```

input

Compilation failed due to following error(s).

```
main.cpp: In function ‘int main()’:
main.cpp:6:5: error: ‘printMessage’ was not declared in this scope
    6 |     printMessage(); //printMessage ֆունկցիան կիրառվել է մինչև հայտարարվելը:
      |     ^~~~~~~~~~~~
```

Կատարման ընթացքի սխալներ

Կատարման ընթացքի սխալներն ի հայտ են գալիս այն ժամանակ, երբ ծրագիրն արդեն թարգմանվել է և սկսել նրա կատարումը: Սրանք չեն հայտնաբերվում կոմպիլյացիայի ընթացքում և հաճախ առաջացնում են ծրագրի խափանում կամ անկանխատեսելի ելք:

Կատարման ընթացքի սխալների տեսակներ են՝

- բաժանում զրոյի,
- հասանելիություն սահմաններից դուրս,
- տրամաբանական սխալներ և այլն:

Բաժանում զրոյի

Օրինակ 5

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int a = 5, b = 0;
7      int c = a / b;
8      cout << c;
9  }
```

```
...Program finished with exit code 136
Press ENTER to exit console.
```

Հասանելիություն սահմաններից դուրս

Այս սխալն առաջանում է, երբ ծրագիրը, օրինակ, փորձում է դիմել զանգվածի այն տարրին, որը դուրս է նախապես հայտարարված զանգվածի սահմաններից:

Օրինակ 6

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int arr[3] = {3,7,45};
7      cout << arr[5];
8  }
```

```
input
Compilation failed due to following error(s).
cc1plus: fatal error: main.cpp: Permission denied
compilation terminated.
```

Տրամաբանական սխալներ

Այսպիսի սխալները դժվար են հայտնաբերվում, քանի որ կոմպիլյատորը չի տրամադրում հուշումներ: Դրանց հետևանքով ծրագրի կատարման ընթացքում սխալ արդյունքներ են արտածվում՝ չնայած կոդը ճիշտ թարգմանվում է և աշխատում է առանց խափանումների: Դրանք հիմնականում դրսևորվում են այն դեպքում, երբ

ծրագիրը չի կատարում ծրագրավորողի ակնկալած գործառույթը կամ տալիս է սխալ արդյունքներ: Տրամաբանական սխալները սովորաբար հայտնաբերվում են թեստավորման ընթացքում, և դրանց շտկումը պահանջում է կոդի տրամաբանության մանրամասն ուսումնասիրություն և ստուգում:

Օրինակ 7. Ենթանդրենք՝ անհրաժեշտ է հաշվել երկու թվերի միջին թվաբանականը:

```
#include <iostream>
using namespace std;

int main()
{
    int a = 12, b = 23;
    double average = (a + b) / 2; // հաշվում է միջին թվաբանականը
    cout << "The average is: " << average << endl;
}
```

```
input
The average is: 17

...Program finished with exit code 0
Press ENTER to exit console.
```

Առաջին հայացքից խնդիրը լուծված է ճիշտ, սակայն արտածվել է սխալ արդյունք:

Պատճառն այն է, որ երկու ամբողջ տիպի թվերի բաժանման արդյունքը չի կարող լինել իրական թիվ: Նման դեպքերում կարելի է փոփոխականը անմիջապես հայտարարել իրական տիպի:

Այլ տարբերակով հնարավոր է ծրագրի կատարման ընթացքում փոխել փոփոխականի տիպը: Դա կարելի է անել հետևյալ կերպ.

```
#include <iostream>
using namespace std;

int main()
{
    int a = 12, b = 23;
    double average = (double (a) + b) / 2; // հաշվում է միջին թվաբանականը
    cout << "The average is: " << average << endl;
}
```

```
input
The average is: 17.5

...Program finished with exit code 0
Press ENTER to exit console.
```

Ամփոփում

Սխալները ծրագրավորողի ամենօրյա ուղեկիցն են, սակայն դրանց հաղթահարումը դառնում է սովորողի աճի կարևոր պայմանը: Ուսումնասիրելով սխալների տեսակները՝ սովորողը ձեռք է բերում անհրաժեշտ հմտություններ՝ կոդը ավելի վստահ ու արդյունավետ մշակելու համար: Շարահյուսական սխալները սովորաբար հեշտ են հայտնաբերվում՝ շնորհիվ գործիքների, սակայն տրամաբանական սխալներն ու աշխատանքի ընթացքի սխալները պահանջում են խորքային մտածողություն: Այս թեման նպաստում է սովորողի վերլուծական ունակությունների զարգացմանը, կոդում սխալները տեսնելու, հասկանալու և ուղղելու գործընթացը քայլ առ քայլ մոտեցնում է նրան վստահ ծրագրավորողի կերպարին:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Որո՞նք են թարգմանական սխալները: Ե՞րբ են դրանք հայտնաբերվում:
2. Նկարագրե՛ք հայտարարման սխալները:
3. Որո՞նք են շարահյուսական սխալները: Բերե՛ք օրինակներ:
4. Որո՞նք են կատարման ընթացքի սխալները:
5. Բերե՛ք տրամաբանական սխալների օրինակներ:

Գործնական աշխատանք

Տրված առաջադրանքներում գտե՛ք առկա սխալները, շտկե՛ք և նկարագրե՛ք, թե ինչ տիպի սխալներ են դրանք:

1. *Ծրագիրը պետք է ստուգի՝ արդյոք թիվը զույգ է, թե՝ ոչ:*

```
#include <iostream>
using namespace std;

int main()
{
    int number
    cout << "Մուտքագրել թիվ: ";
    cin >> number;

    if (number % 2 = 0)
    {
        cout << "Թիվը զույգ է" << endl;
    }
    else
    {
        cout << "Թիվը կենա է" << endl;
    }
}
```

2. *Ծրագիրը պետք է գտնի զանգվածի ամենամեծ տարրը:*

```
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {4, 2, 7, 1, 9};
    max = arr[0];
    for (int i = 1; i < 5; i++)
    {
        if (arr[i] > max)
        {
            max = arr[i];
        }
    }
    cout << "Զանգվածի ամենամեծ տարրն է՝ " << max << endl;
}
```

3. *Ծրագիրը պետք է արտածի [1; 20] միջակայքի բոլոր զույգ թվերը:*

```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 1; i <= 20, i++)
    {
        if (i / 2 == 1)
        {
            cout << i << " ";
        }
    }
}
```

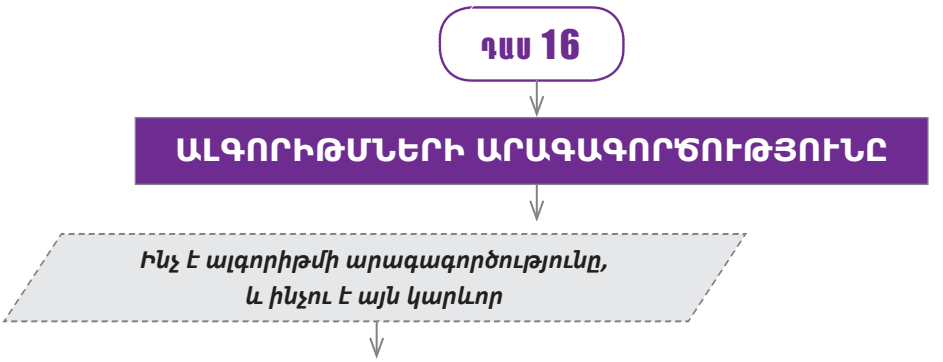
ԹԵՄԱ 14

ԱԼԳՈՐԻԹՄՆԵՐԻ ԱՐԱԳԱԳՈՐԾՈՒԹՅՈՒՆ

Ծրագրավորման մեջ միշտ չէ, որ բավական է միայն աշխատող կոդ գրել: Կարևոր է նաև հասկանալ, թե որքան արդյունավետ է ծրագիրը՝ օգտագործված ռեսուրսների, կատարման արագության և հիշողության օգտագործման տեսանկյունից: Նույն խնդիրը կարելի է լուծել տարբեր եղանակներով, սակայն դրանցից որոշները կարող են անհամեմատ ավելի արագ կամ ավելի քիչ ռեսուրսներ օգտագործել: Այս թեմայի շրջանակներում կուսումնասիրենք, թե ինչպես վերլուծել և բարելավել գոյություն ունեցող ծրագրեր՝ հաշվի առնելով դրանց կառուցվածքը, ալգորիթմները և օգտագործվող տվյալների տեսակները:

Ուսումնասիրելով այս թեման՝

- Կսովորեք, թե ինչպես գտնել ծրագրում առկա թերությունները և ոչ արդյունավետ լուծումները:
- Կկարողանաք առաջարկել և կիրառել բարելավումներ՝ փոփոխելով տվյալների կառուցվածքները կամ ալգորիթմները:
- Կգիտակցեք, թե ինչպես են ձեր կատարած փոփոխությունները ազդում ծրագրի աշխատանքի արդյունավետության վրա:



Ալգորիթմի արագագործությունը ցույց է տալիս, թե որքան արագ և արդյունավետ է ալգորիթմը լուծում իր առջև դրված խնդիրը: Արագագործությունը կարևոր է, քանի որ ալգորիթմների որոշակի կառուցվածքներ կարող են շատ դանդաղ աշխատել մեծ քանակի տվյալների դեպքում:

Ալգորիթմի կատարման ընթացքի վերլուծություն

Որպեսզի հասկանանք, թե ինչ է ալգորիթմի արագագործությունը, նախ պետք է ուսումնասիրել նրա կատարած գործողությունների քանակը, որոնք կախված են ալ–գորիթմում կիրառվող տվյալների քանակից:

Ալգորիթմի կատարման արագագործությունը հաճախ չափվում է գործողությունների քանակով: Դա ալգորիթմի բարդությունն է, որը նշանակում ենք O() տառով:

Այն ցույց է տալիս, թե տվյալների քանակի աճի դեպքում ինչքան է աճում ալգո–րիթմի կատարման ժամանակը:

Օրինակ՝ երբ ալգորիթմն ունի O(n) բարդություն, ապա գործողությունների քա–նակը ուղիղ համեմատական է տվյալների քանակին՝ n–ին: Եթե տվյալների քանակը կրկնապատկվում է, ալգորիթմի կատարման ժամանակը նույնպես կրկնապատկ–վում է:

Բերենք օրինակներ.

- O(1) — ալգորիթմի կատարման ժամանակը նույնն է՝ անկախ տվյալների քա–նակից:
- O(n) — ալգորիթմի կատարման ժամանակը կախված է տվյալների քանակից:
- $O(n^2)$ — ալգորիթմի կատարման ժամանակը կախված է տվյալների քանակի քառակուսուց: Այն հաճախ հանդիպում է ներդրված ցիկլերում:

Ինչպես բարելավել ալգորիթմների արագագործությունը

Ալգորիթմի արագագործությունը բարելավելու համար անհրաժեշտ է նվազեցնել կատարվող գործողությունների քանակը՝ հիմնականում օգտագործելով ավելի ար–դյունավետ կառուցվածքներ և մաթեմատիկական գործողություններ:

Օրինակ՝ **Տվյալների որոնում** զանգվածում:

Դիցուք ունենք տասնյակ հազարավոր տարրեր պարունակող զանգված, և պետք է փնտրենք որևէ թիվ այդ զանգվածում: Այս դեպքում կարող ենք օգտագործել մեզ արդեն ծանոթ գծային որոնման ալգորիթմը:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[] = {12, 36, 124, -564, 0, 65, 2, 1,-5};
    int x = 2;
    bool k = false;
    int n = sizeof(arr) / sizeof(arr[0]);
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == x)
        {
            cout<<" Տարրի ինդեքսը զանգվածում = "<< i << endl;
            k = true;
        }
    }
    if (k == false)
    {
      cout<<" Տրված արժեքը զանգվածում չի գտնվել";
    }
}
```

Այս ալգորիթմը հերթով համեմատում է զանգվածի բոլոր տարրերը որոնվող արժեքի հետ: Դրա բարդության աստիճանը ուղիղ համեմատական է զանգվածի տարրերի քանակին՝ O(n): Եթե պետք է անհրաժեշտ արժեքը որոնել մի քանի միլիոն տարրերի մեջ, ապա ալգորիթմը կարող է բավականին դանդաղ աշխատել:

Սակայն մենք գիտենք տարբեր տեսակավորման և փնտրման ալգորիթմներ, որոնք առավել արդյունավետ են զանգվածների հետ աշխատելու ժամանակ, և դրանց կիրառումը կմեծացնի ալգորիթմի արդյունավետությունը:

ԱԼԳՈՐԻԹՄՆԵՐԻ ԲԱՐԴՈՒԹՅԱՆ ԳՆԱՀԱՏՈՒՄԸ

Ալգորիթմի բարդությունը չափելու ժամանակ ծրագրավորողները սովորաբար կենտրոնանում են ալգորիթմի արագության վրա: Սակայն պակաս կարևոր չէ նաև հիշողությունում առկա ազատ տարածության ցուցանիշը: Արագ աշխատող ալգորիթմը չի հանգեցնի ակնկալվող արդյունքներին, եթե այն պահանջում է ավելի շատ հիշողություն, քան ունի համակարգիչը:

Շատ ալգորիթմներ առաջարկում են ընտրություն հիշողության ծավալի և արագության միջև: Խնդիրը կարելի է լուծել արագ՝ օգտագործելով մեծ քանակությամբ հիշողություն, կամ ավելի դանդաղ՝ օգտագործելով ավելի քիչ հիշողություն:

Ծրագրի ամենաբարդ մասերը սովորաբար ցիկլերն ու ֆունկցիաների կանչերն են: Ալգորիթմի բարդությունը հատկապես մեծացնում են ներդրված ցիկլերը և այն ֆունկցիաները, որոնք կանչում են այլ ֆունկցիաներ:

Բավականին դժվար է նաև հաշվել ռեկուրսիվ ֆունկցիաների բարդությունը, քանի որ ռեկուրսիվ ֆունկցիան կանչում է ինքն իրեն բազմակի անգամ:

Ամփոփում

Այս թեման ընդգծեց ծրագրերի վերլուծության և օպտիմալացման կարևորությունը: Գոյություն ունեցող ծրագրերի ուսումնասիրությունը և դրանցում բարելավումների կատարումը նպաստում են ավելի արագ, արդյունավետ և հուսալի լուծումների ստեղծմանը: Իրական ծրագրավորման գործընթացում անընդհատ անհրաժեշտ է վերանայել կոդը, կատարել կառուցվածքային կամ ալգորիթմական փոփոխություններ՝ պահպանելով ծրագրի գործառույթը, բայց դարձնելով այն ավելի արդյունավետ: Այս հմտությունները շատ կարևոր են թե՛ սկսնակ, թե՛ փորձառու ծրագրավորողների համար:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ է ալգորիթմի արագագործությունը:
2. Ի՞նչ է ալգորիթմի բարդությունը:
3. Ի՞նչ է նշանակում՝ ալգորիթմն ունի O(5), O(n), O(2n) բարդություն:
4. Ինչպե՞ս է հնարավոր բարելավել ալգորիթմների արագագործությունը:
5. Ո՞ր ալգորիթմների բարդությունն է սովորաբար լինում ավելի մեծ:

ԹԵՄԱ 15

ԾՐԱԳՐԵՐԻ ՄՇԱԿՄԱՆ ԳՈՐԾԸՆԹԱՑԸ

Ծրագրավորման գործընթացը շատ ավելին է, քան պարզապես կոդ գրելը: Իրականում ցանկացած ծրագիր անցնում է մի շարք փուլեր՝ գաղափարից մինչև օգտագործվող արտադրանք: Այս ճանապարհին կարևոր դեր ունեն թարգմանության, թեստավորման և գործարկման փուլերը: Դրանք ապահովում են, որ ծրագիրը լինի արդյունավետ, անսխալ և պատրաստ օգտագործման: Այս թեմայի շրջանակում մենք կուսումնասիրենք այդ կարևոր փուլերը՝ հասկանալու համար, թե ինչպես է գաղափարը դառնում գործնական ու կիրառելի տեխնոլոգիական լուծում:

Ուսումնասիրելով այս թեման՝

- Կբացատրեք բարձր մակարդակի ծրագրավորման լեզվով գրված ծրագրերի թարգմանության էությունը:
- Կհասկանաք, թե ինչու է անհրաժեշտ ծրագրի թեստավորում տարբեր փուլերում:
- Կիմանաք, թե մշակման ինչ փուլերով է անցնում ծրագիրը՝ մինչ վերջնական արտադրանք դառնալը:

ԴԱՍ 17

ԾՐԱԳՐԵՐԻ ՄՇԱԿՄԱՆ ԳՈՐԾԸՆԹԱՑԸ

Ծրագրերի մշակման գործընթացը կարևոր փուլ է, որը բաղկացած է մի շարք քայլից՝ սկսած գաղափարի ձևակերպումից մինչև պատրաստի ծրագրի թողարկումը: Ծրագրավորումը բարդ գործընթաց է, բայց այն կարելի է հեշտացնել հստակ պլանավորված քայլերի օգնությամբ: Եկե՛ք պարզենք, թե ինչպես պլանավորել ծրագրերի մշակման գործընթացը:

1. Ծրագրի գաղափար և նպատակ

Ծրագրի մշակումը սկսվում է գաղափարի և նպատակների սահմանումից: Պետք է հստակ պատկերացնել, թե ինչ խնդիր է լուծելու ծրագիրը, և ում համար է այն նախատեսված: Օրինակ՝ եթե ստեղծում ենք խաղ, ապա պետք է իմանալ՝ ինչ ժանրի

խաղ է լինելու, ինչպիսի կերպարներ և կանոններ է ունենալու: Սա ծրագրի պլանավորման առաջին քայլն է, որը պետք է լինի հստակ ձևակերպված:

Օրինակ՝ կազմենք հետևյալ ծրագրի մշակման գործընթացը:

Գրել ծրագիր, որը կհաշվի և կարտածի աշակերտների գնահատականների միջին թվաբանականը:

Ծրագրի գաղափարը և նպատակը: Այս ծրագիրը նպատակ ունի արագ և հստակ հաշվարկելու աշակերտի միջին գնահատականը: Օգտագործողը պետք է ներմուծի գնահատականները, և ծրագիրը կհաշվի դրանց միջին թվաբանականը:

2. Ծրագրի նախագծում

Ծրագրի պլանավորումն ավարտելուց հետո անհրաժեշտ է կազմել ծրագրի նախագիծը: Այս փուլում մշակվում է ծրագրի հիմնական կառուցվածքը և տրամաբանությունը, որոշվում է, թե ինչպես է ծրագիրն օգտագործողը փոխազդելու ծրագրի հետ:

Կազմենք մեր ծրագրի նախագիծը.

- Օգտագործողը պետք է ներմուծի գնահատականների քանակը:
- Այնուհետև ներմուծի յուրաքանչյուր գնահատականը:
- Ծրագիրը կգումարի բոլոր գնահատականները և կբաժանի քանակին՝ միջին թվաբանականը ստանալու համար:

Այս քայլերի հիման վրա հստակեցնում ենք ծրագրի կառուցվածքը և հիմնական գործառույթները:

3. Ծրագրավորման լեզուներ և կոդավորում

Ծրագրերը գրվում են տարբեր ծրագրավորման լեզուներով, որոնք կարելի է բաժանել երկու մակարդակի՝ ցածր և բարձր: Բարձր մակարդակի ծրագրավորման լեզուները, ինչպիսիք են Python–ը, Java–ն և C++–ը, թույլ են տալիս ծրագրավորողներին գրել կոդ, որը հեշտ է կարդալ և հասկանալ: Ընտրելով ճիշտ լեզուն՝ սկսվում է ծրագրի կոդավորման փուլը:

Խնդրի կոդը C++ ծրագրավորման լեզվով կունենա հետևյալ տեսքը.

```cpp
#include <iostream>
using namespace std;
int main()
{
    int count, i; // Աշակերտների գնահատականների քանակը
    double grade, sum, average;
    sum = 0;
    cout << "Ներմուծեք գնահատականների քանակը`  ";
    cin >> count;
    // Մուտքագրում ենք յուրաքանչյուր գնահատականը և գումարում
    for (i = 1; i <= count; i++)
    {
        cout << "Ներմուծեք աշակերտի " << i << "-րդ գնահատականը ";
        cin >> grade;
        if(grade >0 and grade <=10)
        {
        sum = sum + grade;
        }
        else
        {
        cout<<"Այդպիսի գնահատական գոյություն չունի"<< endl;
        break;
        }
    }
    average = sum / count; //Հաշվում ենք միջին թվաբանականը:
    cout << "Միջին գնահատականը հավասար է` " << average << endl;
}
```

```
input
Ներմուծեք գնահատականների քանակը`  3
Ներմուծեք աշակեր՝ 1-րդ գնահատականը 8
Ներմուծեք աշակեր՝ 2-րդ գնահատականը 9
Ներմուծեք աշակեր՝ 3-րդ գնահատականը 6
Միջին գնահատակա՝ հավասար է` 7.66667
```

4. Կոդի թարգմանում

Բարձր մակարդակի լեզվով գրված կոդը չի կարող անմիջապես գործարկվել համակարգչում, քանի որ այն դեռ պետք է վերածել մեքենայական լեզվի, որը կհասկանա համակարգիչը: Դրա համար կոդը պետք է թարգմանել, որը կատարվում է հատուկ ծրագրերի՝ կոմպիլյատորների և ինտերպրետատորների միջոցով: Կոմպիլյատորը փոխակերպում է ամբողջ կոդը մեքենայական լեզվի, իսկ ինտերպրետատորը կարդում և գործարկում է կոդը տող առ տող:

Խնդրի կոդի թարգմանում իրականացվում է գործարկման արդյունքում:

Եթե ծրագիրը պահանջում է տվյալների մուտքագրում, օրինակ՝ գնահատականների քանակը և գնահատականները, ապա տվյալների ներմուծման համար նախատեսված տիրույթում ներմուծում ենք տվյալները:

5. Ծրագրի թեստավորում

Ծրագիրը թարգմանելուց հետո պետք է համոզվել, որ այն ճիշտ է աշխատում: Թեստավորումը այս փուլում շատ կարևոր է, քանի որ դրանով ստուգվում է ծրագրի աշխատանքը սխալները գտնելու և շտկելու նպատակով: Սովորաբար ծրագրավորողները գրում են հատուկ թեստեր, որոնք ստուգում են ծրագրի տարբեր հատվածները՝ համոզվելու համար, որ դրանք ճիշտ են աշխատում: Օրինակ՝ խաղի դեպքում կարելի է ստուգել՝ արդյոք կերպարը ճիշտ է շարժվում, և արդյոք խաղի կանոնները ճիշտ են գործում:

Խնդրի թեստավորումը: Ծրագրում առկա սխալները ցուցադրվում են սխալի մասին հաղորդագրություններով, որոնք օգնում են գտնելու և շտկելու դրանք: Սխալների բացակայության դեպքում արտածվում է ծրագրի արդյունքը:

Ծրագիրը գործարկելուց հետո այն թեստավորում ենք տարբեր գնահատականներով:

Օրինակ՝

- Մուտքագրում ենք 3 գնահատական՝ 5, 7, 6: Ծրագիրը պետք է վերադարձնի միջին գնահատականը՝ 6:
- Մուտքագրում ենք 5 գնահատական՝ 9, 10, 8, 9, 9: Ծրագիրը պետք է վերադարձնի միջին գնահատականը՝ 9:

6. Ծրագրի թողարկում

Երբ թեստավորումն ավարտված է, և սխալները շտկված են, ծրագիրը պատրաստ է թողարկման, որը նշանակում է, որ ծրագիրը պատրաստ է օգտագործման համար: Այս փուլում անհրաժեշտ է հետևել ծրագրի աշխատանքին և հետադարձ կապի արդյունքում բարելավել այն:

Խնդրի թողարկում: Վերջին փուլում, երբ վստահ ենք, որ ծրագիրը ճիշտ է աշխատում, այն պատրաստ է օգտատերերի կողմից օգտագործման:

Այսպիսով՝ ծրագրերի մշակման այս փուլերն օգնում են ստանալու որակյալ ծրագիր՝ առանց սխալների: Պլանավորումը, նախագծումը, կոդավորումը, թարգմանումը, թեստավորումը և թողարկումը միասին ծրագրերի մշակման գործընթացն են: Երբ ծրագրավորողը հետևում է այս քայլերին, նա ստեղծում է բարձրորակ ծրագիր, որը ծառայում է իր նպատակին և օգտատերերին տրամադրում է սպասված արդյունքները:

Ամփոփում

Այս թեման հնարավորություն տվեց հասկանալ, որ ծրագրավորումը միայն ստեղծագործական գործընթաց չէ, այլ նաև տրամաբանական փուլերով կառուցված տեխնիկական ընթացք: Ծրագրերի մշակման գործընթացը բաղկացած է մի շարք փոխկապակցված փուլերից, որոնցից յուրաքանչյուրը կարևոր դեր ունի վերջնական արդյունքի վրա: Թարգմանությունը թույլ է տալիս, որ համակարգիչը «հասկանա» մեր գրված կոդը, թեստավորումը հայտնաբերում և ուղղում է սխալները, իսկ գործարկումը ծրագրին «կյանք» է տալիս՝ այն հասանելի դարձնելով օգտագործողներին: Այս թեմայի ուսումնառությունը մեզ սովորեցրեց ծրագրավորման հիմքային գործընթացները՝ օգնելով պատկերացնել ծրագրի ամբողջ ճանապարհը՝ գաղափարից մինչև պատրաստի արտադրանք:

ՀԱՐՑԵՐ ԵՎ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

1. Ի՞նչ փուլեր է ընդգրկում ծրագրի մշակման գործընթացը:
2. Ի՞նչ կարևորություն ունի ծրագրի պահանջների վերլուծությունը:
3. Ինչո՞ւ է կարևոր ծրագրի թեստավորումը, և ինչպե՞ս է նպաստում սխալների նվազեցմանը:
4. Ինչո՞ւ է կարևոր նախ պլանավորել ալգորիթմը, հետո սկսել կոդավորումը:
5. Հնարավո՞ր է արդյոք ծրագիր մշակել առանց նախագծման: Եթե այո՛, ապա ի՞նչ դժվարությունների կհանդիպեք:
6. Կազմակերպել բանավեճ հետևյալ թեմայով. «Արդյոք ծրագրավորողը պատասխանատվություն է կրում իր ստեղծած կոդի հետևանքների համար»:

Գործնական աշխատանք

Կիրառելով ծրագրի մշակման գործընթացի փուլերը՝ գրել ծրագիր:

1. *Գրել ծրագիր, որը լուծում է քառակուսի հավասարումներ:*

Օրինակ՝ մուտքագրում ենք a, b, c արժեքները, և ծրագիրը գտնում է հավասարման լուծումները կամ արտածում, որ լուծումներ չկան:

2. *Գրել ծրագիր, որը օգտատիրոջից հարցնում է տարիքը և հաշվում, թե քանի օր է նա ապրել:*

3. *Գրել ծրագիր, որը հաշվում է դասարանի սովորողների միջին գնահատականները որևէ առարկայից և ցուցադրում լավագույն արդյունքներն ունեցող սովորողի անունը:*

4. *Ուսումնասիրել տրված ծրագրային կոդը: Բացատրե՛ք, թե ինչ խնդիր է այն լուծում: Թվարկե՛ք և նկարագրե՛ք տրված ծրագրի մշակման գործընթացի փուլերը:*

```
#include <iostream>
using namespace std;
int main()
{
    int a,b,c,s,n;
    cout<<"Մուտքագրեք որևէ եռանիշ թիվ"<<endl;
    cin>>n;
    a=n/100;
    b=(n-100*a)/10;
    c=n%10;
    s=a+b+c;
   cout<<a<<"    "<<b<<"     "<<c<<endl;
   cout<<s;
}
```

ԼՐԱՑՈՒՑԻՉ ՆՅՈՒԹ

Գրադարանները C++–ում

Ծրագրավորման գործընթացն արագացնելու, պարզեցնելու, կոդի հաճախ կիրառվող հատվածները բազմակի օգտագործելու հնարավորություններ ունենալու համար ստեղծվել են գրադարաններ:

Գրադարանները հատուկ ձևով ստեղծված ֆայլեր են, որոնք պարունակում են նախապես մշակված կոդեր և ֆունկցիաներ: Գրադարանները թույլ են տալիս ծրագրավորողներին օգտագործել արդեն իսկ լուծված խնդիրների լուծումները՝ դրանով իսկ տալով հնարավորություն խնայելու ժամանակ, ունենալու առավել արդյունավետ և հակիրճ ծրագրային կոդ:

Գրադարանները կարող են լինել ստանդարտ կամ օգտատիրոջ կողմից ստեղծված:

1. **Ստանդարտ գրադարանները սովորաբար** ներառում են մաթեմատիկական ֆունկցիաներ, ալգորիթմներ, ֆայլերի կառավարում և այլն: Օրինակ՝ C++–ում <iostream> գրադարանը ապահովում է ներմուծման և արտածման հիմնական գոր–ծառույթները, ինչպիսիք են cin–ը և cout–ը:
2. **Օգտատիրոջ գրադարանները** ստեղծվում են անմիջապես օգտատիրոջ կողմից, նրա ծրագրերի կամ նախագծերի պահանջները բավարարելու համար:

Դիտարկենք C++–ի **<cmath>** ստանդարտ գրադարանը**, որը նախատեսված է մաթեմատիկական հաշվարկներ իրականացնելու համար:**

Գրադարանները օգտագործելուց առաջ անհրաժեշտ է դրանք ավելացնել ծրագրային կոդում: Դա արվում է հետևյալ կերպ.

```
#include  <գրադարանի անունը>
```

<cmath> գրադարանի մի քանի ֆունկցիաներ

sqrt (x) — վերադարձնում է x արգումենտի քառակուսի արմատը:

Օրինակ՝

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int x = 25;
    cout << sqrt(x); // Կարտածվի 5 արժեքը
}
```

pow (x, y) — վերադարձնում է x արգումենտի y աստիճանը

Օրինակ՝

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int x = 8, y = 2;
    cout << pow( x, y ); // Կարտածվի 64 արժեքը
}
```

fabs (x) — վերադարձնում է իրական տիպի x արգումենտի բացարձակ արժեքը

Օրինակ՝

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double x = –23.4;
    cout << fabs ( x ); // Կարտածվի 23.4 արժեքը
}
```

ceil (x) — վերադարձնում է x արգումենտից մեծ և նրան մոտ ամբողջ թիվը

Օրինակ՝

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double x = 7.25;
    cout << fabs ( x ); // Կարտածվի 8 արժեքը
}
```

floor (x) — վերադարձնում է x արգումենտը չգերազանցող և նրան մոտ ամբողջ թիվը

Օրինակ՝

```
#include <iostream>
#include <cmath>
using namespace std;
```

```
int main() {
    double x = 7.85;
    cout << floor ( x ); // Կարտածվի 7 արժեքը
}
```

fmod (x, y) — վերադարձնում է իրական տիպի x արգումենտը իրական տիպի y արգումենտի վրա բաժանման արդունքում ստացված մնացորդը:

Օրինակ՝

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double x = 7.85, y = 3.0;
    cout << fmod ( x, y ); // Կարտածվի 1.85 արժեքը
}
```

ՕԳՏԱԿԱՐ ՀՂՈՒՄՆԵՐ

1. Տրամաբանական շղթաներ՝

2. Տվյալների ֆիլտրում՝

3. Զանգվածներ.

4. Բազմաչափ զանգվածներ.

5. Ֆունկցիաներ.

6. Ռեկուրսիա.

7. Դասեր.

8. Ծրագրավորման կրթական հարթակ Profound՝

9. Ծրագրավորման կրթական հարթակ Sololearn՝

10. GDB առցանց կոմպիլյատոր՝

11. Programiz առցանց կոմպիլյատոր՝

Օտարալեզու առանցքային հասկացությունների և եզրույթների բացատրական բառարան

Apple Pay — Apple ընկերության վճարային համակարգ

Closed–source software — փակ կոդով ծրագիր, որի սկզբնական կոդը հասանելի չէ հանրությանը

Creative Commons (CC) — հեղինակային իրավունքների ազատ արտոնագիր, որը թույլ է տալիս որոշակի պայմաններով տարածել բովանդակություն

CVV կոդ — քարտի հակառակ կողմում նշված 3 կամ 4 նիշանոց անվտանգային կոդ

Filter — զտիչ՝ որոշակի պայմաններին բավարարող տվյալները դուրս բերելու համար

GNU General Public License (GPL) — ազատ ծրագրային արտոնագիր, որը թույլ է տալիս ծրագրի ազատ օգտագործում, փոփոխում ու տարածում

Google Live Transcribe — տրամադրում է խոսակցությունների ենթագրեր՝ հեշտացնելով լսողության խնդիրներ ունեցող մարդկանց հաղորդակցումը

Google Pay — Google–ի վճարային համակարգ

MIT License / Apache License — բաց կոդով ծրագրերի արտոնագրեր՝ լայն օգտագործման իրավունքներով

NVDA (NonVisual Desktop Access) — տեսողության խնդիր ունեցողների համար նախատեսված խոսող ծրագրային միջոց

Payoneer — ֆինանսական միջազգային թվային հարթակ

PayPal — ֆինանսական միջազգային թվային հարթակ

Ping — ցանցային հրահանգ՝ սերվերի արձագանքման ժամանակը ստուգելու համար

Revolut — ֆինանսական միջազգային թվային հարթակ

Software — ծրագրային ապահովում

Tobii Dynavox — Ֆիզիկական սահմանափակումներով անձանց համար նախատեսված միջոց. հայացքին հետևող տեխնոլոգիա

Tracert — ցանցում տվյալների փոխանցման երթուղին դիտելու հրաման

Հայերեն տերմիններ և ծրագրավորման հասկացություններ

Ազատ օգտագործման ծրագրեր (Freeware) — փակ կոդով ազատ, անվճար օգտագործման ծրագրեր

Ազատ/վճարովի (Freemium) — ծրագրեր, որոնք առաջարկում են անվճար հիմնական հնարավորություններ, իսկ առավել բարդ գործառույթները՝ վճարովի

Ալգորիթմների արագագործություն — ալգորիթմի կատարման արագություն

Ամբողջական գումարիչ — թվային սարք, որն իրականացնում է երկու բիթերի գումարում՝ հաշվի առնելով գումարման նախորդ կարգից ստացված փոխանցված արժեքը

Բազային դեպք — ռեկուրսիայի դադարեցման պայման

Բանկային քարտ — ֆինանսական գործարքների համար նախատեսված քարտ

Բաց կոդով արտոնագիր (Open–Source License) — արտոնագիր, որը թույլ է տալիս ծրագրային կոդի ազատ հասանելիություն և փոփոխում

Գծային որոնում — տվյալների որոնման պարզ մեթոդ

Դասեր — օբյեկտ–կողմնորոշված ծրագրավորման կառուցվածքներ

Երկչափ զանգված — տվյալների կառուցվածք, երբ տվյալները ներկայացվում են տողերով և սյուներով

Թարգմանական սխալ — սխալ, որն առաջանում է կոմպիլյատորների կողմից ծրագրի թարգմանության ընթացքում

Թվային կենսագրություն — մարդու առցանց վարքագծից մնացած տեղեկատվական հետք

Ժառանգում — օբյեկտ–կողմնորոշված ծրագրավորման հատկություն, որով դասը ժառանգում է մյուս դասի հատկությունները

Ծրագրային ապահովման արտոնագիր — պայմանագիր, որը սահմանում է ծրագրի օգտագործման կանոնները

Ծրագրային սխալ — սխալ, որը խոչընդոտում է ծրագրի ճիշտ աշխատանքին

Ծրագրի թեստավորում — ծրագրի ստուգում՝ սխալները գտնելու համար

Ծրագրի թողարկում — ծրագրի վերջնական տարբերակի հրապարակում

Կատարման ընթացքի սխալ — սխալ, որը տեղի է ունենում ծրագրի աշխատանքի ընթացքում

Կիսագումարիչ — տրամաբանական բարդ շղթա, որն օգտագործվում է երկու երկուական թվերի գումարման համար

Կրիպտոարժույթ — թվային կամ վիրտուալ արժույթ

Հայտարարման սխալ — սխալ, երբ փոփոխականը կամ ֆունկցիան ճիշտ չի հայտարարվել

Հեղինակային իրավունք — ստեղծագործության օգտագործման իրավական պաշտպանություն

Հեռավար և հիբրիդ ուսուցում — ուսուցման ձևեր՝ առցանց կամ խառը ձևով

Մեթոդներ — ծրագրավորման լեզվով գրված գործողություններ, որոնք իրակա–նացվում են դասի միջոցով

Միաչափ զանգված — նույնատիպ տվյալների շարք

Շարահյուսական սխալ — սխալ, որը տեղի է ունենում կոդի շարահյուսության խախտման պատճառով

Պարզ տրամաբանական փական — տրամաբանական միավոր, որն իրականաց–նում է մեկ գործողություն

Պղպջակի մեթոդ — տեսակավորման ալգորիթմ

Ռեկուրսիա — ֆունկցիայի ինքն իրեն կանչելու մեթոդ

Ռեկուրսիվ կանչ — ռեկուրսիայի կոնկրետ իրականացում

Տեսակավորման ալգորիթմներ — տվյալների դասավորման մեթոդներ

Տիպային սխալ — սխալ, երբ օգտագործվում է սխալ տիպի տվյալ

Տվյալների զտում — միայն անհրաժեշտ տվյալների դուրսբերում

Տրամաբանական սխալ — սխալ, երբ ծրագիրը չի վերադարձնում սպասվող արդյունք

Տրամաբանական փականներ — էլեկտրոնիկայում և ծրագրավորման մեջ օգտա–գործվող պայմաններ

Ցանցի արդյունավետություն — ցանցի օգտագործման արդյունավետության չա–փանիշ

Ցանցի թողունակություն — մեկ վայրկյանում ցանցում փոխանցվող տվյալների քանակ

Ցանցի ծանրաբեռնվածություն — տվյալ պահին ցանցի ծանրաբեռնվածության աստիճանը

Ցանցի կապի հեռավորություն — տվյալ ուղարկողի և ստացողի ֆիզիկական կամ տրամաբանական հեռավորություն

Ցանցի հապաղում — ցանցում տվյալների փոխանցման ժամանակահատված

Փակ արտոնագիր (Proprietary Software License) — արտոնագիր, որը առաջաց–նում է ամբողջական վերահսկողություն ծրագրի տարածման և օգտագործ–ման վրա

Ֆունկցիա — կոդի հատուկ ձևով սահմանված հատված, որը կատարում է որոշակի գործողություն

ԲՈՎԱՆԴԱԿՈՒԹՅՈՒՆ

ՆԱԽԱԲԱՆ 3

ԹԵՄԱ 1. ԷԼԵԿՏՐՈՆԱՅԻՆ ԱՊԱՀՈՎՈՒԹՅՈՒՆ 5
ԴԱՍ 1. ԱՌՑԱՆՑ ՎՃԱՐԱՅԻՆ ՀԱՄԱԿԱՐԳԵՐ 5

ԹԵՄԱ 2. ԷԼԵԿՏՐՈՆԱՅԻՆ ՀԱՂՈՐԴԱԿՑՈՒԹՅՈՒՆ 12
ԴԱՍ 2 . ԷԼԵԿՏՐՈՆԱՅԻՆ ՀԱՂՈՐԴԱԿՑՈՒԹՅՈՒՆ: ԹՎԱՅԻՆ ԿԵՆՍԱԳՐՈՒԹՅԱՆ ՁԵՎԱՎՈՐՈՒՄ 12

ԹԵՄԱ 3. ՏԵՂԵԿԱՏՎԱԿԱՆ ՏԵԽՆՈԼՈԳԻԱՆԵՐԸ ԵՎ ՀԱՍԱՐԱԿՈՒԹՅՈՒՆԸ 18
ԴԱՍ 3. ԾՐԱԳՐԱՅԻՆ ԱՐՏՈՆԱԳՐԵՐ ԵՎ ԴՐԱՆՑ ԱԶԴԵՑՈՒԹՅՈՒՆԸ ՀԱՍԱՐԱԿՈՒԹՅԱՆ ՎՐԱ 18

ԹԵՄԱ 4. ՆԵՐԱՌԱԿԱՆՈՒԹՅՈՒՆՆ ԱՊԱՀՈՎՈՂ ՏԵԽՆՈԼՈԳԻԱՆԵՐ 29
ԴԱՍ 4. ՆԵՐԱՌԱԿԱՆՈՒԹՅՈՒՆՆ ՈՒ ՏԵԽՆՈԼՈԳԻԱՆ 29

ԹԵՄԱ 5. ԲԱՐԴ ՏՐԱՄԱԲԱՆԱԿԱՆ ՇՂԹԱՆԵՐ 34
ԴԱՍ 5. ԿԻՍԱԳՈՒՄԱՐԻՉ ԵՎ ԱՄԲՈՂՋԱԿԱՆ ԳՈՒՄԱՐԻՉ 34

ԹԵՄԱ 6. ՑԱՆՑԻ ՀԱՂՈՐԴԱԿՑՄԱՆ ՀԻՄՈՒՆՔՆԵՐ 42
ԴԱՍ 6. ՑԱՆՑԻ ԱՐԴՅՈՒՆԱՎԵՏՈՒԹՅՈՒՆ 42

ԹԵՄԱ 7. ԲԱՐԴ ՏՎՅԱԼՆԵՐ, ՊԱՐԶ ՆԵՐԿԱՅԱՑՈՒՄ 50
ԴԱՍ 7. ՄԵԾԱԾԱՎԱԼ ՏՎՅԱԼՆԵՐՆ ԱՂՅՈՒՍԱԿՈՒՄ 50

ԹԵՄԱ 8. ՄԻԱՉԱՓ ԵՎ ԵՐԿՉԱՓ ՁԱՆԳՎԱԾՆԵՐ 61
ԴԱՍ 8. ՄԻԱՉԱՓ ԵՎ ԵՐԿՉԱՓ ՁԱՆԳՎԱԾՆԵՐ 61

ԹԵՄԱ 9. ՏԵՍԱԿԱՎՈՐՄԱՆ ԵՎ ՓՆՏՐՄԱՆ ԱԼԳՈՐԻԹՄՆԵՐ 69
ԴԱՍ 9. ՏԵՍԱԿԱՎՈՐՄԱՆ ԵՎ ՓՆՏՐՄԱՆ ԱԼԳՈՐԻԹՄՆԵՐ 69

ԹԵՄԱ 10. ՖՈՒՆԿՑԻԱՆԵՐ ԵՎ ԸՆԹԱՑԱԿԱՐԳԵՐ 76
ԴԱՍ 10. ՖՈՒՆԿՑԻԱՆԵՐ 76

ԹԵՄԱ 11. ՌԵԿՈՒՐՍԻԱ 85
ԴԱՍ 11. ՌԵԿՈՒՐՍԻԱՅԻ ԳԱՂԱՓԱՐԸ: ՌԵԿՈՒՐՍԻԱՆ ԲՆՈՒԹՅԱՆ ՄԵՋ 85
ԴԱՍ 12. ՌԵԿՈՒՐՍԻՎ ԱԼԳՈՐԻԹՄՆԵՐԻ ԾՐԱԳՐԱՅԻՆ ԻՐԱԿԱՆԱՑՈՒՄԸ 88

ԹԵՄԱ 12. ԴԱՍԵՐ 91
ԴԱՍ 13. ՕԲՅԵԿՏ–ԿՈՂՄՆՈՐՈՇՎԱԾ ԾՐԱԳՐԱՎՈՐՈՒՄ: ԴԱՍԵՐ 91
ԴԱՍ 14. ԺԱՌԱՆԳՄԱՆ ԳԱՂԱՓԱՐԸ 97

ԹԵՄԱ 13. ՍԽԱԼՆԵՐ ԵՎ ԴՐԱՆՑ ՏԵՍԱԿՆԵՐԸ 101
ԴԱՍ 15. ՍԽԱԼՆԵՐ ԵՎ ԴՐԱՆՑ ՏԵՍԱԿՆԵՐԸ 101

ԹԵՄԱ 14. ԱԼԳՈՐԻԹՄՆԵՐԻ ԱՐԱԳԱԳՈՐԾՈՒԹՅՈՒՆ 109
ԴԱՍ 16. ԱԼԳՈՐԻԹՄՆԵՐԻ ԱՐԱԳԱԳՈՐԾՈՒԹՅՈՒՆԸ 109

ԹԵՄԱ 15. ԾՐԱԳՐԵՐԻ ՄՇԱԿՄԱՆ ԳՈՐԾԸՆԹԱՑԸ 112
ԴԱՍ 17. ԾՐԱԳՐԵՐԻ ՄՇԱԿՄԱՆ ԳՈՐԾԸՆԹԱՑԸ 112

ԼՐԱՑՈՒՑԻՉ ՆՅՈՒԹ 119

ՕԳՏԱԿԱՐ ՀՂՈՒՄՆԵՐ 122

Օտարալեզու առանցքային հասկացությունների և եզրույթների բացատրական բառարան 123

Հայերեն տերմիններ և ծրագրավորման հասկացություններ 124

ՌՈՒԶԱՆՆԱ ՍՏԵՓԱՆՅԱՆ, ԱՆՈՒՀ ԹՈՐՈՍՅԱՆ, ՍԱՄՎԵԼ ՄԱՐՏԻՐՈՍՅԱՆ,
ՀՈՒՀԱՆԻԿ ԱՄԻՐԽԱՆՅԱՆ, ԼՈՒՍԻՆԵ ՄԱՏԻՆՅԱՆ

ԹՎԱՅԻՆ ԳՐԱԳԻՏՈՒԹՅՈՒՆ ԵՎ ՀԱՍԱԿԱՐԳՉԱՅԻՆ ԳԻՏՈՒԹՅՈՒՆ

11

Հանրակրթական ավագ դպրոցի
11–րդ դասարանի դասագիրք

Հրատարակչության տնօրեն՝ Էմին Մկրտչյան
Տեխնիկական խմբագիր՝ Նվարդ Փարսադանյան
Խմբագիր և սրբագրիչ՝ Նվարդ Փարսադանյան
Մակետավորումն ու ձևավորումը՝ Գոհար Գրիգորյանի
Շապիկի ձևավորումը՝ Գոհար Գրիգորյանի

Օգտագործված նկարներ՝
https://www.freepik.com, «Զանգակ» հրատարակչության արխիվ

Տպագրությունը՝ օֆսեթ: Չափսը՝ 70 x 100 1/16
Թուղթը՝ օֆսեթ: Ծավալը՝ 8 տպ. մամուլ
Հիմնական տառատեսակը՝ GHEA Granshan

ԶԱՆԳԱԿ
ՀՐԱՏԱՐԱԿՉՈՒԹՅՈՒՆ

ՀՀ, 0051, Երևան, Կոմիտասի պող. 49/2, հեռ.՝ +37410 23 25 28
Էլ. փոստ՝ info@zangak.am, էլ. կայքեր՝ www.zangak.am, www.book.am
Ֆեյսբուքյան կայքէջ՝ www.facebook.com/zangak